



RAPPORT DE STAGE DE FIN D'ÉTUDES

Développeur d'outils d'automatisation

Projet SDV

Power Activation & Distribution — S53

Enzo CHERIF

Double diplôme SEATECH INP TOULON & Master RISE — Université de Toulon

Période : 01/03/2025 — 31/08/2025

Tuteur École

Cédric ANTHIERENS

cedric.anthierens@univ-tln.fr

Tuteur Entreprise

Baptiste ROMIEU

baptiste.romieu@ampere.cars

Engagement de non plagiat.

Je soussigné, ...Enzo CHERIF.....

N° carte d'étudiant : ...22206913.....

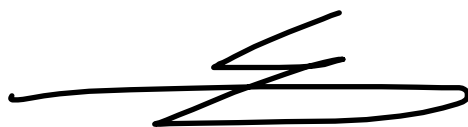
Déclare avoir pris connaissance de la charte des examens et notamment du paragraphe spécifique au plagiat.

Je suis pleinement conscient(e) que la copie intégrale sans citation ni référence de documents ou d'une partie de document publiés sous quelques formes que ce soit (ouvrages, publications, rapports d'étudiants, internet, etc....) est un plagiat et constitue une violation des droits d'auteur ainsi qu'une fraude caractérisée.

En conséquence, je m'engage à citer toutes les sources que j'ai utilisées pour produire et écrire ce document.

Fait le 17/08/2025
.....

Signature(s)



Ce document doit être inséré en première page de tous les rapports, dossiers et/ou mémoires.

Document du chapitre 10 annexe 5, issu de la Charte des examens adoptée en Conseil d'Administration le 11 juillet 2013 après avis du CEVU du 27 juin 2013 - Délibération N°2013-73 – Modifié suite au CFVU du 12/03/2015.

Remerciements

Ce **stage de fin d'études** a marqué une **étape déterminante** dans mon parcours. Au-delà de l'apprentissage technique, il m'a appris à apporter une **valeur immédiate** à une équipe d'ingénierie et à transformer des besoins parfois flous en outils concrets et utilisés. Il a confirmé que le croisement **automobile/logiciel** est le terrain où je suis le plus utile et où je souhaite poursuivre ma trajectoire professionnelle.

Je souhaite remercier tout particulièrement **Baptiste Romieu**, mon tuteur. Plus qu'un encadrant, il a été un **repère quotidien** : toujours **disponible**, pleinement **confiant**, et d'une **exigence** aussi juste qu'utile. Son **expertise** du métier, sa capacité à **vulgariser** des sujets complexes et à **prendre le temps** d'expliquer ont été déterminantes pour ma progression. Grâce à ses retours précis, ses **revues de code structurantes** et ses conseils d'architecture, j'ai pu livrer vite sans sacrifier la qualité.

Au-delà de l'aspect technique, l'**entente humaine** a compté : Baptiste a su créer un cadre à la fois **bienveillant** et **stimulant**, qui m'a donné envie de me dépasser et de « faire bien pour lui » comme pour l'équipe. Il a trouvé l'équilibre entre **autonomie** et **guidage**, m'a **challenge** au bon moment et m'a ouvert les bonnes portes. Beaucoup des **réflexes professionnels** que je garderai — clarté des objectifs, rigueur des livrables, sens du service — me viennent de cet accompagnement. Pour tout cela : **merci, Baptiste**.

Mes remerciements vont également à **Sylvain Henry, responsable de stage et CUET**, pour son cadrage clair, ses attentes formulées avec exigence et bienveillance, et son soutien dans les moments clés. Il m'a aidé à structurer mon travail, à prioriser et à garder le cap sur les résultats qui comptent vraiment pour l'équipe.

Je tiens aussi à remercier **Vincent Grecco, AMS de S53**, pour son implication tout au long du stage. Ses sollicitations, ses retours concrets et ses exigences ont été de véritables leviers de progression. Il m'a fait **gagner en maturité**, m'a aidé à prendre conscience de mes axes d'amélioration et m'a donné l'occasion de montrer ma réactivité, notamment sur les **écrans DDT** et le **CCVC**. Sa confiance m'a encouragé à viser juste et à livrer vite.

Une mention particulière à **Karim Senoun — AEES** — qui, bien qu'il ne soit pas mon tuteur, m'a accompagné sur mes **problèmes de code** et choix techniques. Son **œil expert**, sa franchise et ses challenges constants ont été très formatifs, et m'ont permis d'affiner mes décisions d'architecture, de renforcer la robustesse des scripts et de gagner en efficacité. **Merci Karim** pour cette exigence stimulante et cette disponibilité.

Je remercie l'ensemble de l'équipe **S53 — Power Activation & Distribution** pour l'accueil et l'entraide : **Thibaut Cochet, Vincent Selvaraj, Benoît Hanotte**, et tous ceux que je ne cite pas mais qui ont contribué, par une question, un fichier d'exemple, un avis critique ou un test, à faire avancer les outils. Cette **dynamique d'équipe**, à la fois professionnelle et humaine, a énormément facilité mon intégration et accéléré mes progrès.

Merci à **Séverine Rausz (CDS Architecture Transversale)** pour sa **disponibilité** et ses **revues régulières**, qui ont clarifié les priorités et maintenu une vision nette des objectifs.

Enfin, côté académique, merci à **Monsieur Anthierens** pour son accompagnement et la **méthode** transmise — rigueur et curiosité —, essentielle pour tirer le meilleur de ce stage.

À toutes et tous — responsables, tuteurs, collègues, mentors — **merci**. Votre confiance, vos retours et votre exigence ont fait de ce stage une expérience **fondatrice**. Je pars avec le sentiment d'avoir appris, contribué, et trouvé ma place au service d'une équipe et d'un produit.

Résumé et Mots-clés

Abstract

Completed at Ampere (Renault Group), within team S53 “Power Activation & Distribution,” this internship aimed to **automate** the production, sizing, and verification of electrical deliverables around the **SDE** (electrical distribution scheme) and AUTOSAR, in order to **reduce manual errors**, **save time**, and **ground** engineering decisions in evidence. The approach relies primarily on **Python** (Excel/XML), **Tkinter/Flask** interfaces, and **Plotly** visualization, with normalized reference data, systematic validations, and executable packaging. Code and outputs are versioned and shared (SharePoint, GitLab).

Eight complementary modules were delivered and adopted : **Wire Protection Calib** (SDE/APSW cross-check), **PowerSW** (extraction/structuring of supplier calibration parameters), **ScreenBuilder** (automatic DDT screen generation in XML), **ENSOFIT** (central launcher), **Recorder generation**, **SDE Check** (SDE consistency), **CCVC** (Creta/AUTOSAR checks by PIU and by function), and **ASCS** (Automatic Suitable Curve Selection) for **SmartMOS sizing** via a single overlay of current profile, $\pm 10\%$ tolerance band, and software curves, with an “SDE Available” view.

Results are tangible : **standardized** (Excel/CSV/XML/PNG), **traceable**, and **reproducible** deliverables, integrated into the **FlexEvan** flow and actively used in reviews and production. In terms of **time savings**, the most significant is **ScreenBuilder** : automatic generation **saves about 2 days per DDT screen** ; for ~ 30 screens this is ~ 60 person-days (~ 12 weeks on a 5-day basis). **CCVC** also saves several hours per campaign by detecting calibration discrepancies earlier. The overall contribution has been recognized by the team ; **ASCS** is part of an **invention disclosure**, and a **GitLab CI** track is underway for **CCVC** (inputs evolving to XML/JSON).

Beyond the tools, the internship consolidated skills in **advanced Python**, **data-services-interface** architecture, domain **data engineering**, and **software quality**, as well as **technical communication** and **change management**. Clear personal perspectives follow : industrialization (**CI/CD**, testing/monitoring), deeper **AUTOSAR**, and continued roles at the **embedded-tooling-data** interface.

Résumé

Réalisé au sein d'Ampere (Renault Group), au pôle S53 « Power Activation & Distribution », ce stage a eu pour objectif d'**automatiser** la production, le dimensionnement et la vérification des livrables électriques autour du **SDE** (schéma de distribution électrique) et d'AUTOSAR, afin de **réduire les erreurs manuelles**, **gagner du temps** et **objectiver** les décisions techniques. L'approche s'appuie sur un outillage majoritairement **Python** (Excel/XML), des interfaces **Tkinter/Flask** et une visualisation **Plotly**, avec normalisation des référentiels, validations systématiques et packaging exécutable. Le code et les livrables sont versionnés et partagés (SharePoint, GitLab).

Huit briques complémentaires ont été livrées et adoptées : **Wire Protection Calib** (vérification croisée SDE/APSW), **PowerSW** (extraction/structuration des paramètres fournisseur), **ScreenBuilder** (génération automatique des écrans DDT en XML), **ENSOFT** (interface centrale), **génération des Recorder**, **SDE Check** (cohérences SDE), **CCVC** (contrôles Creta/AUTOSAR par PIU et par fonction) et **ASCS** (Automatic Suitable Curve Selection) pour le **dimensionnement SmartMOS** via une superposition unique du profil de courant, du couloir de tolérance $\pm 10\%$ et des courbes software, avec vue « Disponibles SDE ».

Les résultats sont concrets : livrables **standardisés** (Excel/CSV/XML/PNG), traçables et reproductibles, intégrés au flux **FlexEvan** et réellement utilisés en revue et en production. Côté **gains de temps**, le plus marquant concerne **ScreenBuilder** : la génération automatique **économise en moyenne 2 jours par écran DDT** ; sur un lot d'environ **30 écrans**, cela représente ≈ 60 **jours-homme**, soit ≈ 12 **semaines** de charge (base 5 jours) — un saut d'échelle en termes de productivité et d'uniformité. **CCVC** fait également gagner plusieurs heures par campagne tout en détectant plus tôt les écarts de calibration. L'ensemble a été reconnu par l'équipe ; **ASCS** fait l'objet d'une **déclaration de brevet** et une **piste CI GitLab** est engagée pour **CCVC** (évolution des entrées en XML/JSON).

Au-delà des outils, le stage a consolidé des compétences en **Python avancé**, **architecture données-services-interface**, **data engineering** métier et **qualité logicielle**, ainsi qu'en **communication technique** et **conduite du changement**. Il ouvre des **perspectives personnelles** claires : industrialisation (CI/CD, tests/monitoring), approfondissement AUTOSAR et poursuite d'un rôle à l'interface **embarqué – outillage – données**.

Key words

- FlexEVan program (100% electric utility vehicles)
- Software Defined Vehicle (SDV) architecture
- Renault Group – Ampere Software Technology
- SmartMOS (programmable electronic power switches)
- Power Interface Unit (PIU) management
- Electrical Distribution Schema (SDE) validation
- Systems engineering and MBSE (Model-Based Systems Engineering)
- Automotive functional safety (ISO 26262, ASIL levels)
- AUTOSAR standards and cross-verification (CDFX/XML)
- Automotive diagnostics and DDT2000 automation
- Python development (pandas, openpyxl, lxml, Flask, Tkinter, CustomTkinter)
- Automatic generation of DDT2000 screens (Distribution, Shutdown, OEM, Recorder)
- Wire Protection Calibration (i^2t curves and profiles)
- Leoni PowerSW calibration parameter extraction
- EnSoft (centralized graphical user interface)
- Multi-sheet Excel reporting, conditional formatting and KPI tracking
- Data traceability, robustness and reproducibility
- Integration of XML, Excel, CSV workflows
- Human–Machine Interface (HMI) and usability
- Energy flow optimization and electrical system reliability
- Embedded systems and automotive software integration
- Industrialization of engineering processes and tool deployment
- Over-the-Air (OTA) update readiness
- Scalability and maintainability of automotive systems
- Collaboration with system architects, functional leaders and engineers
- Innovation in automation, AI assistance and software engineering

Mots-clés

- Programme FlexEVan (utilitaires 100 % électriques)
- Architecture Software Defined Vehicle (SDV)
- Renault Group – Ampere Software Technology
- SmartMOS (interrupteurs électroniques programmables)
- Power Interface Unit (PIU)
- Validation du Schéma de Distribution Électrique (SDE)
- Ingénierie système et MBSE (Model-Based Systems Engineering)
- Sécurité de fonctionnement automobile (ISO 26262, niveaux ASIL)
- Norme AUTOSAR et vérification croisée (CDFX/XML)
- Diagnostic automobile et automatisation DDT2000
- Développement Python (pandas, openpyxl, lxml, Flask, Tkinter, CustomTkinter)
- Génération automatique d'écrans DDT2000 (Distribution, Shutdown, OEM, Recorder)
- Calibration Wire Protection (i^2t)
- Extraction des paramètres de calibration Leoni PowerSW
- EnSoft (interface graphique centralisée)
- Rapports Excel multi-onglets, mise en forme conditionnelle et suivi KPI
- Traçabilité, robustesse et reproductibilité des données
- Intégration de flux XML, Excel, CSV
- Interfaces homme-machine (IHM) et ergonomie
- Optimisation énergétique et fiabilité des systèmes électriques
- Systèmes embarqués et intégration logicielle automobile
- Industrialisation et déploiement d'outils d'ingénierie
- Préparation aux mises à jour OTA (Over-The-Air)
- Scalabilité et maintenabilité des systèmes automobiles
- Collaboration avec architectes systèmes et ingénieurs
- Innovation en automatisation, IA assistive et ingénierie logicielle

Table des matières

Résumé et Mots-clés	3
1 Introduction	10
1.1 Contexte académique	10
1.1.1 Formation suivie	10
1.1.2 Objectifs pédagogiques du stage	10
1.1.3 Position du stage dans le cursus	11
1.2 Présentation du stage	11
1.2.1 Lieu et durée du stage	11
1.2.2 Structure d'accueil : Ampere Software Technology – Renault Group	12
1.2.3 Tuteur : Baptiste Romieu / Responsable : Sylvain Henry	12
1.3 Motivations personnelles	13
1.3.1 Intérêt pour l'automobile électrique et l'automatisation	13
1.3.2 Objectifs techniques et professionnels	13
2 Présentation de l'entreprise	14
2.1 Renault et Ampere	14
2.1.1 Bref historique de Renault	14
2.1.2 Création et rôle d'Ampere dans le SDV	15
2.1.3 Lien avec les partenaires stratégiques (Google, Qualcomm, Valeo...)	16
2.2 L'entité Ampere	17
2.2.1 Rôle dans l'organisation Ampere	17
2.2.2 Lien entre énergie, activation, et software-defined vehicle	17
2.3 L'équipe S53 – Power Activation & Distribution	18
2.3.1 Missions : wake-up, shut-down, protection électrique	18
2.3.2 SmartMOS : définition, rôle et différence avec les fusibles	19
2.3.3 Outils clés : SDE (Schéma de Distribution Électrique), DDT	20
2.4 Projet FlexEvan	22
2.4.1 Description du véhicule	22
2.4.2 Innovations : SDV, architecture centralisée	23
2.4.3 Implication de S53 dans le projet	24
3 Objectifs et périmètre du stage	24
3.1 Objectif principal	24
3.2 Objectifs techniques secondaires	25
4 Méthodologie et environnement de travail	26
4.1 Fonctionnement de l'équipe et intégration du stagiaire	26
4.1.1 Une organisation agile et structurée	26
4.1.2 Un suivi régulier au cœur de l'organisation	26
4.2 Outils et langages	27
5 Développement des outils et scripts	27
5.1 Outil Wire Protection Calib	28
5.1.1 Contexte et besoin métier – Vérification croisée SDE / APSW	28
5.1.2 Fonctionnement général du script	28
5.1.3 Résultats obtenus	31
5.1.4 Apports, plus-value et perspectives	32

5.2	Outil PowerSW : Extraction des paramètres de calibration Leoni	33
5.2.1	Contexte et besoin métier – Vérification croisée SDE / APSW	33
5.2.2	Fonctionnement général du script	33
5.2.3	Résultats obtenus	35
5.2.4	Apports, plus-value et perspectives	35
5.3	Outil ScreenBuilder : Générateur d'écrans DDT	36
5.3.1	Introduction générale	36
5.3.2	Contexte et besoin métier	37
5.3.3	Fonctionnement global du script	38
5.3.4	Les trois cas métiers traités : Cas Distribution	40
5.3.5	Les trois cas métiers traités : Cas Shutdown	41
5.3.6	Les trois cas métiers traités : Cas OEM	43
5.3.7	Apports, plus-value et perspectives	44
5.4	ENSOFIT : Infrastructure graphique centralisée	45
5.4.1	Objectifs et justification	45
5.4.2	Choix techniques	46
5.4.3	Page d'accueil	47
5.4.4	Module ScreenBuilder	48
5.4.5	Module Wire Protection Calib	49
5.4.6	Module PowerSW	50
5.4.7	Accessibilité et déploiement	50
5.4.8	Design et retour utilisateurs	51
5.5	Génération automatique des écrans Recorder	52
5.5.1	Objectifs et contexte	52
5.5.2	Organisation des Recorders	52
5.5.3	Processus de génération	53
5.5.4	Avantages et perspectives	55
5.6	SDE Check	56
5.6.1	Objectifs et contexte	56
5.6.2	Règles de validation intégrées : Comparaison des courants IN et IP	57
5.6.3	Règles de validation intégrées : Vérification du niveau ASIL	57
5.6.4	Règles de validation intégrées : System Controlling SmartMOS	58
5.6.5	Règles de validation intégrées : Vérifications MBSE Energy Flow	59
5.6.6	Règles de validation intégrées : Vérification Exclusive	59
5.6.7	Règles de validation intégrées : Vérification I (A) Watt Man	60
5.6.8	Fonctionnement technique du script	61
5.6.9	Structure du rapport généré	62
5.6.10	Avantages et perspectives	63
5.7	CCVC - Creta Cross Verification Check	64
5.7.1	Objectifs et contexte	64
5.7.2	Fonctionnement technique de l'outil	64
5.7.3	Résultats obtenus	66
5.7.4	Apports, plus-value et perspectives	68
5.8	ASCS — Automatic Suitable Curve Selection	70
5.8.1	Objectifs et contexte	70
5.8.2	Architecture et choix technologiques	71
5.8.3	Modèle de données et règles	73
5.8.4	Algorithme de classement	74
5.8.5	Parcours utilisateur	75
5.8.6	Livrables et formats	77

5.8.7	Apports, plus-value et ROI	78
5.8.8	Perspectives	79
6	Résultats et impact	80
6.1	Résultats techniques	80
6.2	Bénéfices concrets pour l'équipe	81
6.3	Motivations personnelles	83
6.3.1	Retours positifs des responsables d'équipe	83
6.3.2	Outils réellement utilisés par les collègues	83
6.3.3	Projet intégré au flux de production FlexEvan	84
7	Bilan personnel	84
7.1	Compétences techniques acquises	84
7.1.1	Python avancé	84
7.1.2	Architecture d'outils	85
7.1.3	Web & visualisation	85
7.1.4	Data engineering « métier »	85
7.1.5	Qualité logicielle	85
7.1.6	Data science (pilote)	85
7.2	Compétences humaines et organisationnelles	85
7.2.1	Communication technique	85
7.2.2	Travail en équipe	85
7.2.3	Autonomie & initiative	86
7.2.4	Leadership & pédagogie	86
7.3	Méthodes de travail et bonnes pratiques	86
7.3.1	Cycle de développement court	86
7.3.2	Traçabilité & auditabilité	86
7.3.3	Documentation	86
7.3.4	Standardisation	86
7.4	Difficultés rencontrées et solutions	87
7.4.1	Hétérogénéité des sources	87
7.4.2	Contrainte de temps	87
7.4.3	Robustesse poste-à-poste	87
7.5	Ouverture professionnelle	87
7.5.1	Périmètres ciblés	87
7.5.2	Évolutions envisagées	88
7.5.3	Environnement souhaité	88
7.6	Axes d'amélioration plan d'apprentissage	88
7.6.1	Industrialisation (CI/CD & packaging)	88
7.6.2	Tests & monitoring	88
7.6.3	IA & recommandations (Data)	89
7.6.4	Domaine (AUTOSAR & flux SDE → CDFX → DDT)	89
7.6.5	Stack technique élargie	89
8	Conclusion	90
8.1	Compétences techniques acquises	90
8.2	Compétences humaines et organisationnelles	90
8.3	Méthodes de travail et bonnes pratiques	91
	Fiche tuteur entreprise et Fiche d'évaluation étudiant - Glossaire - Bibliographie	92

1 Introduction

1.1 Contexte académique

1.1.1 Formation suivie

Actuellement en dernière année d'études d'ingénieur, j'effectue un **double cursus** au sein de **SeaTech**, l'école d'ingénieurs de l'**Institut National Polytechnique de Toulon**, et de l'**Université de Toulon**.

À SeaTech, je suis une formation généraliste en mécatronique et systèmes embarqués, qui allie l'électronique, l'informatique industrielle, la robotique et les systèmes de contrôle. Ce parcours m'a permis d'acquérir une solide base en programmation embarquée, modélisation de systèmes complexes, automatisme, et électronique de puissance.

En parallèle, je poursuis un **Master RISE** (*Robotics and Intelligent Systems Engineering*) à l'Université de Toulon. Ce master est orienté vers l'intelligence artificielle, la robotique autonome et l'ingénierie des systèmes intelligents. Il m'a permis de développer des compétences avancées en traitement de données, apprentissage automatique, vision par ordinateur et systèmes distribués.

Ce double diplôme me permet de conjuguer une approche concrète et technologique issue de la formation d'ingénieur, avec une vision plus théorique et algorithmique issue du master, tout en favorisant la pluridisciplinarité, l'autonomie, et l'adaptabilité — des qualités essentielles dans un environnement industriel innovant tel que celui de **Renault Group**.

1.1.2 Objectifs pédagogiques du stage

Ce stage de fin d'études s'inscrit dans le cadre de la validation de mon **double diplôme** d'ingénieur à **SeaTech (INP Toulon)** et du **Master RISE** à l'**Université de Toulon**. Il constitue l'aboutissement de mon parcours académique, en me confrontant aux réalités techniques, humaines et organisationnelles d'un **projet industriel d'envergure**.

L'objectif principal de ce stage est de **mettre en pratique les compétences acquises** au cours de ma formation, à travers la conduite de projets concrets dans un environnement technologique avancé. Il s'agit notamment de :

- Développer des solutions logicielles **robustes, évolutives** et adaptées aux besoins industriels, en m'appuyant sur des langages et outils modernes (Python, Flask, HTML/CSS/JavaScript, etc.) ;
- Participer activement à un **projet d'ingénierie système embarqué**, en lien avec la **mobilité électrique**, la **distribution d'énergie**, et l'**architecture logicielle** des véhicules de nouvelle génération ;
- Approfondir ma maîtrise des **méthodologies de développement**, de **validation**, et de **déploiement**, en environnement collaboratif ;
- Travailler dans une **équipe pluridisciplinaire**, en interagissant avec des ingénieurs spécialisés en électronique, architecture véhicule, qualité logicielle et diagnostic.

Au-delà de l'application technique, ce stage vise également à développer des **compétences transversales** : autonomie, rigueur, communication technique, gestion des priorités, et prise d'initiatives dans un cadre professionnel exigeant. Il constitue une étape déterminante pour ma future insertion dans le monde de l'ingénierie, que ce soit en France ou à l'international.

1.1.3 Position du stage dans le cursus

Ce stage s'inscrit dans le cadre de la dernière année de mon cursus d'ingénieur à **SeaTech (INP Toulon)**, et vient également conclure le parcours du **Master RISE** (*Robotics and Intelligent Systems Engineering*) de l'Université de Toulon. Il constitue un élément central de la formation, à la fois comme validation académique et comme préparation à l'insertion professionnelle.

Ce stage de six mois est obligatoire et fait l'objet d'une évaluation intégrée au diplôme d'ingénieur. Il est positionné en fin de cycle, afin de permettre la mobilisation de l'ensemble des connaissances et compétences acquises durant le cursus, tant en matière de sciences de l'ingénieur que de gestion de projet, de développement logiciel, ou encore de communication technique.

À ce titre, le stage a une fonction charnière : il doit permettre de passer du statut d'étudiant à celui d'ingénieur autonome, capable de prendre en charge des projets complexes, d'interagir avec des équipes pluridisciplinaires, et de s'intégrer dans un environnement industriel exigeant. Il représente également une opportunité de préciser mon projet professionnel, d'explorer de nouvelles technologies, et d'acquérir une expérience significative dans un grand groupe innovant du secteur automobile.

1.2 Présentation du stage

1.2.1 Lieu et durée du stage

Le stage a débuté le **3 mars 2025** et s'est déroulé sur une durée de **six mois à temps plein**, conformément aux exigences des cursus d'ingénieur et de master.

Il a eu lieu au sein du **Renault Group**, plus précisément dans l'entité **Ampere Software Technology** [25], rattachée à la direction de l'ingénierie logicielle dédiée aux véhicules électriques et connectés. Le stage s'est tenu dans les locaux du **Technocentre Renault** de Guyancourt, l'un des plus importants centres de recherche et développement automobile en Europe (voir figure 1).



FIGURE 1 – Le Technocentre Renault à Guyancourt, site du stage.

Le site de Guyancourt constitue un environnement particulièrement stimulant, rassemblant une diversité d'expertises techniques, d'équipes projets et de moyens d'essais. Il offre un cadre

propice à l'innovation, à la collaboration interdisciplinaire et à la mise en œuvre concrète de solutions techniques à grande échelle.

1.2.2 Structure d'accueil : Ampere Software Technology – Renault Group

Le stage s'est déroulé au sein du **Renault Group**, un acteur majeur de l'industrie automobile mondiale, fondé en 1898 et aujourd'hui engagé dans une profonde transformation vers la mobilité durable et connectée. En réponse aux enjeux climatiques, réglementaires et technologiques, Renault a créé **Ampere**, une entité dédiée au développement des véhicules électriques et des logiciels embarqués.

Ampere Software Technology est l'une des branches stratégiques de cette entité. Elle regroupe les expertises nécessaires au développement, à la validation et à l'intégration des logiciels embarqués dans les véhicules 100 % électriques de nouvelle génération. Cette structure a pour mission de concevoir les architectures logicielles qui piloteront les véhicules de demain, à travers une approche dite **SDV – Software Defined Vehicle**.

Le SDV (véhicule défini par le logiciel) [27] représente un changement de paradigme dans l'industrie automobile. Il repose sur une centralisation des fonctions électroniques autour de calculateurs puissants et reprogrammables, capables de faire évoluer les fonctionnalités du véhicule au fil du temps. Dans ce contexte, le logiciel devient un **élément différenciateur clé**, au même titre que le design ou les performances mécaniques.

Mon stage s'est inséré dans l'une des équipes de cette organisation, l'équipe **S53 – Power Activation and Distribution**, qui joue un rôle fondamental dans le **réveil**, l'**endormissement** et la **distribution intelligente de l'énergie** à bord du véhicule. Cette équipe travaille notamment sur le remplacement progressif des fusibles traditionnels par des composants électroniques intelligents appelés **SmartMOS**, dans le cadre de la conception du futur projet utilitaire électrique de Renault : le **FlexEvan**.

1.2.3 Tuteur : Baptiste Romieu / Responsable : Sylvain Henry

Durant ce stage, j'ai été encadré par **Monsieur Baptiste Romieu, Pilote Fonctionnel Système (PFS)** au sein de l'équipe **S53 – Power Activation and Distribution**. En tant que tuteur technique, il m'a accompagné tout au long de mon stage, en me confiant des missions concrètes et formatrices autour du développement d'outils logiciels pour l'automatisation, la validation métier et la génération de livrables. Grâce à son encadrement attentif, j'ai pu monter rapidement en compétence sur les outils internes, comprendre les enjeux du projet, et mener mes développements en autonomie.

Mon responsable de stage était **Monsieur Sylvain Henry, Chef d'Unité d'Entité Technique (CUET)** de l'équipe S53. Il pilote l'activité globale de l'unité en lien avec la stratégie du véhicule électrique et du **Software Defined Vehicle**. Son rôle a été clé pour m'intégrer au projet **FlexEvan**, en m'exposant aux objectifs long terme de l'équipe, aux interactions avec les autres entités d'Ampere, et aux défis liés à l'électrification du parc utilitaire Renault.

Ce tandem m'a permis d'évoluer dans un environnement à la fois technique, opérationnel et stratégique, propice à l'apprentissage et à l'engagement.

1.3 Motivations personnelles

1.3.1 Intérêt pour l'automobile électrique et l'automatisation

Mon attrait pour le domaine de l'automobile électrique s'inscrit dans une démarche de **sens** et de **technologie**. En tant que futur ingénieur, je suis convaincu que la **transition énergétique** représente l'un des défis majeurs de notre époque, et que l'électrification des véhicules en est un **levier stratégique**. Contribuer à cette transformation en intégrant un acteur de référence comme Renault était donc une opportunité motivante, en parfaite cohérence avec mes convictions et mes aspirations professionnelles.

Au-delà de l'aspect énergétique, c'est également la dimension logicielle et intelligente du véhicule qui m'intéresse particulièrement. L'émergence du **Software Defined Vehicle (SDV)**, dans lequel le comportement du véhicule dépend de logiciels évolutifs et reconfigurables, ouvre un nouveau champ d'innovation dans lequel mes compétences en systèmes embarqués, automatisation et développement logiciel trouvent pleinement leur place.

Par ailleurs, j'ai toujours eu une appétence marquée pour l'**automatisation des processus techniques**. Ce stage m'a permis de mettre en œuvre concrètement cette passion : identifier des tâches manuelles fastidieuses, les analyser, puis développer des outils **robustes** et **intuitifs** pour en améliorer la **fiabilité**, la **traçabilité** et la **rapidité**. Ce goût pour l'automatisation ne se limite pas à un gain de productivité : il s'inscrit dans une volonté plus large de créer de la **valeur** pour les équipes, en libérant du temps humain pour des tâches à plus forte valeur ajoutée.

Ainsi, ce stage a parfaitement répondu à mon désir de contribuer à une industrie plus **intelligente**, plus **durable** et plus **efficace**, tout en m'offrant un terrain technique exigeant et formateur.

1.3.2 Objectifs techniques et professionnels

Ce stage de fin d'études constituait pour moi une étape stratégique, à la fois pour approfondir mes **compétences techniques** dans un contexte industriel exigeant, et pour clarifier mon **projet professionnel** en tant qu'ingénieur en systèmes embarqués et logiciels.

Sur le plan technique, mes objectifs étaient multiples :

- **Renforcer mes compétences en développement logiciel**, en particulier en Python, dans un cadre concret et appliqué à des problématiques métiers complexes.
- Approfondir ma maîtrise des **traitements de fichiers hétérogènes** (Excel, XML, texte, PDF) et de leur automatisation via des scripts robustes, documentés et réutilisables.
- Acquérir de l'expérience dans le **développement d'interfaces utilisateur**, aussi bien en environnement desktop (Tkinter, CustomTkinter) qu'en environnement web (Flask, HTML/CSS/JavaScript).
- Participer à la **conception de solutions logicielles intégrées**, s'interfaçant avec les outils existants de l'entreprise (DDT, SDE, environnement de validation), tout en respectant les exigences métier, les normes de qualité, et les besoins des utilisateurs.
- Comprendre l'architecture des **véhicules électriques modernes**, notamment les enjeux liés à la distribution d'énergie, au réveil/à l'endormissement des calculateurs, et à la mise en place des composants intelligents (SmartMOS).

Sur le plan professionnel, ce stage avait pour objectif de :

- Valider mon intérêt pour les **projets complexes à l'intersection entre matériel et logiciel** (hardware/software), typiques des systèmes embarqués.

- Travailler en équipe dans un grand groupe international, structuré, avec une forte **culture de l'ingénierie et de l'innovation**.
- Gagner en **autonomie**, en **rigueur** et en **capacité d'initiative**, à travers la gestion de plusieurs projets techniques en parallèle.
- Me positionner concrètement sur le type de poste que je souhaiterais occuper à l'avenir, que ce soit en ingénierie logicielle embarquée, en automatisation de processus ou en architecture système.

Ce stage a donc été pensé comme un **tremplin vers l'exercice du métier d'ingénieur**, en mobilisant des compétences transverses : techniques, analytiques, mais aussi humaines et organisationnelles.

2 Présentation de l'entreprise

2.1 Renault et Ampere

2.1.1 Bref historique de Renault

Fondé en **1898** par les frères **Louis, Marcel et Fernand Renault**, le **Groupe Renault** est l'un des pionniers de l'industrie automobile mondiale (voir figure 2). Depuis ses débuts, l'entreprise s'est illustrée par sa capacité à innover, que ce soit à travers ses premières voitures de tourisme, ses succès en compétition automobile ou encore ses avancées technologiques majeures.

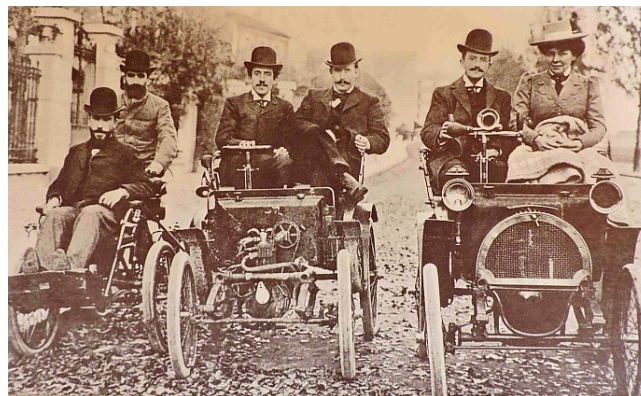


FIGURE 2 – Photographie historique des frères Renault accompagnés des premiers modèles Type A (vers 1900).

Tout au long du **XX^e siècle**, Renault s'est affirmé comme un acteur industriel de premier plan en France et à l'international, en développant une large gamme de véhicules pour particuliers, professionnels et services publics. Nationalisé après la Seconde Guerre mondiale, le groupe a connu une forte croissance dans les décennies suivantes, avant d'être privatisé en **1996**.

Renault est également connu pour son approche avant-gardiste en matière d'innovation, avec des modèles emblématiques comme la **4CV**, la **R4**, la **R5**, la **Clio** ou encore la **Twingo**, qui ont marqué des générations d'automobilistes.

À partir des années 2000, le groupe a accéléré sa stratégie de globalisation, notamment via des alliances stratégiques. En **1999**, il crée l'**Alliance Renault-Nissan**, à laquelle viendra s'ajouter **Mitsubishi** en **2016**, formant l'un des plus grands groupes automobiles mondiaux.

Dans les années 2010, Renault prend une position de **leader européen du véhicule électrique**, en lançant des modèles pionniers comme la **ZOE** ou le **Kangoo ZE**, tout en

investissant massivement dans la recherche et le développement autour des **mobilités durables, connectées et intelligentes**.

Aujourd'hui, avec la création d'**Ampere**, sa nouvelle entité dédiée au véhicule électrique et au software embarqué, Renault affirme sa volonté de jouer un rôle de premier plan dans la transformation de l'industrie automobile. L'entreprise s'inscrit pleinement dans la dynamique du **Software Defined Vehicle (SDV)** [28], en mettant le logiciel au cœur de la conception, du pilotage et de l'évolution de ses futurs véhicules.

2.1.2 Création et rôle d'Ampere dans le SDV

Face aux transformations majeures que connaît l'industrie automobile — **électrification, connectivité, digitalisation, exigences environnementales** — le **Groupe Renault** a décidé de structurer son organisation autour d'entités spécialisées. C'est dans cette logique qu'a été créée **Ampere**, une filiale à part entière dédiée aux véhicules électriques (EV) et aux logiciels embarqués.

Lancée officiellement en **2023**, Ampere a pour ambition de faire du Groupe Renault un **leader européen du Software Defined Vehicle (SDV)**. Cette entité réunit plus de **11 000 collaborateurs**, dont **3 500 ingénieurs software**, et dispose d'une organisation indépendante afin d'accélérer le développement de solutions logicielles innovantes pour les véhicules du futur.

Le **SDV** (*Software Defined Vehicle*) désigne une nouvelle génération de véhicules conçus pour être définis, contrôlés et mis à jour majoritairement via des logiciels. Contrairement aux architectures traditionnelles, où les fonctionnalités sont figées dans des calculateurs aux fonctions spécifiques, les SDV s'appuient sur des architectures **centralisées, évolutives et reprogrammables à distance**. Ce paradigme permet :

- des mises à jour **OTA (Over-The-Air)** pour enrichir ou corriger les fonctionnalités du véhicule tout au long de son cycle de vie ;
- une **réduction de la complexité matérielle**, au profit d'un pilotage intelligent via des plateformes logicielles communes ;
- une **adaptabilité accrue** aux nouveaux services de mobilité, à la connectivité et à la personnalisation client.

Ampere joue un rôle central dans cette mutation, en pilotant le développement logiciel des véhicules électriques de Renault et en collaborant étroitement avec les autres entités du groupe. Elle conçoit non seulement l'**architecture logicielle globale** (gestion de l'énergie, sûreté de fonctionnement, interfaces homme-machine, diagnostic embarqué, etc.), mais aussi les outils nécessaires à leur **validation**, leur **déploiement** et leur **industrialisation**.

Dans ce cadre, mon stage au sein d'**Ampere Software Technology**, et plus précisément dans l'équipe **S53 – Power Activation and Distribution**, s'est inscrit au cœur de cette dynamique SDV. J'ai pu participer activement à la construction d'**outils internes** liés au pilotage logiciel de composants critiques, notamment les **SmartMOS**, dans une logique d'**optimisation**, de **traçabilité** et d'**automatisation** des processus métiers.

Il convient toutefois de souligner que le passage au SDV représente un **défi technologique et organisationnel** de très grande ampleur. Si certains constructeurs comme **Tesla**, pionnier américain, ou **BYD**, géant chinois, intègrent déjà des architectures *full-software* dans leurs véhicules, aucun constructeur européen ne l'a encore totalement concrétisé à grande échelle. Plusieurs tentatives ont été amorcées par des acteurs européens et nord-américains, sans aboutir, ce qui témoigne de la complexité du virage SDV.

Dans ce contexte, Renault ne peut se permettre une transition précipitée. Fort de son histoire centenaire, le constructeur se distingue par une culture d'ingénierie rigoureuse et une relation de confiance durable avec ses clients. La priorité reste la **fiabilité**, la **sécurité** et l'**autonomie réelle** des véhicules, des critères jugés essentiels par une clientèle exigeante, tant pour les véhicules particuliers que professionnels. Ainsi, si certains observateurs peuvent percevoir un retard, il s'agit avant tout d'un **choix stratégique** fondé sur l'**excellence industrielle** et le **pragmatisme**.

Avec Ampere, Renault affirme sa volonté de concilier **performance logicielle** et **exigence automobile**, en s'appuyant sur un savoir-faire reconnu et une démarche de transformation maîtrisée.

2.1.3 Lien avec les partenaires stratégiques (Google, Qualcomm, Valeo...)

Dans le cadre de sa transformation vers le **Software Defined Vehicle (SDV)**, Renault s'est entouré de **partenaires technologiques majeurs** afin d'accélérer l'innovation, de mutualiser les expertises et de rester compétitif dans un écosystème en profonde mutation. Ces collaborations stratégiques permettent à Renault, via **Ampere**, d'intégrer les dernières avancées en matière de **software**, de **connectivité**, de **traitement embarqué** et de **systèmes intelligents**.

Parmi ces partenariats clés :

- **Google** : Renault collabore étroitement avec Google pour intégrer les services **Google Automotive Services (GAS)** et le système d'exploitation **Android Automotive OS** directement dans ses véhicules. Cette intégration native offre une expérience utilisateur fluide, intuitive et connectée, avec des applications telles que Google Maps, Google Assistant ou Google Play, tout en conservant une couche logicielle propre à Renault, garantissant l'identité du constructeur.
- **Qualcomm** : L'accord stratégique signé avec **Qualcomm Technologies** repose sur l'intégration de la plateforme **Snapdragon Digital Chassis™**, une architecture de calcul haute performance dédiée aux véhicules connectés. Cette plateforme permet une puissance de traitement embarqué optimisée, indispensable pour la gestion des capteurs, des flux de données, des fonctionnalités **ADAS** (aide à la conduite), de la **connectivité 5G**, et bien sûr, du pilotage logiciel global du SDV.
- **Valeo** : Valeo est un partenaire de longue date de Renault dans le domaine des composants automobiles, mais la collaboration se renforce dans le cadre de la transition vers le SDV. En particulier, les travaux sur les **SmartMOS** (remplaçants intelligents des fusibles traditionnels) s'appuient en partie sur des composants issus de partenaires comme Valeo. Le lien est renforcé par le **partage de données techniques**, la **co-validation sur bancs d'essai**, et une volonté commune de garantir la **sécurité** et la **fiabilité** des systèmes de distribution électrique pilotés par logiciel.

Ces partenariats illustrent l'approche stratégique d'**Ampere** : ne pas réinventer l'ensemble des briques technologiques, mais s'appuyer sur les meilleurs acteurs mondiaux pour construire une **architecture logicielle et matérielle** robuste, cohérente et durable. Cette capacité à coopérer tout en conservant la maîtrise des couches critiques du système est un levier d'**agilité** et un gage de **compétitivité** dans un secteur en pleine redéfinition.

2.2 L'entité Ampere

2.2.1 Rôle dans l'organisation Ampere

Au sein de l'organisation **Ampere Software Technology**, l'équipe **S53 – Power Activation and Distribution** occupe une position stratégique dans l'architecture logicielle des véhicules électriques de nouvelle génération. Sa mission principale est de concevoir, développer et valider les fonctions liées à l'alimentation électrique des véhicules, depuis l'activation (réveil) jusqu'à la mise en veille (endormissement) des différents calculateurs, composants et sous-systèmes électroniques.

Dans un véhicule électrique entièrement piloté par logiciel, la gestion fine de la distribution de puissance est essentielle, tant pour la performance énergétique que pour la sûreté de fonctionnement. L'équipe S53 est ainsi responsable de fonctions critiques telles que :

- la séquence de **wake-up** du véhicule et des unités de contrôle (par appui sur la clé, déverrouillage, détection de présence, etc.) ;
- l'activation et la désactivation des charges via des composants programmables appelés **SmartMOS** (en remplacement des fusibles mécaniques traditionnels) ;
- la gestion de l'**endormissement progressif** pour optimiser la consommation et préserver la batterie à l'arrêt ;
- le **diagnostic électrique embarqué**, permettant de détecter, localiser et anticiper les défauts électriques sur le réseau de puissance.

Pour remplir ces missions, l'équipe S53 combine une **expertise système** forte avec une **maîtrise approfondie des outils logiciels**, en interaction permanente avec les autres entités d'Ampere (connectivité, HMI, sûreté de fonctionnement, etc.) ainsi qu'avec les fournisseurs externes de composants critiques.

Dans le cadre de cette réorganisation vers une architecture **SDV**, S53 a également la responsabilité de développer ses propres outils de validation et de diagnostic, à la fois pour **sécuriser le développement** et pour **faciliter l'intégration** dans la chaîne de production logicielle de Renault.

C'est dans ce contexte que s'est inscrit mon stage, en lien étroit avec les missions de l'équipe S53, avec pour objectif de **concevoir des outils d'automatisation et d'analyse** destinés à renforcer la robustesse et la traçabilité des activités liées à l'alimentation et à la distribution électrique dans les véhicules du futur.

2.2.2 Lien entre énergie, activation, et software-defined vehicle

Dans un véhicule thermique traditionnel, l'activation des composants électroniques repose principalement sur des architectures électriques relativement figées : des relais, des fusibles mécaniques, et des séquences de mise en marche déterminées par le fonctionnement du moteur thermique. Avec l'arrivée du véhicule électrique, et plus encore du **Software-Defined Vehicle (SDV)**, cette logique est complètement repensée.

Le **SDV** repose sur une architecture **centralisée, évolutive**, et hautement **logicielle**, dans laquelle l'ensemble des fonctions critiques du véhicule (moteur, climatisation, tableau de bord, éclairage, ADAS...) peuvent être activées, désactivées ou mises à jour à distance. Dans ce nouveau paradigme, l'activation des organes du véhicule devient une **fonction logicielle à part entière**, à gérer avec précision, sécurité, et intelligence.

C'est là qu'intervient le rôle fondamental de la **gestion d'énergie** et des équipes comme **S53 – Power Activation and Distribution**. Il ne s'agit plus simplement d'alimenter ou de

couper un composant : il faut désormais piloter dynamiquement l'alimentation des différentes charges du véhicule en fonction :

- de l'état du véhicule (véhicule éteint, en veille, en route, en charge, etc.) ;
- du comportement utilisateur (déverrouillage à distance, approche du véhicule, utilisation de services connectés, etc.) ;
- de la stratégie d'optimisation énergétique (préservation de l'autonomie, réduction de la consommation en veille...) ;
- et bien sûr, des règles de **sûreté de fonctionnement** (ISO 26262 [1]).

Dans ce contexte, le passage des fusibles traditionnels à des **SmartMOS** représente un tournant clé. Ces composants intelligents permettent de commander finement l'alimentation de chaque ligne, tout en intégrant des fonctions de **diagnostic**, de **sécurité** et de **calibration**. Grâce aux SmartMOS, chaque ligne de puissance devient une **entité logicielle configurable**, ce qui ouvre la voie à une gestion électrique complètement pilotée par logiciel.

Ainsi, la **gestion d'énergie et d'activation** n'est plus un simple sous-système du véhicule, mais un **pilier du Software-Defined Vehicle**. Elle conditionne à la fois la réactivité du véhicule, sa consommation, sa sécurité, et son évolutivité dans le temps (via les mises à jour logicielles ou OTA).

Le travail mené par **S53**, notamment dans le cadre du projet **FlexEvan**, illustre parfaitement cette convergence entre **hardware électrique** et **pilotage logiciel intelligent**, dans le but de bâtir une nouvelle génération de véhicules **robustes, sûrs, économes** et entièrement **reprogrammables**.

2.3 L'équipe S53 – Power Activation & Distribution

2.3.1 Missions : wake-up, shut-down, protection électrique

L'équipe **S53 – Power Activation and Distribution**, au sein de l'entité **Ampere Software Technology**, est responsable de plusieurs fonctions critiques liées à la gestion de l'alimentation électrique dans un véhicule **Software-Defined (SDV)**. Ces missions s'articulent autour de trois axes principaux : le **wake-up** (réveil), le **shut-down** (endormissement), et la **protection électrique**.

1. Wake-up : Séquences d'activation du véhicule

Dans un véhicule SDV, le démarrage ne se limite plus à l'allumage d'un moteur. Il s'agit de déclencher, dans un ordre précis et sécurisé, l'ensemble des calculateurs, organes de puissance et équipements embarqués nécessaires au bon fonctionnement du véhicule.

L'équipe S53 conçoit et pilote les scénarios de réveil, déclenchés par différents événements : appui sur une touche de la carte, ouverture de porte, détection de présence via la clé ou encore réception d'une commande à distance. Ces séquences doivent respecter des contraintes temporelles strictes, tout en garantissant la **sûreté de fonctionnement** et l'**intégrité des données** lors de l'alimentation des lignes.

2. Shut-down : mise en veille sécurisée du véhicule

Inversement, le shut-down correspond à l'extinction progressive et maîtrisée des systèmes du véhicule, dans une logique d'économie d'énergie et de préservation de la batterie. Il ne s'agit pas d'un simple arrêt brutal : certaines unités doivent rester alimentées temporairement pour effectuer des tâches de fond (enregistrement, sauvegarde, veille connectée...), tandis que d'autres doivent être coupées immédiatement.

S53 établit les règles de désactivation intelligente selon des cas d'usage multiples (verrouillage du véhicule, absence prolongée, fin de session de conduite, etc.) et s'assure que la **consommation résiduelle** reste inférieure aux **seuils critiques**, pour garantir une autonomie longue même à l'arrêt.

3. Protection électrique via les SmartMOS

Avec l'arrivée des **SmartMOS**, composants intelligents remplaçant les fusibles traditionnels, la protection des lignes électriques devient **programmable, dynamique, et évolutive**.

L'équipe S53 est responsable :

- de la définition des calibrations des SmartMOS (courants seuil, temporisations, typologie des charges...);
- de la vérification des protections câblage selon les lois i^2t (intégrale de courant au carré);
- du pilotage logiciel des activations/désactivations ligne par ligne;
- et du diagnostic embarqué des surintensités, défauts ou ouvertures involontaires.

Ces SmartMOS sont pilotés directement depuis le logiciel embarqué, ce qui permet une **flexibilité de gestion inédite : reconfiguration à distance, tolérance aux défauts, surveillance en temps réel**... autant de leviers pour sécuriser l'ensemble du réseau électrique embarqué.

2.3.2 SmartMOS : définition, rôle et différence avec les fusibles

Définition d'un SmartMOS

Un **SmartMOS** (ou *Smart Metal-Oxide-Semiconductor*) est un composant électronique de puissance qui combine un interrupteur MOSFET (transistor à effet de champ) avec une **intelligence embarquée** permettant de **piloter, surveiller et protéger une ligne électrique** de manière dynamique. Contrairement à un fusible traditionnel, le SmartMOS est **reconfigurable par logiciel**, capable de mesurer le courant, de réagir à des dépassements, et d'envoyer des informations de diagnostic en temps réel.

Il s'agit d'un élément central de l'architecture électrique des véhicules **Software-Defined (SDV)**, où chaque ligne d'alimentation devient **une entité contrôlée numériquement**, comme un module logiciel.

Dans le contexte du SDV, et en particulier dans les projets menés par l'équipe **S53** d'Ampere, les SmartMOS jouent un rôle fondamental dans la **gestion de l'énergie et la protection du réseau de distribution électrique**. Leurs fonctions principales incluent :

- **Activation / désactivation des charges électriques** (calculateur, pompe, éclairage...) en fonction de l'état du véhicule;
- **Surveillance des courants** circulant dans les lignes;
- **Protection contre les surintensités**, les courts-circuits ou les défauts d'isolation;
- **Diagnostic embarqué**, avec remontée d'informations vers le système central;
- **Adaptabilité logicielle**, via des paramètres configurables à distance (courant de coupure, temporisation, etc.).

Ces fonctionnalités permettent une **flexibilité accrue**, une **réduction des erreurs humaines** lors des phases de calibration, et une **meilleure traçabilité** des incidents électriques.

Le remplacement des fusibles par des SmartMOS constitue donc **un changement de paradigme majeur**, à la fois sur le plan **technique, logiciel et organisationnel**. Cette transition permet de gagner en **fiabilité**, en **efficacité énergétique**, et en **agilité logicielle**,

Critère	Fusible classique	SmartMOS
Nature	Composant passif, destructif en cas de défaut	Composant actif, réutilisable
Mode de fonctionnement	Rupture thermique du conducteur	Coupure électronique commandée par logiciel
Calibration	Fixe, imposée par la nature du fusible	Paramétrable dynamiquement (courant, temporisation, $i^2t...$)
Diagnostic	Aucun retour d'information	Diagnostic en temps réel intégré
Réactivité	Temps de réponse limité	Réaction rapide et contrôlée électroniquement
Reconfiguration	Remplacement manuel nécessaire	Reprogrammation possible à distance ou via mises à jour logicielles
Adapté au SDV	Non	Oui, élément clé d'une architecture pilotée par logiciel

TABLE 1 – Comparaison entre fusibles classiques et SmartMOS

mais elle impose aussi de **redéfinir entièrement les outils, les processus de validation, et les stratégies de diagnostic**.

C'est précisément dans cette optique que s'inscrivent les travaux menés durant ce stage, avec le développement de **scripts d'automatisation** et d'**outils de vérification** autour de la calibration, de la protection câblage, et de la cohérence entre les données SmartMOS et le système d'ingénierie SDE.

2.3.3 Outils clés : SDE (Schéma de Distribution Électrique), DDT

Le schéma SDE : une représentation visuelle essentielle

Le **schéma SDE** (Schéma de Distribution Électrique) offre une vue normalisée des connexions internes du véhicule (voir figure 3). Il permet de visualiser :

- la **topologie de câblage** et la **distribution de puissance**,
- l'**implantation/usage des SmartMOS** et les **fonctions alimentées**.

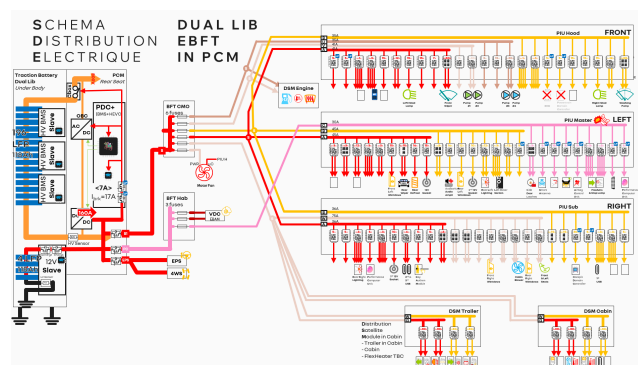


FIGURE 3 – Exemple de Schéma de Distribution Électrique (SDE).

Le schéma est segmenté par **PIU** (Power Interface Unit) : *PIU Hood (Front)*, *PIU Master (Left)*, *PIU Sub (Right)*, et le cas échéant *DSM Trailer/Cabin*.

Éléments clés représentés

- Sources et réseaux : **batteries** (traction, 12 V), **HV/LV**.
- **Protections et liaisons** : lignes protégées par SmartMOS (courant en A), chemins colorés.
- **Consommateurs et modules** : lampes, moteurs, pompes, ECU, *PDC+*, *BFT*, *DSM*.
- **Symboles normalisés** pour une identification rapide (essuie-glace, clignotants, ventilateurs, capteurs...).

Usages

- **Validation fonctionnelle** (sur banc),
- **Conception électrique** (avec les données Excel),
- **Diagnostic** (génération d'écrans DDT).



Complément

Le schéma est **visuel** : il n'embarque pas les niveaux **ASIL**, ni les **calibrations** ou logiques de reconfiguration. Pour ces données, la référence reste le **fichier Excel du SDE (SSOT)**.

Le fichier Excel du SDE : *Single Source of Truth* (SSOT)

Le fichier Excel du SDE est la **référence unique** pour la distribution électrique d'un véhicule SDV. Il structure les informations nécessaires au **dimensionnement**, à la **validation** et au **diagnostic**.

Contenus principaux (par ligne)

- **Identifiants & localisation** : *SmartMOS ID*, *Référence*, *PIU*.
- **Fonctions** : *Distribution Function Name*, champs d'usage (*System Controlling*, *MBSE Function*, *Load ID*).
- **Courants** : *IN* (nominal), *IP* (protection).
- **Protections** : groupes *Référence* et *Sanitaire* (BTG/VNF).
- **Sûreté** : niveaux *ASIL* (ISO 26262 [1]).

Principaux usages

- **Validation de cohérence** et vérifications critiques (ex. *SDE Check*),
- **Génération d'écrans DDT** (XML),
- **Analyses de calibration/protection** (ex. *2CVC*, *ASCS*).

En résumé, l'Excel SDE **centralise** les paramètres critiques liés aux SmartMOS et sert de support **opérationnel** à la conception, la validation, les tests et le diagnostic — véritable **SSOT** des équipes projet.



FIGURE 4 – Les trois modèles FlexEVan (de gauche à droite) : Estafette, Goelette, Trafic

2.4 Projet FlexEvan

2.4.1 Description du véhicule

Le **FlexEVan** est un **programme stratégique de véhicules utilitaires 100 % électriques**, développé dans le cadre de la coentreprise **Flexis**, réunissant **Renault Group (45 %)**, **Volvo Group (45 %)** et **CMA CGM (10 %)** (voir figure 4). Ce projet englobe le développement de **trois véhicules distincts**, destinés à répondre aux besoins variés du transport professionnel, notamment pour le **dernier kilomètre**. La production est prévue à partir de **2026** dans l'usine de **Sandouville** (Normandie), avec pour ambition de faire de la France un acteur de référence dans le domaine du transport utilitaire électrique bas carbone en Europe.

Ce projet regroupe trois modèles d'utilitaires modulaires :

- **Estafette** : le plus grand, directement pensée pour les **livreurs urbains** avec ses **dimensions compactes (4,87 m)**, son **rayon de braquage optimisé**, son **plancher plat** et sa **cabine connectée**,
- **Goelette** : un modèle **polyvalent** adapté aux artisans, collectivités ou flottes techniques,
- **Trafic** : le modèle **le plus compact**, idéal pour les zones d'accès difficile.



FIGURE 5 – Concept Estafette (2024) et intérieur du FlexEVan — design inspiré du passé, fonctionnalités modernes (hayon vertical, portes coulissantes) et interface Android Automotive sur architecture centralisée.

Le design de l'Estafette Concept, présenté en 2024 (voir figure 5), reprend des éléments visuels inspirés du modèle des années 60, tout en intégrant des fonctionnalités modernes : **hayon vertical**, **portes coulissantes**, **ergonomie pensée avec des professionnels de la logistique**, etc.

La plateforme technique utilisée pour le FlexEVan repose sur une **architecture SDV (Software Defined Vehicle)**, conçue pour permettre une **évolutivité logicielle poussée**, une **connectivité avancée**, et une **centralisation électronique**. Cette plateforme sera développée plus en détail dans la section suivante.

Les véhicules de la gamme proposeront deux capacités d'autonomie : **200 km et 400 km**, une **recharge ultra-rapide** grâce à l'**architecture 800 V** (système de charge conductive [10]), et des fonctionnalités avancées comme le **V2G (Vehicle-to-Grid [8])**.

Le FlexEVan promet également une **réduction de 30 % du coût total d'usage**, grâce à une diminution des frais de maintenance, à la fiabilité du système électrique, et à la modularité matérielle et logicielle.

Enfin, ce projet est emblématique d'un **savoir-faire industriel français**, soutenu par l'État et salué par les observateurs. Là où certains constructeurs américains ou européens ont **abandonné** l'ambition SDV, Renault a choisi de **prendre le temps** de bien faire. Ce n'est pas un retard, mais un **choix stratégique** : Renault est reconnu pour la **fiabilité** de ses véhicules, leur **autonomie** et leur **robustesse**. Le FlexEVan incarne cette exigence : un véhicule **prêt à durer**, conçu avec rigueur, et répondant réellement aux besoins du marché.

2.4.2 Innovations : SDV, architecture centralisée

L'une des grandes forces du projet **FlexEVan** réside dans son **architecture électronique et énergétique unifiée**, pensée dès la conception pour s'adapter à plusieurs configurations de véhicules tout en mutualisant les composants critiques. Sur l'image ci-dessous (voir figure 6), on peut observer les trois variantes de l'utilitaire (inspirées respectivement de la *Goélette*, du *Trafic* et de l'*Estafette*) partageant un même **châssis modulaire**. Cette base technique commune n'est pas qu'un simple choix industriel : elle illustre l'essence même du **Software Defined Vehicle (SDV)**, qui repose sur une gestion logicielle homogène des fonctions physiques du véhicule.



FIGURE 6 – Gamme FlexEVan : Estafette, Goelette, Trafic, et plateforme technique commune.

Au cœur de cette architecture : les **PIU (Power Interface Units)**. Ceux-ci jouent un rôle fondamental dans la **distribution intelligente de l'énergie** à travers le véhicule. Concrètement, chaque véhicule est découpé en plusieurs zones, chacune gérée par un PIU localisé stratégiquement :

- **PIU Hood (avant)** : alimente les composants frontaux comme les *phares*, *essuie-glaces*, *pompes*, etc.,
- **PIU Master (côté conducteur)** : gère des fonctions centrales telles que les *vitres électriques*, *ports USB*, *dégivrage* ou encore les *antennes*,

- **PIU Sub (côté passager)** : couvre les équipements arrière et latéraux comme les *sièges chauffants*, *feux arrière* ou les *contrôleurs de confort*.

Chaque PIU est équipé de **SmartMOS**, des composants semi-conducteurs programmables remplaçant les fusibles classiques. Cela permet un **pilotage dynamique** des fonctions électriques, une **meilleure résilience**, et surtout une **réduction significative du câblage** dans le véhicule, avec des **diagnostics précis en cas de panne**.

Grâce à cette **centralisation intelligente**, Renault parvient à rationaliser l'assemblage, faciliter la maintenance, tout en laissant la possibilité à chaque modèle d'avoir ses propres **spécificités logicielles et matérielles**. C'est ce **socle technologique commun** qui permettra à la gamme FlexEVan d'évoluer facilement, tant au niveau des **mise à jour OTA (Over-The-Air)** que des extensions futures de service pour différents métiers (*logistique, collectivités, artisans...*).

2.4.3 Implication de S53 dans le projet

L'entité **S53 – Power Activation & Distribution** joue un rôle déterminant dans le développement du *FlexEVan*, en particulier dans l'intégration des composants intelligents assurant la distribution, la protection et l'activation de l'énergie dans le véhicule. Cette entité est au cœur des enjeux techniques liés à l'architecture électrique innovante du SDV.

Plusieurs responsabilités majeures sont assurées par S53 :

- **Définition des stratégies de wake-up et shut-down du véhicule** : S53 conçoit la logique de démarrage et d'endormissement, essentielle pour l'efficacité énergétique, la sécurité, et le confort utilisateur (exemple : ouverture sans contact, démarrage automatique en approche, mise en veille intelligente).
- **Sélection, validation et intégration des SmartMOS** : dans le cadre du remplacement des fusibles classiques, S53 travaille sur la sélection des références de SmartMOS adaptées à chaque besoin du *FlexEVan*. L'équipe assure également leur *calibration*, leur *protection thermique*, ainsi que leur *déclenchement différé ou conditionnel* selon les scénarios d'usage.
- **Développement et validation du fichier SDE (Schéma de Distribution Électrique)** : la conception du SDE est pilotée par S53. Elle représente la *source unique de vérité (SSOT)* sur l'architecture électrique du véhicule. Elle est utilisée aussi bien pour la documentation interne que pour les validations fonctionnelles et les diagnostics.
- **Automatisation des outils de conception** : dans le cadre du projet *FlexEVan*, S53 a également investi dans l'*automatisation d'outils logiciels* afin de faciliter la mise à jour des fichiers Excel SDE, la génération d'écrans de diagnostic (via DDT), ou encore l'optimisation du choix des SmartMOS. Cette automatisation, souvent initiée par les stagiaires et jeunes ingénieurs, a permis de **gagner du temps** et de **sécuriser la qualité des livrables**.

Enfin, l'implication de S53 s'inscrit dans une **vision plus large de standardisation** des architectures électriques sur plusieurs plateformes. *FlexEVan* sert donc aussi de **pilote pour d'autres projets SDV**, et les choix réalisés aujourd'hui auront des répercussions concrètes sur les futurs véhicules connectés de la gamme Renault.

3 Objectifs et périmètre du stage

3.1 Objectif principal

L'objectif de mon stage a été de développer des **outils d'automatisation robustes et modulaires** pour **produire et valider** les livrables issus du *Schéma de Distribution Électrique*

(SDE) du projet *FlexEVan*, tout en **réduisant les erreurs manuelles** et en **accélérant** les cycles de vérification. Le SDE, considéré comme *Single Source of Truth (SSOT)*, pilote la distribution d'énergie et relie SmartMOS, PIU et éléments de diagnostic.

⚠ Problème identifié

Les vérifications manuelles du SDE sont **lentes** et **sources d'erreurs** : la feuille *Power Distribution* regroupe **~500 SmartMOS**, **>70 colonnes/ligne** et plusieurs feuilles liées, rendant la détection d'incohérences (courants, affectations, ASIL, etc.) laborieuse.

En pratique, la volumétrie et l'entrelacement des données rendent les contrôles manuels chronophages. D'où la nécessité d'**automatiser les vérifications critiques** et de **générer automatiquement les livrables**, notamment :

- des **rapports Excel/PDF** listant les incohérences et violations métier ;
- des **fichiers XML** pour les écrans DDT ;
- des **filtres de validation** pour sécuriser la chaîne de conception.

✓ Solution mise en œuvre

Des scripts Python analysent automatiquement le SDE, appliquent des règles complexes, **signalent les anomalies**, **génèrent des rapports personnalisés** et **produisent les écrans DDT (XML)**. L'ensemble **standardise** les livrables et **fiabilise** la validation.

Les règles imbriquées (p. ex. ASIL) sont appliquées **sans omission**. Exemple de logique métier typique :

« Si $X=Y$ alors vérifier A ; si $X=Z$, vérifier B et C , sauf si $B=W$ »

Une vérification ASIL typique passe de **~1 journée** à **< 1 s** de calcul + **< 1 h** de relecture, et sur des contrôles complexes le cumul économise **plusieurs jours** par cycle projet. Les livrables sont **homogènes**, **filtrés**, **colorés** et **triés**, ce qui facilite relecture, partage et archivage, indépendamment de l'auteur. Les outils restent **modulaires**, **réutilisables** et **adaptables** à d'autres véhicules/variantes : un **levier structurant** pour la qualité et la productivité de Renault.

3.2 Objectifs techniques secondaires

Bien que mon stage ait été centré sur l'automatisation autour du *SDE*, j'ai rapidement contribué à des sujets transverses à **fort impact** (réutilisables, alignés métier), au point d'inspirer des **pistes de brevet**.

Génération automatisée d'écrans DDT2000

J'ai conçu un outil de **génération automatique d'écrans XML** pour **DDT2000** à partir du SDE (*SSOT*). Historiquement, la création manuelle dans l'interface DDT2000 (années 1990) était lente, peu ergonomique et sujette à erreurs. L'automatisation construit dynamiquement différents écrans :

- **Distribution** : *Distribution Function Name, Vehicle Power Mode, Current, Error State* ;
- **Shutdown** : conditions d'endormissement ;
- **Activation, OEM et Recorder** : signaux et enregistrements dédiés.

Bénéfices : création passée de **plusieurs jours** à **quelques secondes**, livrables **standardisés** et **robustes**. Une **valorisation** (brevet) est envisagée.

Sélection des protections câblage (ASCS)

J'ai développé un **outil d'aide à la décision** pour l'analyse automatisée des **protections électriques** (SmartMOS, BTG, VNF) et la sélection des composants adaptés à un **profil de courant**. Fonctions clés :

- **Chargement** d'un profil et comparaison à l'ensemble des courbes ;
- **Validation** (ou non) selon les critères i^2t , seuils et tolérances ;
- **Classement visuel** : **vert** (valide), **orange** (limite), **rouge** (invalide) ;
- **Suggestions** de composants alternatifs (type, SW/HW, disponibilité).

Impact : gain de temps substantiel et **fiabilisation** des décisions techniques.

Interfaces utilisateurs (Desktop & Web)

Pour rendre ces automatisations accessibles à tous (experts ou non), j'ai livré des **interfaces ergonomiques** :

- **Desktop** : Tkinter / CustomTkinter ; **Web** : Flask (HTML/CSS/JS) ;
- Navigation entre fonctionnalités (génération, analyse, comparaison) ;
- Sélection guidée des fichiers sources ; **visualisation dynamique** (scores, erreurs) ;
- **Export** en un clic (Excel, PDF, XML).

Résultat : outils **intuitifs**, **robustes**, faciles à **réutiliser** sur d'autres projets.

Synthèse d'impact

- **Standardisation** et **qualité** des livrables (moins d'erreurs, mieux tracés) ;
- **Productivité** : tâches de plusieurs jours → **secondes** + relecture ;
- **Pérennité** : solutions **modulaires**, **réutilisables** et **scalables** ;
- **Innovation** : contributions **valorisables** (pistes de brevet).

4 Méthodologie et environnement de travail

4.1 Fonctionnement de l'équipe et intégration du stagiaire

4.1.1 Une organisation agile et structurée

L'entité *Ampere Software Technology* adopte une méthode de travail fondée sur l'**agilité**, avec une organisation en **sprints** sous la supervision d'un *Scrum Master*. Ce fonctionnement permet une planification efficace des tâches tout en restant flexible face aux évolutions et imprévus. La répartition progressive des projets et des responsabilités favorise l'autonomie tout en garantissant un encadrement adapté.

4.1.2 Un suivi régulier au cœur de l'organisation

Dès les premières semaines, j'ai été intégré au fonctionnement quotidien de l'équipe grâce à un **cadre de réunions structurant** et un accompagnement bienveillant :

- Deux **réunions Cotech** par semaine (le lundi et le mercredi) permettaient de suivre l'avancement global des projets et de contribuer aux orientations techniques collectives.
- Un **daily meeting** chaque vendredi matin, durant lequel chaque membre partageait ses avancées, ses difficultés éventuelles et ses priorités à venir.

- Une réunion hebdomadaire nommée **RUET**, chaque lundi matin, présidée par *Sylvain Henry*, avait pour objectif de traiter les aspects de vie d'équipe : intégration de nouveaux membres, périodes de congés, et informations générales internes.
- Une **bilatérale planifiée en début de semaine avec mon tuteur Baptiste Romieu**, permettant de définir les priorités hebdomadaires, clarifier certaines problématiques et valider les orientations techniques. D'autres points ponctuels pouvaient s'ajouter selon les besoins : Baptiste s'est montré particulièrement disponible tout au long du stage, favorisant un travail fluide et une montée en compétence efficace.
- Enfin, j'ai pris l'initiative de rédiger un **rapport d'avancement chaque fin de semaine**, à la fois pour informer Baptiste et Sylvain de mes progrès, mais aussi pour **structurer mon propre travail** et garder une trace claire des tâches effectuées et des prochaines étapes.

4.2 Outils et langages

L'ensemble des scripts et outils développés pendant le stage s'appuie essentiellement sur **Python**, sélectionné pour sa **puissance d'automatisation**, sa **lisibilité**, et la **richesse de ses bibliothèques** adaptées au traitement de données industrielles. Afin de garantir une adoption fluide par les équipes, des interfaces graphiques ont été intégrées et des exécutables **.exe** ont été générés pour une portabilité immédiate.

Catégorie	Outils / Langages	Utilisation
Langage principal	Python	Traitement de données, automatisation, parsing, génération de rapports
Traitement de fichiers	pandas, openpyxl, os, re, difflib	Analyse et manipulation avancée de fichiers Excel et XML
Interfaces utilisateur	Tkinter, Flask	Création d'interfaces desktop ou web pour simplifier l'usage des outils
Portabilité	pyinstaller, cx_Freeze (ou équivalent)	Génération d'exécutables .exe pour utilisation sans installation préalable
Types de fichiers traités	Excel (.xlsx), XML, CSV, TXT	Données internes SDE, calibrations, cartographies, diagnostics...

TABLE 2 – Technologies et usages — synthèse

5 Développement des outils et scripts

Dans cette partie, sont présentés de manière autonome les différents outils que j'ai développés au cours de mon stage. Chacun de ces outils répond à une problématique spécifique rencontrée par les équipes dans la manipulation, la vérification ou la génération de livrables liés au SDE ou à d'autres fichiers techniques (MBSE, calibration, XML...).

L'accent a été mis non seulement sur l'automatisation pour gagner en efficacité, mais aussi sur l'ergonomie pour faciliter leur appropriation par les utilisateurs. Chaque script ou interface est donc pensé comme un **outil opérationnel réutilisable**, robuste, évolutif, et **conçu pour intégrer les pratiques de l'équipe sur le long terme**.

5.1 Outil Wire Protection Calib

5.1.1 Contexte et besoin métier – Vérification croisée SDE / APSW

Dans le processus de validation des configurations électriques du véhicule, l'un des enjeux majeurs est de garantir **l'alignement des données** entre différents fichiers de référence, notamment entre le **SDE** (Schéma de Distribution Électrique), considéré comme le *Single Source of Truth (SSOT)*, et des fichiers externes comme `APSW_calib_filtered1.xlsx`, qui regroupe des données de calibration exploitées en aval.

Avant l'automatisation, cette vérification était réalisée de façon entièrement manuelle par les ingénieurs, notamment par mon tuteur. Elle impliquait :

- De parcourir ligne par ligne le fichier de calibration,
- D'identifier chaque composant (généralement un SmartMOS),
- D'aller chercher manuellement dans le SDE les valeurs de référence (courants nominaux, de protection, BTG, etc.),
- De comparer les deux fichiers à l'œil nu,
- Et en cas d'écart, de corriger à la main la valeur dans le fichier APSW et de noter la modification dans un commentaire ou un rapport séparé.

Ce processus présentait plusieurs limites :

- **Chronophage** : navigation manuelle entre fichiers Excel volumineux, souvent composés de dizaines de feuilles ;
- **Peu fiable** : risques d'erreurs humaines (copier-coller, mauvaise cellule, omission...);
- **Peu traçable** : modifications dispersées, difficilement centralisées ou documentées de manière standardisée.

Le besoin métier était donc clair : automatiser l'ensemble du processus pour gagner en fiabilité, en rapidité et en transparence.

🎯 Objectif du script

Détecter automatiquement les écarts entre le fichier APSW et le SDE, appliquer les corrections nécessaires, et générer un rapport synthétique clair et traçable listant toutes les modifications.

5.1.2 Fonctionnement général du script

Le script repose sur un processus automatisé de comparaison entre deux fichiers Excel :

- **Le SDE**, référentiel interne de Renault, faisant autorité (Single Source of Truth, SSoT),
- **Le fichier fournisseur** `APSW_calib_filtered1.xlsx`, à vérifier et potentiellement corriger.

L'objectif est d'**aligner les données critiques du fournisseur avec les données officielles du SDE**, tout en assurant la traçabilité des corrections apportées. Ce processus se décompose en plusieurs étapes clés :

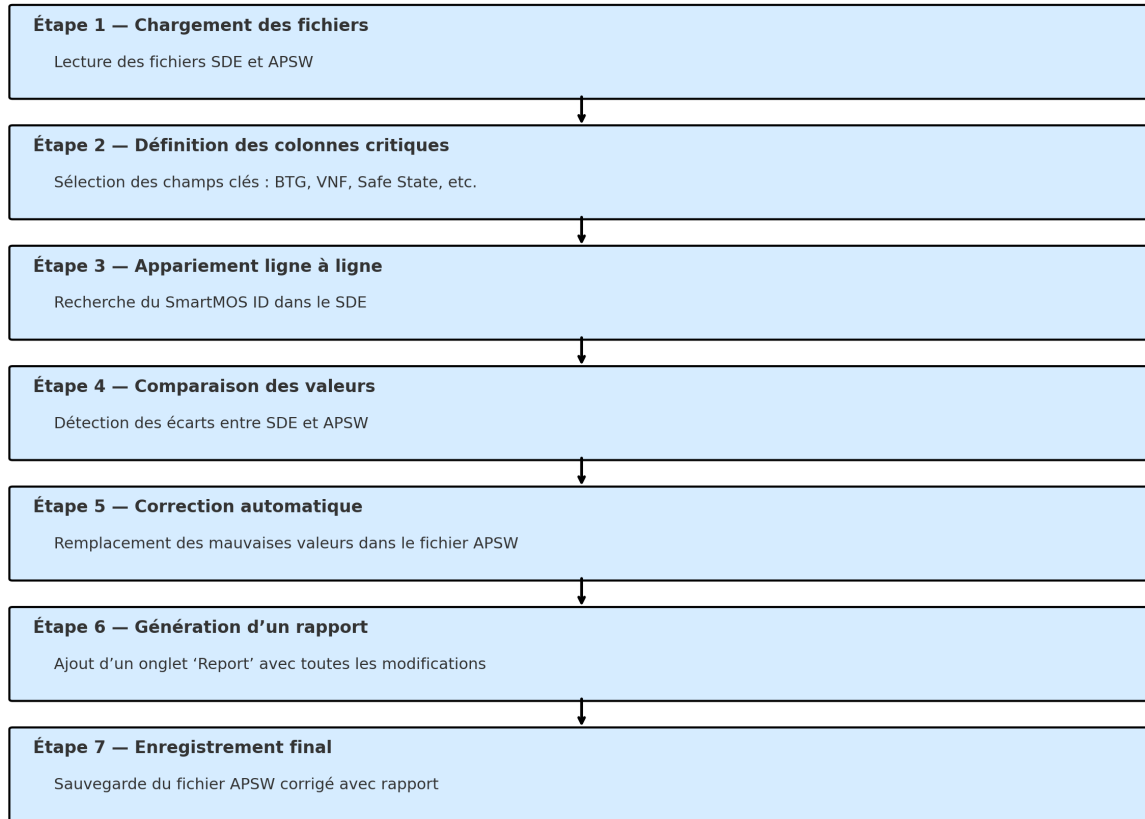


FIGURE 7 – Schéma du fonctionnement général du script de comparaison SDE / APSW

Étape 1 — Chargement des fichiers

- Ouverture et lecture des deux fichiers Excel.
- Le fichier SDE contient plusieurs feuilles, dont une dédiée à la distribution électrique.
- Le fichier APSW contient une liste de SmartMOS à vérifier.

Notation mathématique : on note S le SDE et A le fichier APSW. L'ensemble des identifiants communs est

$$\mathcal{K} = \text{IDs}(S) \cap \text{IDs}(A).$$

Étape 2 — Définition des colonnes à comparer

- Sélection de colonnes critiques comme les courbes de protection (BTG, VNF, Software Curve),
- Inclusion de l'état de sécurité (Safe State), pour garantir la conformité fonctionnelle.

Formalisation : si $\mathcal{C}$ désigne l'ensemble des colonnes d'intérêt, on considère pour chaque $c \in \mathcal{C}$ les fonctions de valeur

$$v_S(k, c), v_A(k, c) \quad \text{pour } k \in \mathcal{K}.$$

Étape 3 — Appariement ligne à ligne

- Identification du SmartMOS ID dans chaque ligne du fichier APSW,

- Recherche de la ligne correspondante dans le SDE en utilisant cet identifiant comme clé de jointure.

Jointure (équijointure) : avec la clé $K = \text{SmartMOS ID VALEO}\backslash\text{n v10}\backslash\text{n PIU BdV}$, on réalise

$$A \bowtie_K S = \{ (k, a, s) : a.K = s.K = k \in \mathcal{K} \}.$$

Étape 4 — Comparaison des valeurs

- Comparaison de chaque champ d'intérêt entre les deux fichiers,
- Détection des écarts entre la version fournisseur et la référence officielle.

Règles de comparaison :

- **Valeurs numériques** : ratio de variation relative

$$r(k, c) = \frac{|v_S(k, c) - v_A(k, c)|}{|v_A(k, c)|} \quad (\text{défini si } v_A(k, c) \neq 0).$$

- **Chaînes de caractères** : similarité normalisée $\text{sim}(v_S, v_A) \in [0, 1]$ (p. ex. mesure type *SequenceMatcher*).

Étape 5 — Correction automatique

- Remplacement automatique de la valeur APSW erronée par celle du SDE,
- Mise à jour directe du fichier APSW pour assurer la cohérence.

Règle d'alignement :

$$v_A^{\text{new}}(k, c) \leftarrow v_S(k, c) \quad \text{dès qu'un écart est détecté.}$$

Qualification de l'écart : avec des seuils $\tau_{\text{num}} = 0,2$ et $\tau_{\text{str}} = 0,8$,

$$\text{Rec}(k, c) = \begin{cases} \text{Significant} & \text{si } (r(k, c) > \tau_{\text{num}}) \text{ (numérique)} \\ \text{Significant} & \text{si } (\text{sim}(v_S, v_A) < \tau_{\text{str}}) \text{ (chaîne)} \\ \text{New added value} & \text{si } v_A(k, c) = \emptyset \wedge v_S(k, c) \neq \emptyset \\ \text{Removed value} & \text{si } v_S(k, c) = \emptyset \wedge v_A(k, c) \neq \emptyset \\ \text{Minor} & \text{sinon.} \end{cases}$$

Étape 6 — Génération d'un rapport de modifications

- Ajout d'une feuille **Report** dans le fichier APSW,
- Détail de chaque modification : identifiant concerné, colonne, valeur avant/après.

Traçabilité : l'ensemble des modifications est

$$\mathcal{E} = \{ (k, c, v_A^{\text{old}}(k, c), v_S(k, c), \text{Rec}(k, c)) : k \in \mathcal{K}, c \in \mathcal{C}, v_A^{\text{new}}(k, c) \neq v_A^{\text{old}}(k, c) \}.$$

Étape 7 — Enregistrement du nouveau fichier

- Sauvegarde du fichier corrigé sous un nom différent pour conserver la version originale,
- Le fichier exporté contient également le rapport des modifications.

Ce processus automatisé, à la fois rigoureux et reproductible, permet de gagner un temps considérable, d'éviter les erreurs manuelles et de garantir la conformité des données entre les différents acteurs (Renault et fournisseurs).

5.1.3 Résultats obtenus

L'exécution du script produit un **fichier Excel corrigé et enrichi**, dans lequel deux éléments principaux sont à distinguer :

1. **Mise à jour automatique des valeurs critiques** Après traitement, toutes les cellules des colonnes cibles (BTG Curve, VNF tN, Software curve, Safe state) dont la valeur divergeait du SDE sont automatiquement mises à jour avec la **valeur de référence issue du SDE**.

Exemple : Supposons qu'une ligne du fichier APSW initialement contient :

SmartMOS ID	BTG Curve	VNF tN	Safe state
MOS_12345_VA_PIU1	3	7.2	OFF

Et que le SDE de référence contient pour ce même composant :

SmartMOS ID	BTG Curve	VNF tN	Safe state
MOS_12345_VA_PIU1	4	7.2	ON

Alors, après exécution du script, le fichier APSW est automatiquement mis à jour comme suit (en **vert** les valeurs modifiées) :

SmartMOS ID	BTG Curve	VNF tN	Safe state
MOS_12345_VA_PIU1	4	7.2	ON

2. **Génération d'une feuille "Report" dédiée** Le fichier Excel est automatiquement enrichi d'un nouvel onglet nommé "Report" qui récapitule, pour chaque écart corrigé :
 - l'identifiant du SmartMOS concerné,
 - le nom de la colonne modifiée,
 - l'ancienne et la nouvelle valeur,
 - une **recommandation contextualisée** indiquant le niveau de vérification nécessaire.

Exemple de sortie dans la feuille Report :

SmartMOS ID	Column	Old Value	New Value	Recommendation
MOS_12345_VA_PIU1	Infineon BTG Curve number	3	4	Significant modification – Verify the change
MOS_67890_VA_PIU3	Safe state	OFF	ON	Minor modification
MOS_54321_VA_PIU2	Component without I2T	(vide)	Curve 1	New added value – Verify the value

Cette feuille joue un double rôle stratégique :

- **Support de communication technique** lors des échanges d'équipe (ex. : réunions, arbitrages, audits),
- **Documentation formelle** des corrections apportées, avec un **niveau de criticité** clair pour chaque modification.

5.1.4 Apports, plus-value et perspectives

Avant la mise en place de ce script, la comparaison entre le fichier SDE et les données APSW était une tâche réalisée **manuellement**, ligne par ligne, par un ingénieur. Cette méthode, bien qu'efficace à petite échelle, devenait rapidement fastidieuse, chronophage et sujette à erreurs dès que le volume de données augmentait.

Le script développé permet désormais d'**automatiser entièrement ce processus**, de générer un **rapport clair, structuré et traçable**, et surtout de fournir **des recommandations intelligentes** en fonction des différences détectées. Cela permet de passer d'une tâche rébarbative à une vérification fine, orientée analyse et décision.

Critère	Avant (manuel)	Après (avec script)
Temps de traitement	Plusieurs heures par fichier	< 1 min d'exécution + ~1 h de relecture
Risque d'erreur humaine	Élevé (copier/coller, oublis...)	Fortement réduit (vérifications automatiques)
Traçabilité des modifications	Faible (notes éparses)	Totale (rapport dans l'onglet Report)
Pertinence de l'analyse	Dépend de la rigueur de l'ingénieur	Standardisée (seuils, filtres, règles métier)
Réutilisabilité du processus	Faible (à refaire à chaque projet)	Forte (réutilisable pour les APSW similaires)
Accessibilité pour l'équipe	Réservée aux utilisateurs avancés d'Excel	Exécutable .exe, sans installation Python

TABLE 3 – Comparatif entre le processus manuel et Wire Protection Calib

Calcul du gain de temps

Le traitement manuel complet prenait en moyenne environ 5,5 heures, contre 1 heure aujourd'hui (script + relecture). Le gain peut donc être estimé par la formule :

$$\left(\frac{5,5 - 1}{5,5} \right) \times 100 = \left(\frac{4,5}{5,5} \right) \times 100 \approx 81,8\% \quad (1)$$

Gain de temps estimé : environ 82%

Ce gain de temps est particulièrement significatif dans un contexte industriel, où la répétition de cette tâche sur plusieurs projets représente **des jours entiers économisés** chaque mois. Il contribue à améliorer la qualité des livrables et à recentrer les ingénieurs sur des tâches à plus forte valeur ajoutée.

5.2 Outil PowerSW : Extraction des paramètres de calibration Leoni

5.2.1 Contexte et besoin métier – Vérification croisée SDE / APSW

Le fichier **APSW** (Automatic Power Software) contient des données essentielles de calibration pour les modules *SmartMOS*. Avant l'automatisation, un ingénieur devait manuellement retrouver, pour chaque combinaison de **SWC** (Software Component), **Calibration**, et **Adresse SmartMOS**, les informations techniques correspondantes : *Index*, *Valeur*, et *Résolution*. Ce processus était **fastidieux**, **source d'erreurs**, et difficilement **reproductible**.

De plus, ces recherches étaient régulièrement sollicitées par d'autres membres de l'équipe, ce qui impliquait des allers-retours fréquents et une **perte de temps considérable**. Le besoin exprimé était donc clair.

🎯 Objectif du script

Disposer d'un outil simple, robuste et rapide, permettant à n'importe quel utilisateur de retrouver instantanément les données de calibration associées à un SmartMOS donné, sans avoir à parcourir manuellement des centaines de lignes et colonnes.

La création du script `processingpowersw()` a permis de répondre à cette problématique en **automatisant totalement cette tâche**. Il intègre non seulement une *logique de recherche efficace*, mais également une **aide intégrée à l'utilisateur**, avec la *gestion des cas d'erreur*, des formats, et un **guidage clair** pour limiter les mauvaises manipulations.

5.2.2 Fonctionnement général du script

Le script `processingpowersw()` prend en entrée un fichier Excel de calibration contenant un grand volume de données réparties sur plusieurs feuilles. L'utilisateur est invité à saisir trois informations essentielles :

- Le **SWC** (Software Component),
- Le nom de la **calibration**,
- L'adresse **SmartMOS**.

Une fois ces informations renseignées, le script effectue les opérations suivantes :

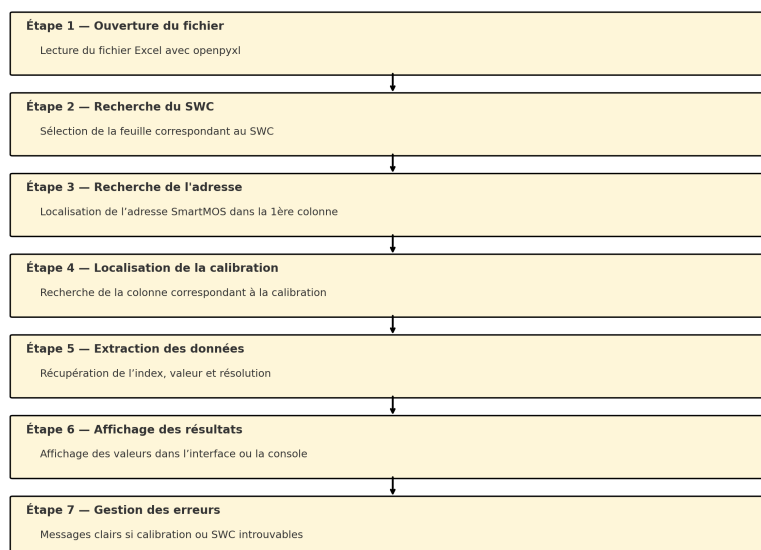


FIGURE 8 – Schéma de fonctionnement du script `processingpowersw()`

1. Ouverture et exploration du fichier Excel :

Le fichier de calibration est lu à l'aide de la bibliothèque `pandas/openpyxl` [13, 16] afin de parcourir l'ensemble des feuilles disponibles. Chaque feuille correspond à un contexte particulier, souvent identifié par le nom du SWC.

Formulation mathématique : on construit une correspondance

$$f : \text{SWC} \mapsto \{\text{calibration}_1, \text{calibration}_2, \dots\},$$

qui servira à valider les couples (SWC, calibration).

2. Recherche de la feuille correspondant au SWC :

Le script identifie la feuille dont le nom contient le SWC sélectionné. Si aucune feuille ne correspond, un message d'erreur clair est retourné à l'utilisateur.

Vérification logique :

$$(\text{SWC}, \text{calibration}) \in \text{graphe}(f).$$

3. Recherche de l'adresse SmartMOS :

Dans la feuille identifiée, le script recherche l'adresse SmartMOS sur la première colonne. Une fois la ligne localisée, il parcourt l'en-tête de la feuille pour repérer la colonne correspondant au nom de la calibration souhaitée.

Notation : on note $\alpha \in \{A, B, C, \dots\}$ la « lettre » associée à la calibration dans l'en-tête.

4. Extraction des données :

Une fois la cellule localisée, le script extrait :

- l'**index** (présent souvent dans une colonne d'offset par rapport à la calibration),
- la **valeur**,
- la **résolution** (souvent dans une cellule associée ou dans une feuille dédiée si elle n'est pas directement disponible).

Jointure implicite pour l'index :

$$\boxed{\text{index} = \text{Index}_\alpha \quad \text{tel que} \quad \text{ID}_\alpha = \text{SmartMOS}}.$$

Conversion physique :

$$\boxed{\text{Valeur physique} = \text{Index} \times \text{Résolution}}.$$

5. Affichage du résultat :

Le script affiche les résultats sous forme lisible :

- L'adresse trouvée,
- La calibration sélectionnée,
- L'index, la valeur et la résolution correspondants.

Exemple de sortie console :

```
Exemple de sortie console
SmartMOS : 0xC80A
Calibration : POWER_SW_CONFIG
Index : 2
Valeur : 200
Résolution : 0.5
```

6. Aide à l'utilisateur et robustesse :

Le script gère les erreurs les plus courantes : mauvaise orthographe, calibration introuvable, SWC mal écrit, adresse absente... et fournit des suggestions si besoin.

Ce fonctionnement permet une automatisation totale d'un processus auparavant manuel et chronophage, réduisant le risque d'erreurs et standardisant l'accès à des informations techniques critiques.

5.2.3 Résultats obtenus

L'exécution de la méthode `processingpowersw(smartmos, swc, calib)` permet d'obtenir, de manière immédiate et structurée, les trois informations clés attendues :

- **L'index associé** au SmartMOS dans le contexte de la calibration sélectionnée,
- **La valeur** de calibration correspondante (sous forme de chaîne ou de liste de valeurs),
- **La résolution**, permettant d'interpréter physiquement cette valeur.

Ces informations sont renvoyées sous forme de triplet Python directement exploitable :

```
1 ("Index_A", "0x78", "0.05")
```

Par exemple, pour l'appel suivant :

```
1 frame.processingpowersw("BM63", "Swc_SwitchCtrl", "Populated")
```

le résultat retourné peut être :

```
1 (120, "0x78", "0.05")
```

Ce triplet peut ensuite être utilisé :

- Pour remplir automatiquement une base de données ou un tableau de calibration,
- Pour vérifier manuellement la cohérence entre le paramétrage et les valeurs attendues,
- Pour affichage dans une interface utilisateur, permettant une visualisation claire du lien entre le SmartMOS, le SWC et la calibration.

En résumé, le script retourne avec fiabilité les informations techniques essentielles pour chaque triplet (SmartMOS / SWC / Calibration), centralisées à partir de différentes feuilles du fichier Excel.

5.2.4 Apports, plus-value et perspectives

L'automatisation de la méthode `processingpowersw()` apporte une réelle plus-value opérationnelle dans les processus de vérification et d'analyse des calibrations associées aux SmartMOS (voir table 4).

Gain de temps estimé

En prenant une hypothèse réaliste :

- Avant : traitement manuel d'un triplet ≈ 7 minutes
- Après : traitement automatisé de 10 triplets ≈ 1 minute

Le gain de temps est donc estimé par :

$$\text{Gain} = \left(1 - \frac{1/10}{7}\right) \times 100 \approx 98,6\% \quad (2)$$

Le temps de traitement est ainsi réduit de plus de **98 %** pour une séquence de 10 SmartMOS.

Perspectives

Cette automatisation ouvre la voie à :

- Une intégration fluide dans des interfaces graphiques accessibles aux non-développeurs,
- Une vérification en masse des calibrations dans le cadre de contrôles qualité,
- Une forte réduction de la pénibilité, en éliminant des tâches répétitives à faible valeur ajoutée.

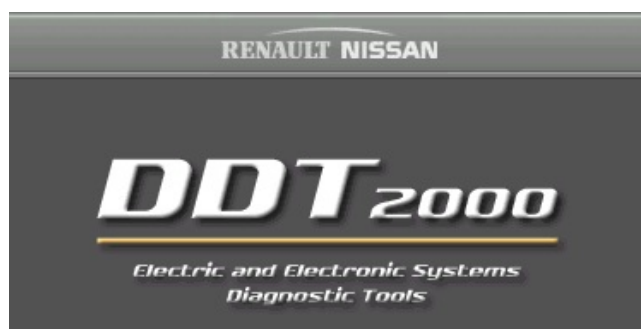
Aspect	Avant (manuel)	Après (automatisé via <code>processingpowersw()</code>)
Accès à la calibration	Recherche manuelle dans <i>Calibration Data</i>	Accès direct avec le nom de la calibration
Identification de la feuille	Deviner ou connaître la feuille d'index selon le SmartMOS	Automatisation via la méthode <code>_smartmos_zone()</code>
Recherche de l'index	Filtrage manuel dans la feuille de zone	Extraction directe par script
Récupération des valeurs	Relevé manuel (valeur, index, résolution)	Retour immédiat des 3 infos (index, valeur, résolution)
Temps moyen par triplet	5 à 10 minutes	Moins de 10 secondes
Erreurs possibles	Risques de lecture, sélection ou copier/coller	Forte réduction des erreurs humaines
Traitement en série	Très lent et fatigant	10 à 20 SmartMOS traités par exécution
Pénibilité de la tâche	Répétitive et fastidieuse	Simplifiée, centrée sur l'analyse

TABLE 4 – Comparatif entre le processus manuel et l'outil PowerSW

5.3 Outil ScreenBuilder : Générateur d'écrans DDT

5.3.1 Introduction générale

Le logiciel **DDT (Diagnostic Data Tool)** est un outil **historique** de Renault, apparu au **début des années 1990** avec l'introduction des premiers systèmes de diagnostic embarqué dans les véhicules, notamment suite à la généralisation de la **prise OBD (On-Board Diagnostic)**. Ce contexte marque l'arrivée progressive de l'électronique et de l'informatique dans l'automobile, avec le besoin croissant de suivre, tester et valider le comportement des composants électroniques embarqués.

FIGURE 9 – Logo de l'outil **DDT2000**, utilisé chez Renault-Nissan pour le diagnostic des systèmes électriques et électroniques

DDT a été conçu pour répondre à ces enjeux. Il s'agit d'un logiciel de **diagnostic interne** à

Renault, utilisé par les équipes d'ingénierie pour :

- simuler et visualiser le comportement de composants comme les **SmartMOS**, fusibles électroniques intelligents intégrés aux véhicules modernes,
- tester des conditions logiques, des états électriques ou des fonctions de sécurité,
- **valider les configurations** en fonction du système électrique du véhicule.

Ce qui rend DDT **particulièrement remarquable**, c'est sa **stabilité** :

- Depuis sa création, **le logiciel en lui-même n'a quasiment pas changé**.
- Ce sont uniquement **les bases de données (fichiers XML)** qui sont mises à jour pour suivre les évolutions des véhicules.

DDT est donc devenu une **référence incontournable** dans les processus de validation électronique chez Renault. Il est **robuste, fiable et universellement adopté** au sein des équipes, sans jamais poser de problèmes de compatibilité ou de performance, même plus de **30 ans** après sa création.

C'est dans ce cadre bien établi que s'inscrivent les travaux liés à la **génération des écrans DDT**, éléments visuels affichant les paramètres clés d'un système électrique donné, et qui seront détaillés dans les sections suivantes.

5.3.2 Contexte et besoin métier

Dans le contexte de l'électrification croissante des véhicules et de la digitalisation des fonctions, **le nombre de composants électroniques embarqués dans un véhicule ne cesse d'augmenter**. Chaque composant doit pouvoir être **testé, surveillé et validé** lors des phases de développement, d'intégration ou de vérification sur bancs de test ou véhicules prototypes.

Le logiciel **DDT**, utilisé en interne chez Renault, permet d'afficher ces composants dans des écrans appelés **écrans DDT**. Ces écrans affichent, sous forme graphique, les *valeurs de courant, états logiques, niveaux d'activation*, etc., pour un composant donné (par exemple un SmartMOS). Ils sont indispensables pour que les ingénieurs puissent :

- comprendre le comportement d'un système électrique,
- valider une nouvelle architecture ou un scénario d'usage,
- identifier des erreurs dans les calibrations ou le câblage,
- assurer la conformité fonctionnelle ou sécuritaire d'un système.

Problème actuel

Jusqu'à présent, la génération de ces écrans DDT se faisait entièrement de manière manuelle :

1. Analyse du fichier *SDE* (référentiel électrique du véhicule),
2. Identification des composants à afficher,
3. Ouverture du générateur XML,
4. Création un à un des éléments graphiques (textes, labels, zones, couleurs...).

Cette méthode était :

- **Chronophage** (jusqu'à 1 jour pour un seul écran simple),
- **Sujette aux erreurs humaines** (copier/coller, mauvais identifiant...),
- **Difficilement réutilisable** pour d'autres projets ou configurations.

L'objectif est de **développer un script Python automatisant la génération des écrans DDT**, à partir du **fichier SDE** fourni au format Excel.

Ce besoin s'applique à plusieurs familles d'écrans :

- **Distribution** : affichage par domaine les fonctions de distributions,
- **OEM** : regroupement par zone véhicule (Front, Cabin, Rear...),
- **Shutdown** : validation des conditions d'endormissement par PIU.

Chaque type fera l'objet d'une section spécifique dans la suite du rapport.

5.3.3 Fonctionnement global du script

Le script de génération automatique des écrans DDT repose sur une architecture modulaire, adaptée au type d'écran à produire (*Distribution*, *Shutdown*, *OEM*). Il traite différents fichiers d'entrée, applique des règles de regroupement métier, puis structure dynamiquement un fichier XML compatible avec le logiciel DDT.

Fichiers d'entrée selon le type d'écran

Type d'écran	Fichiers requis	Utilisation
Distribution	SDE + fichier de mapping	Le fichier SDE contient les SmartMOS ; le fichier de mapping regroupe les fonctions par domaine (<i>ADAS</i> , <i>Architecture</i> , <i>Lighting</i>).
Shutdown	SDE + fichier DOORS [20]	Les SmartMOS sont regroupés par PIU (PIU Master, PIU Mode, PIU Sub). Le fichier DOORS complète la logique métier.
OEM	Aucun fichier requis	Écrans statiques , générés à partir d'un modèle XML pré-enregistré.

TABLE 5 – Types d'écrans et besoins d'entrée

Formulation (groupements) : Soit $\mathcal{S}$ l'ensemble des SmartMOS et $\mathcal{D}$ l'ensemble des domaines fonctionnels. Le mapping induit une application

$$\text{dom} : \mathcal{S} \rightarrow \mathcal{D}, \quad \text{avec des groupes } G_d = \{s \in \mathcal{S} \mid \text{dom}(s) = d\} \text{ pour } d \in \mathcal{D}.$$

Pour *Shutdown*, soit $\mathcal{P} = \{\text{Master}, \text{Mode}, \text{Sub}\}$ et une classification

$$\pi : \mathcal{S} \rightarrow \mathcal{P}, \quad H_p = \{s \in \mathcal{S} \mid \pi(s) = p\}.$$

Étapes principales de traitement

1. Chargement des fichiers :

- Le fichier SDE est analysé avec **pandas** pour extraire les noms de PIU, ID SmartMOS, courants, ASIL, etc.
- Le fichier *mapping* ou *DOORS* est utilisé selon le cas (Distribution ou Shutdown).

Notation : on note S les données issues du SDE et M le fichier métier (*mapping*/DOORS).

2. Filtrage et regroupement :

- Pour les écrans Distribution, les fonctions sont regroupées selon les domaines du mapping.
- Pour les Shutdown, les regroupements se font par type de PIU grâce aux données enrichies du DOORS.
- Les OEM sont générés à partir de blocs XML statiques.

Principe : la partition $\{G_d\}_{d \in \mathcal{D}}$ (ou $\{H_p\}_{p \in \mathcal{P}}$) définit l'ensemble des écrans à générer.

3. Construction dynamique du XML :

- Chaque groupe devient un bloc `<Screen>` structuré avec **lxml.etree**.
- Mise en page automatisée : titres, labels, compteurs, couleurs, tailles, positions.

Modèle de positionnement : pour une liste ordonnée d'éléments $(e_i)_{i=1..n}$ dans un écran,

$$y_i = y_0 + (i - 1) \Delta y, \quad \text{Hauteur écran } H = H_{\text{header}} + n \Delta y + \Delta_{\text{marge}}.$$

Correspondance abstraction $\rightarrow$ XML : si Φ désigne le générateur,

$$\Phi : (S, M) \longmapsto T = \bigsqcup_{g \in \mathcal{G}} \text{Screen}(g),$$

où $\mathcal{G}$ est l'ensemble des groupes (domaines ou PIU) et T l'arbre XML final.

4. Export final :

- L'arborescence XML complète est compilée.
- Le fichier est exporté sous un nom librement défini par l'utilisateur, directement exploitable dans DDT.

Détection automatique des projets véhicules

Une fonctionnalité avancée permet d'identifier dynamiquement les projets véhicules dans le fichier SDE grâce à des marqueurs textuels spécifiques :

- Le mot-clé **vehicle** marque le début d'un projet.
- Le mot-clé **end** marque la fin du périmètre véhicule.

Formalisation : si $(h_j)_{j=1..N}$ est la séquence des en-têtes, on extrait les intervalles

$$I_k = [a_k, b_k] \quad \text{tels que} \quad h_{a_k} = \text{vehicle}, \quad h_{b_k} = \text{end}, \quad a_k < b_k,$$

puis on restreint les données à $\bigcup_k I_k$ ou à un I_{k^*} choisi (ex. *FlexEvan*).

Cela permet de :

- Limiter la génération des écrans à un seul projet (ex. : *FlexEvan*),
- Automatiser le ciblage sans intervention manuelle,
- Générer des écrans spécifiques à chaque projet.

Fichier de sortie

- **Nom** : défini par l'utilisateur (exemple : `screens_distribution.xml`)
- **Format** : fichier au format `.xml`
- **Contenu** : ensemble des écrans DDT prêts à être importés dans le logiciel

Comptage des écrans : si $|\mathcal{G}| = m$, alors le nombre d'écrans générés est

$$N_{\text{screens}} = m \quad (\text{un écran par groupe/domaine ou par PIU}).$$

5.3.4 Les trois cas métiers traités : Cas Distribution

Les écrans **Distribution** représentent visuellement les *SmartMOS* liés aux fonctions de distribution électrique du véhicule, regroupés par **domaine fonctionnel** (*Architecture, ADAS, Lighting...*). Ils facilitent le diagnostic et la validation des architectures, en structurant les données issues du fichier SDE.

Objectif du regroupement

Un fichier SDE peut contenir jusqu'à **500 fonctions SmartMOS**. Sans organisation, retrouver une fonction (ex. clignotants) serait fastidieux.

Le regroupement par **domaine métier** (ex. ExtLighting, ADAS, Architecture) permet de :

- Localiser rapidement une fonction dans son **contexte** (ex. clignotants → ExtLighting),
- Centraliser les informations dans une vue hiérarchisée,
- Limiter les erreurs de lecture,
- Structurer les écrans DDT selon les **logiques projet** Renault.

Fichiers utilisés :

- **SDE (Source de Distribution Électrique)** : IDs SmartMOS, fonctions, courants, ASIL...
- **Mapping** : associe chaque fonction à un domaine (ex. Cooling Fan 1 → Architecture).

Exemple visuel – Domaine *Architecture* :

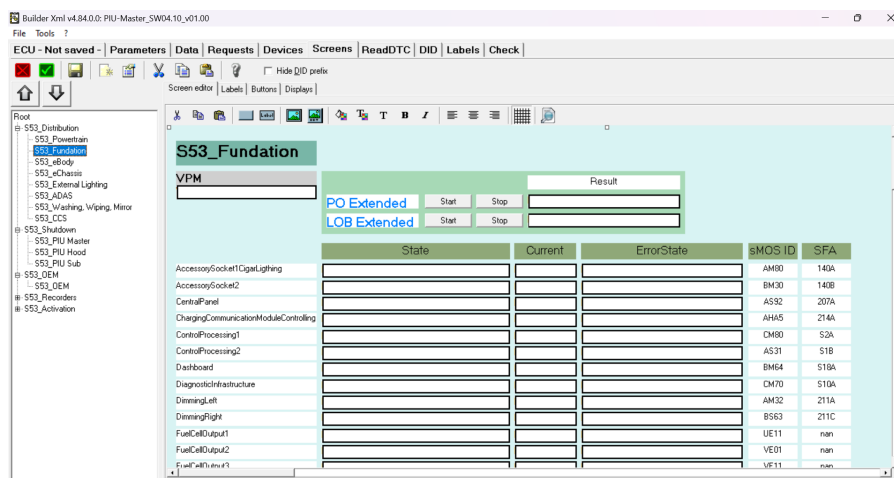


FIGURE 10 – Écran généré pour le domaine S53_Architecture dans DDT2000.

Structure de l'écran :

- **Zone VPM** : état du mode énergétique (Power OFF/ON, Life On Board...),
- **Commandes PO / LOB Extended** : boutons *Start/Stop*,
- **Résult** : champ libre de validation,
- **Tableau principal** : chaque ligne = fonction contrôlée par un SmartMOS :
 - *Nom de fonction* (ex. *AccessorySocket1CigarLighting*),
 - *State*, *Current*, *ErrorState*,
 - *SmartMOS ID* (ex. *AM80*),
 - *SFA* (code de test).

Logique de regroupement (extrait mapping) :

Function Distribution Name	Domain
AccessorySocket1CigarLighting	Architecture
Cooling Fan 1	Architecture
Low Beam Left	Lighting

TABLE 6 – Function–Domain mapping (extrait)

Le script joint le fichier SDE et classe chaque ligne dans le bon écran XML.

Résultat final :

- Génération d'un écran XML par domaine (*S53_Architecture*, *S53_ADAS*, etc.),
- Concaténation des écrans dans un fichier *.xml* final à importer dans DDT2000.

5.3.5 Les trois cas métiers traités : Cas Shutdown

Les écrans **Shutdown** offrent une **vision consolidée de l'endormissement** des unités d'interface de puissance (**PIU**). Contrairement aux écrans Distribution (par domaine fonctionnel), ils sont organisés par **PIU** :

- **PIU Master**
- **PIU Hood**
- **PIU Sub**

Ils identifient les SmartMOS actifs lors de l'endormissement, surveillent les signaux critiques de chaque PIU et aident à analyser les causes de non-endormissement.

Éléments communs avec les écrans Distribution

La partie principale du tableau reprend les informations habituelles :

Colonne	Description
Distribution Function	Fonction associée (ex. : Seat Heater Supply)
State	État du SmartMOS
Error State	Indicateur de défaut
SmartMOS ID	Identifiant unique
SFA Code	Code système pour banc ou diagnostic

TABLE 7 – Colonnes clés d'un écran DDT

Structure et spécificités Shutdown

Chaque écran Shutdown comporte deux zones :

1. **En-tête (header)** : indicateurs globaux liés au PIU,
2. **Tableau SmartMOS** : similaire aux écrans Distribution.

Le contenu de l'en-tête varie selon le PIU :

Type de PIU	Particularités
PIU Master	Le plus complet, gère l'endormissement global
PIU Hood	Quelques indicateurs spécifiques
PIU Sub	En-tête réduit voire vide

TABLE 8 – Variantes d'en-tête selon le PIU

Exemple : écran PIU Master (Shutdown)

L'exemple ci-dessous illustre un écran **S53_PIU Master** dans DDT2000 (mode Shutdown) (voir figure 11).

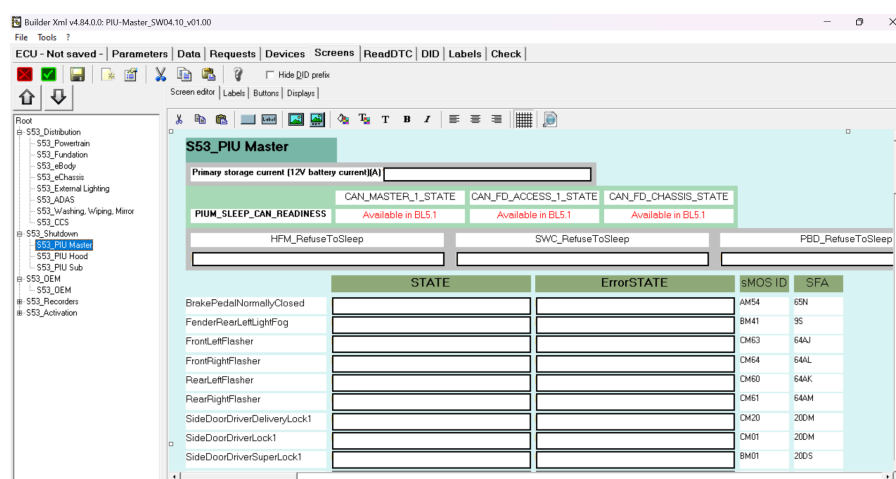


FIGURE 11 – Écran S53_PIU Master dans DDT2000 (mode Shutdown).

Exemples d'indicateurs :

- Primary Storage Current (12V battery current) [A]
- CAN_MASTER_1_STATE, CAN_FD_ACCESS_1_STATE, CAN_FD_CHASSIS_STATE

- HFM_RefuseToSleep, SWC_RefuseToSleep, PBD_RefuseToSleep
- Labels rouges *Available in BL5.1* (ajout temporaire)

Ces signaux montrent visuellement si un composant empêche l'endormissement (ex. HFM_RefuseToSleep)

Règle de regroupement

Les écrans Shutdown sont regroupés par **PIU**. Le script détecte automatiquement les PIU dans le **SDE** et enrichit l'assemblage avec le fichier **DOORS**.

Résultat produit

Pour chaque PIU :

- Un **écran XML** est généré,
- L'ensemble est fusionné dans un fichier **.xml** unique, utilisable dans **DDT**.

5.3.6 Les trois cas métiers traités : Cas OEM

Les **écrans OEM** sont des écrans **statiques**, offrant une **vue synthétique** sur certains éléments propres au mode **OEM**. Contrairement aux écrans **Distribution** et **Shutdown**, ils **ne dépendent d'aucun fichier externe** (SDE, mapping, DOORS) : leur structure est **fixe** et inchangée selon les projets.

Contenu des écrans OEM

Ils regroupent des **éléments de supervision** relatifs aux modes étendus et des indicateurs explicites sur l'état du véhicule.

Élément	Description
Boutons Start / Stop	Pour <i>PO Extended</i> , <i>OE Extended</i> , <i>OEM DINO MODE</i>
Vehicle Power Mode (VPM)	État général du mode de puissance
5 Labels spécifiques	Vérification de signaux critiques OEM

TABLE 9 – Composants clés d'un écran OEM

Exemple visuel

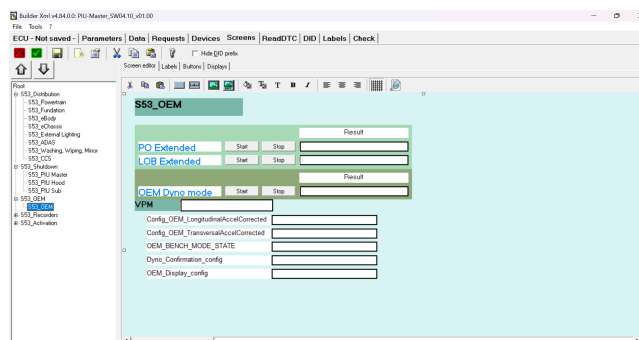


FIGURE 12 – Écran généré pour le mode OEM dans DDT2000.

Construction et stabilité

Ces écrans sont codés **en dur** dans le script, sans parsing ni regroupement :

- Pas de SDE
- Pas de mapping
- Pas de DOORS

Ils sont donc **rapides** à générer et **robustes**, car identiques d'un projet à l'autre.

Utilité métier

Bien que statiques, ils restent **très utiles** pour le **diagnostic** et les **tests OEM**. Les boutons et labels offrent une vérification directe du comportement du véhicule dans ces contextes.

5.3.7 Apports, plus-value et perspectives

L'apport de ce développement dépasse largement l'automatisation d'un fichier : il répond à un **problème structurel majeur de l'ingénierie système** chez Renault.

DDT, **logiciel de diagnostic embarqué** des années 1990, est réputé pour sa **fiabilité** et n'a presque pas évolué depuis 30 ans, hormis sa base de données. Mais cette stabilité a un coût : une interface rigide, vieillissante et **peu ergonomique**.

Créer un écran dans DDT exige :

- **Positionner manuellement** chaque élément, sans grille ni repère,
- **Renseigner à la main chaque identifiant (JID)**, avec risque d'erreur critique,
- Vérifier visuellement l'alignement (labels, compteurs...),
- Recommencer à chaque mise à jour du SDE.

Résultat : un écran complet peut prendre plus d'une semaine.

Le script automatisé permet désormais (voir table 10) :

- **De générer 15 écrans projet** (Distribution, Shutdown, OEM),
- **En moins de 10 secondes**, contre **plusieurs semaines** auparavant,
- Avec **conformité automatique à DDT**, sans erreur de JID ni d'alignement.

Ce n'est pas qu'un gain de productivité, mais un changement de paradigme : ce qui demandait effort et risque est désormais **entièrement industrialisé**.

Calcul du gain de temps

Avant :

- Temps pour générer 15 écrans manuellement $\approx$ **4 semaines** ($\approx$ **20 jours ouvrés**)
- Durée estimée totale : $20 \times 7 = 140$ heures

Après :

- Génération automatique : $\approx$ **10 secondes**
- Avec validation et relecture : $\approx$ **1 heure**

Gain de temps brut :

$$\text{Gain} = \left(\frac{140 - 1}{140} \right) \times 100 = \boxed{99,29 \%} \quad (3)$$

Critère	Avant (manuel)	Après (script automatisé)
Temps pour 1 écran	5 à 7 jours	~1 seconde
Temps pour 15 écrans	3 à 4 semaines	< 10 secondes
Précision des éléments	Risque d'erreur (JID, alignement)	100% fiable et standardisé
Pénibilité	Très élevée (souris, copier/coller)	Nulle
Maintenance après MAJ du SDE	Refaire tous les écrans	Régénérer automatiquement
Scalabilité (autres véhicules)	Très faible	Forte : tout projet documenté dans le SDE est exploitable

TABLE 10 – Gains apportés par la génération automatique d'écrans DDT

Perspectives stratégiques et industrielles

- **Réutilisable multi-projets** — Architecture modulaire indépendante : génère des écrans pour tout véhicule dès qu'un SDE est disponible (**portabilité totale**).
- **Industrialisation CI/CD** — Intégration en pipeline : à chaque version SDE validée, génération et tests automatiques des écrans.
- **Mises à jour économes** — Un simple rechargement du SDE régénère les écrans ; **centaines d'heures économisées** sur un cycle projet.
- **Valorisation (brevet)** — Génération automatique + regroupement métier : innovation interne en cours d'évaluation.
- **Adoption interne** — Fort intérêt des équipes ; solution répliquable vers d'autres projets et divisions.

☀ Remarque

La pose d'un brevet a été suggérée par **Séverine RAUSZ, CDS Architecture Transversale et Fondation**, à la suite de la démonstration du script. Mon stage se terminant avant la finalisation administrative, je ne pourrai pas **assister au dépôt officiel** ni suivre les suites.

5.4 ENSOFT : Infrastructure graphique centralisée

5.4.1 Objectifs et justification

Dans un environnement industriel complexe comme Renault, les outils d'automatisation doivent être accessibles à divers profils : ingénieurs systèmes, architectes, responsables produit, chefs de projet. Or, si l'exécution de scripts Python bruts convient aux développeurs, elle reste une contrainte pour la majorité des utilisateurs métiers.

🖥 Objectif global de l'interface ENSOFT

Offrir une plateforme graphique unique, utilisable par tous (développeurs ou non), pour exploiter efficacement les outils d'automatisation du stage, réduire les erreurs humaines et faciliter la répliquabilité.

L'interface **ENSOFT** vise à **centraliser** les outils développés dans une plateforme ergonomique et **facile à déployer** sur les postes internes, notamment :

- le **générateur automatique d'écrans DDT** (*ScreenBuilder*),
- l'outil d'**extraction des calibrations LEONI** (*PowerSW / APSW Calibre*),
- l'outil de **vérification des protections câblage** (*Wire Protection Calib*).

ENSOFT répond à plusieurs enjeux :

- **Faciliter l'adoption** par des non-développeurs,
- **Réduire les erreurs humaines**,
- **Structurer les usages** dans une interface claire,
- **Standardiser les interactions** avec les fichiers critiques (SDE, calibrations Leoni, écrans DDT),
- **Améliorer traçabilité et répliquabilité** via guides et aides visuelles.

Le choix d'une interface graphique plutôt qu'un simple `.exe` s'est imposé après échanges avec les ingénieurs. ENSOFT devient ainsi **un pont entre la complexité du code et les besoins opérationnels**.

5.4.2 Choix techniques

L'interface **ENSOFT** a été conçue autour de trois enjeux : **ergonomie**, **portabilité** et **intégration** fluide des outils existants.

Framework choisi : CustomTkinter

CustomTkinter (extension moderne de Tkinter [18]) a été retenu pour :

- compatibilité Python *sans dépendances lourdes* ;
- design moderne (widgets arrondis, couleurs, *dark mode*) ;
- mise en œuvre simple et maintenance aisée par d'autres ingénieurs.

Architecture modulaire

Chaque outil est encapsulé et piloté par un fichier principal `main.py` (navigation, routage, affichage dynamique).

- Les modules (*ScreenBuilder*, *PowerSW/APSW Calibre*, *Wire Protection Calib*) sont regroupés dans une **classe dédiée** (UI, actions : chargement, exécution, export, aide).

Gestion de thème et accessibilité

- Deux modes : **clair** (lisibilité) et **sombre** (confort, pratique IDE) ;
- thème par défaut : **lavande**, retenu après retours utilisateurs.

Déploiement et exécutable

Packaging en `.exe` (p. ex. *pyinstaller* [17]) pour un lancement **sans Python** sur tout poste Windows Renault.

- installation **simplifiée** pour les non-techniques ;
- diffusion **rapide** en interne ;
- **stabilité** : code figé dans l'exécutable.

✓ Avantages clés de l'approche technique

- **Autonomie** : pas besoin de Python pour utiliser les outils ;
- **UX améliorée** : interface intuitive, moderne, mode sombre ;
- **Modularité** : extensions et ajouts facilités ;
- **Maintenance** : séparation claire logique / interface / navigation ;
- **Déploiement fiable** : exécutable unique, utilisable partout.

5.4.3 Page d'accueil

Au lancement, l'application **ENSOFT** affiche une **page d'accueil bilingue**, conçue pour orienter l'utilisateur dès les premiers instants (voir figure 13).

Objectifs

- **Présenter** l'application et ses principales fonctions,
- **Faciliter la navigation** via des onglets,
- **Assurer l'accessibilité** avec une description FR/EN de chaque module.



FIGURE 13 – Page d'accueil ENSOFT (mode clair, thème lavande)

Structure

La page comprend :

- Une **barre d'onglets** avec 4 entrées :
 - Welcome,
 - DDT2000 ScreenBuilder,
 - Wire Protection Calib,
 - Leoni Processing.
- Une **zone centrale** présentant chaque module avec traduction :
 - **DDT2000 ScreenBuilder** : Génère des écrans XML conformes au SDE.
Generates compliant XML screens for DDT2000.
 - **Wire Protection Calib** : Crée automatiquement le fichier APSW_Calib.xlsx avec filtrage et rapports.
Automates APSW calibration file creation with filtering and reporting.
 - **Leoni Processing** : Extrait les paramètres de calibration d'un fichier Excel Leoni.
Extracts calibration values from Leoni Excel files.

Design et accessibilité

- Thème **lavande** par défaut,
- Option **Dark Mode**,
- Design sobre et espacé pour une lecture fluide.

5.4.4 Module ScreenBuilder

Le module **DDT2000 ScreenBuilder**, élément central de l'interface *EnSoft*, génère automatiquement les écrans DDT à partir du *SDE*, du fichier de *mapping*, des fichiers *DOORS* et d'un gabarit *XML* [4] conforme aux standards Renault (voir figure 14).

Objectif principal

Simplifier un processus jusqu'alors manuel, long et sujet à erreurs. Grâce à une interface intuitive, la génération d'écrans complexes est réalisable en quelques clics, tout en garantissant la compatibilité avec *DDT2000*.

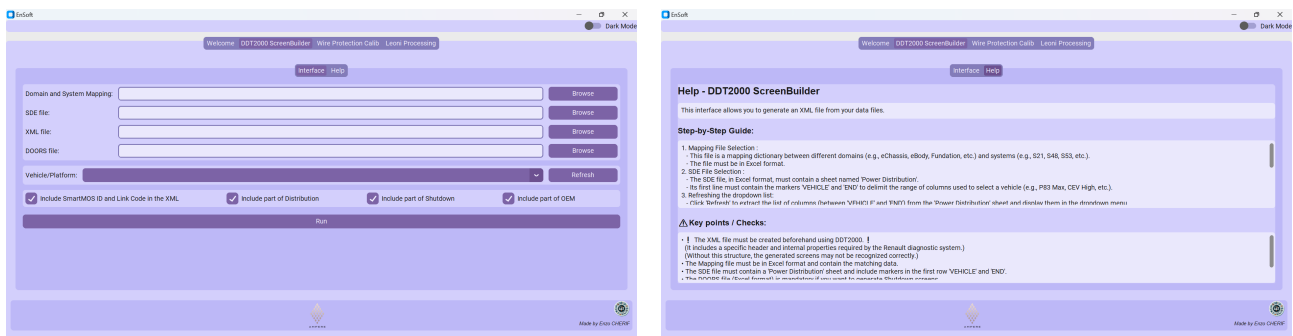


FIGURE 14 – Interface principale et aide du module *ScreenBuilder*

Interface utilisateur

L'onglet *ScreenBuilder* est structuré en quatre zones :

- **Chargement des fichiers :**
 - *Domain and System Mapping* : correspondance domaines–systèmes,
 - *SDE* : feuille *Power Distribution*,
 - *XML* : gabarit DDT2000,
 - *DOORS* : requis pour les écrans Shutdown.
- **Sélection du véhicule :** menu déroulant pour choisir la plateforme (e.g. FlexEvan), bouton *Refresh* pour mettre à jour selon le SDE.
- **Options :** cases à cocher pour inclure SmartMOS, Distribution, Shutdown, OEM.
- **Exécution :** bouton *Run* lançant la génération du fichier XML final.

Aide intégrée

Un onglet *Help* fournit :

- Un tutoriel pas à pas,
- Des rappels de contraintes (structure XML, formats attendus...),
- Des précautions pour garantir la validité des écrans.

5.4.5 Module Wire Protection Calib

Le module **Wire Protection Calib** automatise la validation des protections câblage (courbes i^2t [24]) dans le dimensionnement électrique. Il confronte les courants de fonctionnement du fichier *SDE* aux courbes de protection BTG ou VNF, garantissant la cohérence entre composants et protections (voir figure 15).

Objectif

- Comparer les courants SmartMOS aux limites des courbes,
- Détecter les cas de sous- ou sur-dimensionnement,
- Générer un rapport de calibration structuré par PIU.

Interface

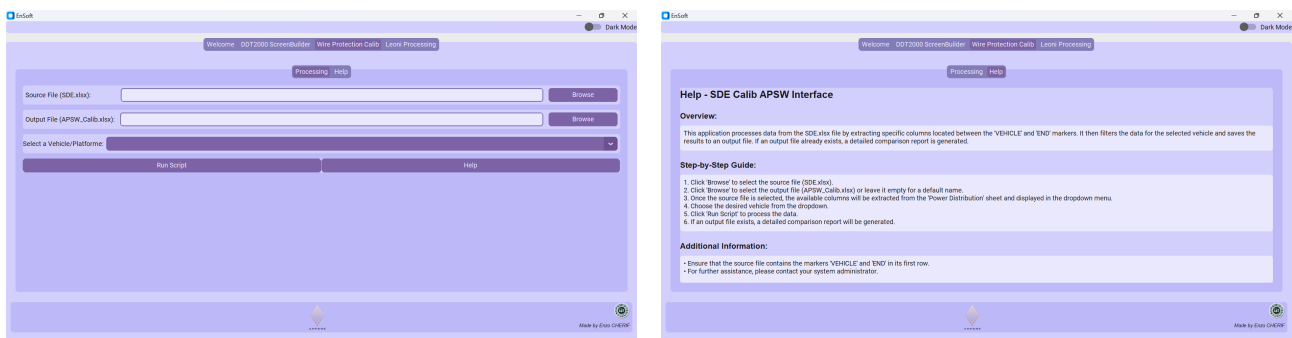


FIGURE 15 – Interface principale et aide intégrée — Wire Protection Calib

L'interface comporte trois champs :

- **Source File (SDE.xlsx)** : données d'entrée,
- **Output File (APSW_Calib.xlsx)** : résultats (nom libre ou par défaut),
- **Vehicle/Platforme** : sélection dynamique de la plateforme.

L'action s'exécute via **Run Script**. Un bouton **Help** fournit une documentation intégrée.

Aide

L'onglet **Help** propose :

- Un guide pas à pas,
- Des remarques sur la structure des fichiers,
- Les contraintes liées aux marqueurs **VEHICLE** et **END**.

Fonctionnement

1. Parcours du **SDE.xlsx** entre **VEHICLE** et **END**,
2. Filtrage des lignes selon la plateforme choisie,
3. Confrontation des courants aux courbes de protection,
4. Génération d'un rapport en cas d'incohérence (i^2t dépassé...).

5.4.6 Module PowerSW

La dernière interface de l'application centralisée permet d'exploiter le **fichier dictionnaire Leoni** utilisé pour les calibrations PowerSW. Elle **automatise l'extraction** des valeurs d'index, des plages de valeurs et de la résolution liées à une adresse SmartMOS donnée, selon le SWC et la calibration choisies.

Cette opération, auparavant manuelle sous Excel, est désormais **simplifiée par une interface graphique réactive**, réduisant le temps de traitement et les erreurs.

Fonctionnalités

L'utilisateur peut :

- **Charger** un dictionnaire Leoni (.xlsx),
- **Sélectionner** un SWC disponible,
- **Choisir** une calibration liée,
- **Saisir** une adresse SmartMOS (BM63, LLDD...),
- **Obtenir automatiquement** :
 - l'index de la valeur,
 - les valeurs possibles,
 - la résolution.

Exemple

Dans l'exemple ci-dessous (voir figure 16), l'utilisateur charge un dictionnaire, sélectionne Swc_SwitchCtrl, la calibration Populated, puis l'adresse BM63. Après clic sur Get, les champs Index value, Values et Resolution sont remplis.

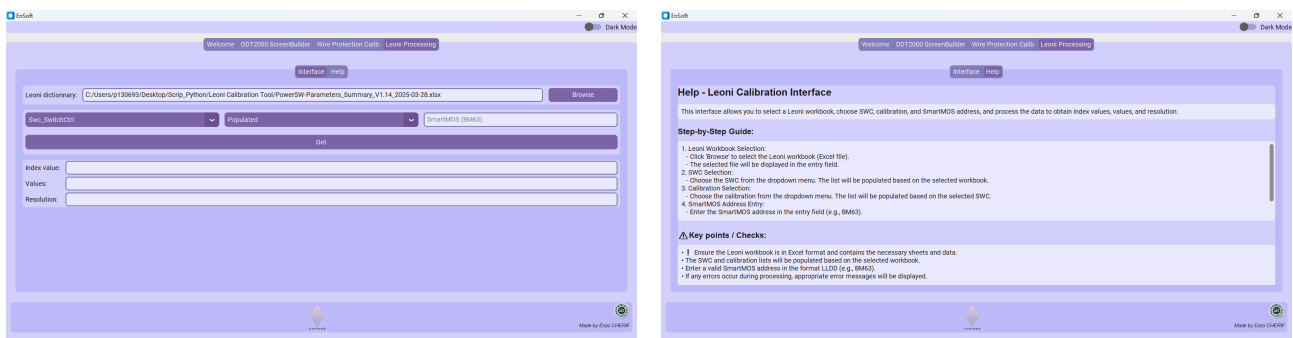


FIGURE 16 – Interface principale et aide intégrée — Leoni Processing

Aide intégrée

Un onglet **Help** guide l'utilisateur :

- sur l'ordre des actions,
- les formats attendus,
- et les vérifications en cas d'erreur.

5.4.7 Accessibilité et déploiement

Pour garantir une utilisation simple et universelle, l'application a été **compilée en exécutable .exe** via **pyinstaller**.

Déploiement

- **Pas d'installation Python requise** : toutes les dépendances sont embarquées.
- **Compatible avec les postes Windows Renault** : exécution directe sans configuration.
- **Portabilité** : aucun droit administrateur nécessaire.

Accessibilité

- Interface **ergonomique** adaptée aux non-développeurs,
- Bouton **Help** sur chaque onglet avec guide pas-à-pas,
- Messages d'erreur **clairs et contextualisés**.

Cette approche garantit une **adoption rapide** et un usage fluide, quel que soit le profil utilisateur.

5.4.8 Design et retour utilisateurs

Le développement de l'interface a suivi une démarche itérative intégrant **les retours réguliers des utilisateurs finaux** (architectes fonctionnels, ingénieurs système), recueillis lors de démonstrations, tests réels et échanges directs, afin d'adapter progressivement l'ergonomie et les fonctionnalités.

Retours intégrés

- Champs de saisie ajustés pour **limiter les erreurs** (ex. menus déroulants dynamiques),
- Ajout de boutons **Help** contextuels,
- Disposition optimisée pour **faciliter les actions répétitives** et **accélérer le traitement**.

Identité visuelle et confort

- Thème **lavande** favorisant le confort visuel,
- **Mode sombre** intégré (Figure 17) pour réduire la fatigue oculaire,
- Navigation fluide, sans menus superflus.

Cette attention portée à l'expérience utilisateur favorise une **adoption rapide et durable**, en rendant l'outil à la fois accessible et agréable.

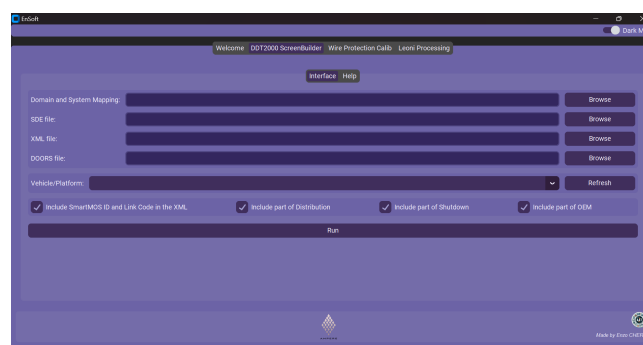


FIGURE 17 – Interface en mode sombre

5.5 Génération automatique des écrans Recorder

5.5.1 Objectifs et contexte

Les écrans **Recorder** ont pour objectif de collecter et présenter, sous forme structurée, des **informations internes au PIU** (*Power Interface Unit*) à des fins de **diagnostic véhicule**. Ces données sont essentielles pour :

- Les **équipes internes Renault** durant la phase de conception,
- Les **flying-docs** et les **valideurs** lors des phases de développement,
- Les **ingénieurs** mobilisés lors de pannes nécessitant une expertise avancée.

Ces écrans sont destinés **exclusivement à un usage interne** : les données qu'ils contiennent sont complexes et nécessitent une interprétation technique approfondie.

Contrairement aux écrans de type **Distribution** ou **Shutdown**, les Recorder ne dépendent d'aucun fichier externe (SDE, mapping, etc.). Une fois leur structure validée, ils sont **stables dans le temps** et ne nécessitent pas de mise à jour à chaque évolution du projet.

La génération se fait via un **script Python simple** :

- Pas d'interface graphique,
- Logique centrée sur la création rapide et fiable des écrans,
- Organisation prédéfinie par catégories et tranches pour optimiser la lisibilité et la stabilité dans DDT.

5.5.2 Organisation des Recorders

Les **Recorders** regroupent sept catégories d'écrans affichant des informations internes au PIU (*Power Interface Unit*) pour le diagnostic. Chaque catégorie a un objectif précis et utilise un découpage en **tranches** afin d'optimiser lisibilité et performance dans DDT.

1. LowSOC Recorder Suivi des événements liés à un faible *State of Charge*, utile pour diagnostiquer les décharges critiques de batterie (voir Figure 18). Tranches : 0–19, 20–39.

2. SDE Recorder Capture des événements d'arrêt système (*Shutdown Events*) pour analyser les causes d'arrêt (normal ou non) (voir Figure 18). Tranches : 0–19, ... jusqu'à 240–243.

3. Wakeup Source Recorder Liste les sources de réveil pour distinguer un réveil attendu (ex. action utilisateur) d'un réveil parasite (voir Figure 19). Tranches : 0–24, 25–49.

4. Refuse to Sleep Recorder Enregistre les causes empêchant la mise en veille, afin d'identifier les modules ou signaux bloquants (voir Figure 19). Tranches : 0–19, 20–44.

5. Unwanted Wakeup Source Recorder Recense les réveils non souhaités pour détecter les perturbations énergétiques (voir Figure 20). Tranches : 0–24, 25–49.

6. Unballast Recorder Suit les désactivations automatiques de charges électriques (délestages pour protection ou économie d'énergie) (voir Figure 20). Tranche : 0–19.

<pre> 00 diag_powerlock_lowsoc_recorder_id DPLR_id_00 diag_powerlock_lowsoc_recorder_mapping_version DPLRM_version_00 diag_powerlock_lowsoc_s10_distance_totalizer D_PL_LS_s10_DT_00 diag_powerlock_lowsoc_s22_vehicle_etc_time D_PL_LS_s22_V_UTC_T_Part1_00 diag_powerlock_lowsoc_s22_vehicle_etc_time D_PL_LS_s22_V_UTC_T_Part2_00 diag_powerlock_lowsoc_s53_vehicle_power_mode D_PL_LS_s53_VPM_00 diag_powerlock_lowsoc_s27_fo_primary_storage_ovc_ecu_raw D_PL_LS_s27_FPSOER_00 (V) diag_powerlock_lowsoc_s27_fo_primary_storage_voltage_1 D_PL_LS_s27_FPSV_1_00 (V) diag_powerlock_lowsoc_hv_battery_soc D_PL_LS_HV_BS_00 diag_powerlock_lowsoc_eem_staticpower_limitation_request D_PL_LS_EEM_SLR_00 diag_powerlock_lowsoc_s27_voltage_regulation_status D_PL_LS_s27_VRS_00 diag_powerlock_lowsoc_s27_producer_output_current D_PL_LS_s27_POC_00 (A) diag_powerlock_lowsoc_s27_primary_storage_absolute_gauge D_PL_LS_s27_PSAQ_00 diag_powerlock_lowsoc_s20_storage_hv_power_connection D_PL_LS_s20_S_HV_PC_00 diag_powerlock_lowsoc_status D_PL_LS_S_Part1_00 diag_powerlock_lowsoc_status D_PL_LS_S_Part2_00 </pre>	<pre> 00 diag_maximal_current D_M_C_00 (A) diag_total_charge D_T_C_00 diag_total_charge_smartmos_over_threshold_1 D_T_C_SOT_1_00 diag_total_charge_smartmos_over_threshold_2 D_T_C_SOT_2_00 diag_total_charge_smartmos_over_threshold_3 D_T_C_SOT_3_00 diag_total_charge_smartmos_over_threshold_4 D_T_C_SOT_4_00 diag_total_charge_smartmos_over_threshold_5 D_T_C_SOT_5_00 diag_total_charge_smartmos_over_threshold_6 D_T_C_SOT_6_00 diag_sde_rec_total_time_smartmos_closed D_S_R_T_T_S_C_00 diag_sde_rec_total_time_smartmos_closed_over_threshold_1 D_T_T_S_C_O_T_1_00 diag_total_time_smartmos_closed_over_threshold_2 D_T_T_S_C_O_T_2_00 diag_total_time_smartmos_closed_over_threshold_3 D_T_T_S_C_O_T_3_00 diag_total_time_smartmos_closed_over_threshold_4 D_T_T_S_C_O_T_4_00 diag_total_time_smartmos_closed_over_threshold_5 D_T_T_S_C_O_T_5_00 diag_total_time_smartmos_closed_over_threshold_6 D_T_T_S_C_O_T_6_00 </pre>
<pre> 01 diag_powerlock_lowsoc_recorder_id DPLR_id_01 diag_powerlock_lowsoc_recorder_mapping_version DPLRM_version_01 diag_powerlock_lowsoc_s10_distance_totalizer D_PL_LS_s10_DT_01 diag_powerlock_lowsoc_s22_vehicle_etc_time D_PL_LS_s22_V_UTC_T_Part1_01 diag_powerlock_lowsoc_s22_vehicle_etc_time D_PL_LS_s22_V_UTC_T_Part2_01 diag_powerlock_lowsoc_s53_vehicle_power_mode D_PL_LS_s53_VPM_01 diag_powerlock_lowsoc_s27_fo_primary_storage_ovc_ecu_raw D_PL_LS_s27_FPSOER_01 (V) diag_powerlock_lowsoc_s27_fo_primary_storage_voltage_1 D_PL_LS_s27_FPSV_1_01 (V) diag_powerlock_lowsoc_hv_battery_soc D_PL_LS_HV_BS_01 diag_powerlock_lowsoc_eem_staticpower_limitation_request D_PL_LS_EEM_SLR_01 diag_powerlock_lowsoc_s27_voltage_regulation_status D_PL_LS_s27_VRS_01 diag_powerlock_lowsoc_s27_producer_output_current D_PL_LS_s27_POC_01 (A) diag_powerlock_lowsoc_s27_primary_storage_absolute_gauge D_PL_LS_s27_PSAQ_01 diag_powerlock_lowsoc_s20_storage_hv_power_connection D_PL_LS_s20_S_HV_PC_01 diag_powerlock_lowsoc_status D_PL_LS_S_Part1_01 diag_powerlock_lowsoc_status D_PL_LS_S_Part2_01 </pre>	<pre> 01 diag_maximal_current D_M_C_01 (A) diag_total_charge D_T_C_01 diag_total_charge_smartmos_over_threshold_1 D_T_C_SOT_1_01 diag_total_charge_smartmos_over_threshold_2 D_T_C_SOT_2_01 diag_total_charge_smartmos_over_threshold_3 D_T_C_SOT_3_01 diag_total_charge_smartmos_over_threshold_4 D_T_C_SOT_4_01 diag_total_charge_smartmos_over_threshold_5 D_T_C_SOT_5_01 diag_total_charge_smartmos_over_threshold_6 D_T_C_SOT_6_01 diag_sde_rec_total_time_smartmos_closed D_S_R_T_T_S_C_01 diag_sde_rec_total_time_smartmos_closed_over_threshold_1 D_T_T_S_C_O_T_1_01 diag_total_time_smartmos_closed_over_threshold_2 D_T_T_S_C_O_T_2_01 </pre>
<pre> 02 diag_powerlock_lowsoc_recorder_id DPLR_id_02 diag_powerlock_lowsoc_recorder_mapping_version DPLRM_version_02 diag_powerlock_lowsoc_s10_distance_totalizer D_PL_LS_s10_DT_02 diag_powerlock_lowsoc_s22_vehicle_etc_time D_PL_LS_s22_V_UTC_T_Part1_02 diag_powerlock_lowsoc_s22_vehicle_etc_time D_PL_LS_s22_V_UTC_T_Part2_02 </pre>	<pre> 02 diag_refuse_to_sleep_recorder_id D_RTS_R_ID_02 diag_refuse_to_sleep_recorder_mapping_version D_RTS_R_M_V_02 diag_refuse_to_sleep_time_counter D_RTS_T_C_02 diag_refuse_to_sleep D_RTS_02 (min) diag_refuse_to_sleep2 D_RTS2_02 (min) diag_refuse_to_sleep3 D_RTS3_02 (min) diag_refuse_to_sleep4 D_RTS4_02 (min) diag_refuse_to_sleep_s10_distance_totalizer D_RTS_s10_D_T_02 diag_refuse_to_sleep_s22_vehicle_etc_time D_RTS_s22_V_UTC_T_Part1_02 diag_refuse_to_sleep_s22_vehicle_etc_time D_RTS_s22_V_UTC_T_Part2_02 diag_refuse_to_sleep_s27_fo_primary_storage_ovc_ecu_raw D_RTS_s27_FPSOER_02 (V) diag_refuse_to_sleep_s27_fo_primary_storage_voltage_1 D_RTS_s27_FPSV_1_02 (V) diag_refuse_to_sleep_s53_vehicle_power_mode D_RTS_s53_VPM_02 </pre>

(a) LowSOC Recorder

(b) SDE Recorder

FIGURE 18 – Exemples d'écrans Recorder : (a) LowSOC, (b) SDE.

<pre> 00 diag_wakeup_source_recorder_id D_WS_R_ID_00 diag_wakeup_source_s10_distance_totalizer D_WS_s10_D_T_00 diag_wakeup_source_s22_vehicle_etc_time D_WS_s22_V_UTC_T_Part1_00 diag_wakeup_source_s22_vehicle_etc_time D_WS_s22_V_UTC_T_Part2_00 diag_wakeup_source_s27_fo_primary_storage_ovc_ecu_raw D_WS_s27_FPSOER_00 (V) diag_wakeup_source_s27_fo_primary_storage_voltage_1 D_WS_s27_FPSV_1_00 (V) diag_wakeup_source_s53_vehicle_power_mode D_WS_s53_VPM_00 diag_wakeup_source_rec_mapping_version D_WS_R_M_V_00 diag_wakeup_trigger_detection_status D_WS_T_D_S_Part1_00 diag_wakeup_trigger_detection_status D_WS_T_D_S_Part2_00 </pre>	<pre> 00 diag_refuse_to_sleep_recorder_id D_RTS_R_ID_00 diag_refuse_to_sleep_recorder_mapping_version D_RTS_R_M_V_00 diag_refuse_to_sleep_time_counter D_RTS_T_C_00 diag_refuse_to_sleep D_RTS_00 (min) diag_refuse_to_sleep2 D_RTS2_00 (min) diag_refuse_to_sleep3 D_RTS3_00 (min) diag_refuse_to_sleep4 D_RTS4_00 (min) diag_refuse_to_sleep_s10_distance_totalizer D_RTS_s10_D_T_00 diag_refuse_to_sleep_s22_vehicle_etc_time D_RTS_s22_V_UTC_T_Part1_00 diag_refuse_to_sleep_s22_vehicle_etc_time D_RTS_s22_V_UTC_T_Part2_00 diag_refuse_to_sleep_s27_fo_primary_storage_ovc_ecu_raw D_RTS_s27_FPSOER_00 (V) diag_refuse_to_sleep_s27_fo_primary_storage_voltage_1 D_RTS_s27_FPSV_1_00 (V) diag_refuse_to_sleep_s53_vehicle_power_mode D_RTS_s53_VPM_00 </pre>
<pre> 01 diag_wakeup_source_recorder_id D_WS_R_ID_01 diag_wakeup_source_s10_distance_totalizer D_WS_s10_D_T_01 diag_wakeup_source_s22_vehicle_etc_time D_WS_s22_V_UTC_T_Part1_01 diag_wakeup_source_s22_vehicle_etc_time D_WS_s22_V_UTC_T_Part2_01 diag_wakeup_source_s27_fo_primary_storage_ovc_ecu_raw D_WS_s27_FPSOER_01 (V) diag_wakeup_source_s27_fo_primary_storage_voltage_1 D_WS_s27_FPSV_1_01 (V) diag_wakeup_source_s53_vehicle_power_mode D_WS_s53_VPM_01 diag_wakeup_source_rec_mapping_version D_WS_R_M_V_01 diag_wakeup_trigger_detection_status D_WS_T_D_S_Part1_01 diag_wakeup_trigger_detection_status D_WS_T_D_S_Part2_01 </pre>	<pre> 01 diag_refuse_to_sleep_recorder_id D_RTS_R_ID_01 diag_refuse_to_sleep_recorder_mapping_version D_RTS_R_M_V_01 diag_refuse_to_sleep_time_counter D_RTS_T_C_01 diag_refuse_to_sleep D_RTS_01 (min) diag_refuse_to_sleep2 D_RTS2_01 (min) diag_refuse_to_sleep3 D_RTS3_01 (min) diag_refuse_to_sleep4 D_RTS4_01 (min) diag_refuse_to_sleep_s10_distance_totalizer D_RTS_s10_D_T_01 diag_refuse_to_sleep_s22_vehicle_etc_time D_RTS_s22_V_UTC_T_Part1_01 diag_refuse_to_sleep_s22_vehicle_etc_time D_RTS_s22_V_UTC_T_Part2_01 diag_refuse_to_sleep_s27_fo_primary_storage_ovc_ecu_raw D_RTS_s27_FPSOER_01 (V) diag_refuse_to_sleep_s27_fo_primary_storage_voltage_1 D_RTS_s27_FPSV_1_01 (V) diag_refuse_to_sleep_s53_vehicle_power_mode D_RTS_s53_VPM_01 </pre>
<pre> 02 diag_wakeup_source_recorder_id D_WS_R_ID_02 diag_wakeup_source_s10_distance_totalizer D_WS_s10_D_T_02 diag_wakeup_source_s22_vehicle_etc_time D_WS_s22_V_UTC_T_Part1_02 diag_wakeup_source_s22_vehicle_etc_time D_WS_s22_V_UTC_T_Part2_02 diag_wakeup_source_s27_fo_primary_storage_ovc_ecu_raw D_WS_s27_FPSOER_02 (V) diag_wakeup_source_s27_fo_primary_storage_voltage_1 D_WS_s27_FPSV_1_02 (V) diag_wakeup_source_s53_vehicle_power_mode D_WS_s53_VPM_02 diag_wakeup_source_rec_mapping_version D_WS_R_M_V_02 diag_wakeup_trigger_detection_status D_WS_T_D_S_Part1_02 diag_wakeup_trigger_detection_status D_WS_T_D_S_Part2_02 </pre>	<pre> 02 diag_refuse_to_sleep_recorder_id D_RTS_R_ID_02 diag_refuse_to_sleep_recorder_mapping_version D_RTS_R_M_V_02 diag_refuse_to_sleep_time_counter D_RTS_T_C_02 diag_refuse_to_sleep D_RTS_02 (min) diag_refuse_to_sleep2 D_RTS2_02 (min) diag_refuse_to_sleep3 D_RTS3_02 (min) diag_refuse_to_sleep4 D_RTS4_02 (min) diag_refuse_to_sleep_s10_distance_totalizer D_RTS_s10_D_T_02 diag_refuse_to_sleep_s22_vehicle_etc_time D_RTS_s22_V_UTC_T_Part1_02 diag_refuse_to_sleep_s22_vehicle_etc_time D_RTS_s22_V_UTC_T_Part2_02 diag_refuse_to_sleep_s27_fo_primary_storage_ovc_ecu_raw D_RTS_s27_FPSOER_02 (V) diag_refuse_to_sleep_s27_fo_primary_storage_voltage_1 D_RTS_s27_FPSV_1_02 (V) diag_refuse_to_sleep_s53_vehicle_power_mode D_RTS_s53_VPM_02 </pre>
<pre> 03 diag_wakeup_source_recorder_id D_WS_R_ID_03 diag_wakeup_source_s10_distance_totalizer D_WS_s10_D_T_03 diag_wakeup_source_s22_vehicle_etc_time D_WS_s22_V_UTC_T_Part1_03 diag_wakeup_source_s22_vehicle_etc_time D_WS_s22_V_UTC_T_Part2_03 diag_wakeup_source_s27_fo_primary_storage_ovc_ecu_raw D_WS_s27_FPSOER_03 (V) </pre>	<pre> 03 diag_refuse_to_sleep_recorder_id D_RTS_R_ID_03 diag_refuse_to_sleep_recorder_mapping_version D_RTS_R_M_V_03 diag_refuse_to_sleep_time_counter D_RTS_T_C_03 diag_refuse_to_sleep D_RTS_03 (min) diag_refuse_to_sleep2 D_RTS2_03 (min) diag_refuse_to_sleep3 D_RTS3_03 (min) diag_refuse_to_sleep4 D_RTS4_03 (min) diag_refuse_to_sleep_s10_distance_totalizer D_RTS_s10_D_T_03 </pre>

(a) Wakeup Source Recorder

(b) Refuse to Sleep Recorder

FIGURE 19 – Exemples d'écrans Recorder : (a) Wakeup Source, (b) Refuse to Sleep.

7. Power Lock Timeout Recorder Enregistre les expirations de verrouillage d'alimentation afin d'identifier les délais anormaux (voir Figure 21). Tranches : 0–24, 25–49.

Justification du découpage Le fractionnement en tranches répond à trois contraintes :

1. **Performance** : éviter le ralentissement ou le blocage DDT,
2. **Lisibilité** : garantir un affichage clair,
3. **Fiabilité** : limiter les plantages liés à des écrans trop chargés.

5.5.3 Processus de génération

La génération des *Recorders* repose sur un **script Python** (`script_simple_creation_ecran.py`) qui automatise la création des écrans statiques dans DDT, sans dépendance au SDE (voir Figure 22).

diag_unwantedwakeup_source_record_id D_UWS_S_R_I_00	65535
diag_unwantedwakeup_source_s10_distance_totalizer D_UWS_s10_D_T_00	ASASASAS
diag_unwantedwakeup_source_s27_fo_primary_storage_occupancy_raw D_UWS_s27_FPSOER	655 35
diag_unwantedwakeup_source_s27_fo_primary_storage_voltage_1 D_UWS_s27_FPSV_1_00	655 35
diag_unwantedwakeup_source_s53_vehicle_power_mode D_UWS_s53_VPM_00	LIFE ON BOARD
diag_unwantedwakeup_source_rec_mapping_version D_UWS_R_M_V_00	255
diag_unwantedwakeup_trigger_detection_status D_UWW_T_D_S_Part1_00	ASASASAS
diag_unwantedwakeup_trigger_detection_status D_UWW_T_D_S_Part2_00	ASASASAS
diag_unwantedwakeup_source_s22_vehicle_utc_time D_UWS_s22_V_UTC_T_Part1_00	ASASASAS
diag_unwantedwakeup_source_s22_vehicle_utc_time D_UWS_s22_V_UTC_T_Part2_00	ASASASAS

diag_unwantedwakeup_source_record_id D_UWS_S_R_I_01	65535
diag_unwantedwakeup_source_s10_distance_totalizer D_UWS_s10_D_T_01	ASASASAS
diag_unwantedwakeup_source_s27_fo_primary_storage_occupancy_raw D_UWS_s27_FPSOER	655 35
diag_unwantedwakeup_source_s27_fo_primary_storage_voltage_1 D_UWS_s27_FPSV_1_01	655 35
diag_unwantedwakeup_source_s53_vehicle_power_mode D_UWS_s53_VPM_01	LIFE ON BOARD
diag_unwantedwakeup_source_rec_mapping_version D_UWS_R_M_V_01	255
diag_unwantedwakeup_trigger_detection_status D_UWW_T_D_S_Part1_01	ASASASAS
diag_unwantedwakeup_trigger_detection_status D_UWW_T_D_S_Part2_01	ASASASAS
diag_unwantedwakeup_source_s22_vehicle_utc_time D_UWS_s22_V_UTC_T_Part1_01	ASASASAS
diag_unwantedwakeup_source_s22_vehicle_utc_time D_UWS_s22_V_UTC_T_Part2_01	ASASASAS

diag_unwantedwakeup_source_record_id D_UWS_S_R_I_02	65535
diag_unwantedwakeup_source_s10_distance_totalizer D_UWS_s10_D_T_02	ASASASAS
diag_unwantedwakeup_source_s27_fo_primary_storage_occupancy_raw D_UWS_s27_FPSOER	655 35
diag_unwantedwakeup_source_s27_fo_primary_storage_voltage_1 D_UWS_s27_FPSV_1_02	655 35
diag_unwantedwakeup_source_s53_vehicle_power_mode D_UWS_s53_VPM_02	LIFE ON BOARD
diag_unwantedwakeup_source_rec_mapping_version D_UWS_R_M_V_02	255
diag_unwantedwakeup_trigger_detection_status D_UWW_T_D_S_Part1_02	ASASASAS
diag_unwantedwakeup_trigger_detection_status D_UWW_T_D_S_Part2_02	ASASASAS
diag_unwantedwakeup_source_s22_vehicle_utc_time D_UWS_s22_V_UTC_T_Part1_02	ASASASAS
diag_unwantedwakeup_source_s22_vehicle_utc_time D_UWS_s22_V_UTC_T_Part2_02	ASASASAS

diag_unwantedwakeup_source_record_id D_UWS_S_R_I_03	65535
diag_unwantedwakeup_source_s10_distance_totalizer D_UWS_s10_D_T_03	ASASASAS
diag_unwantedwakeup_source_s27_fo_primary_storage_occupancy_raw D_UWS_s27_FPSOER	655 35
diag_unwantedwakeup_source_s27_fo_primary_storage_voltage_1 D_UWS_s27_FPSV_1_03	655 35
diag_unwantedwakeup_source_s53_vehicle_power_mode D_UWS_s53_VPM_03	LIFE ON BOARD
diag_unwantedwakeup_source_rec_mapping_version D_UWS_R_M_V_03	255

(a) Unwanted Wakeup Source Recorder

diag_busbar_unballast_request D_BB_U_R_00	65535
diag_fuse_unballast_request D_F_U_R_00	255
diag_unballast_s10_distance_totalizer D_U_s10_D_T_00	ASASASAS
diag_unballast_s22_vehicle_utc_time D_U_s22_V_UTC_T_Part1_00	ASASASAS
diag_unballast_s27_fo_primary_storage_occupancy_raw D_U_s27_FPSOER_00 (V)	655 35
diag_unballast_s27_fo_primary_storage_voltage_1 D_U_s27_FPSV_1_00	655 35
diag_unballast_s53_vehicle_power_mode D_U_s53_VPM_00	LIFE ON BOARD
diag_timer_unballast_request D_T_U_R_00 (s)	65535
diag_unballast_rec_mapping_version D_U_R_M_V_00	255
diag_unballast_record_id D_U_R_I_00	65535
diag_sliding_window_gpio_y_busbar_x D_S_W_P_X_BB_X_00 (A)	65535
diag_sliding_window_fuse_x D_S_W_F_X_00 (A)	65535

diag_busbar_unballast_request D_BB_U_R_01	65535
diag_fuse_unballast_request D_F_U_R_01	255
diag_unballast_s10_distance_totalizer D_U_s10_D_T_01	ASASASAS
diag_unballast_s22_vehicle_utc_time D_U_s22_V_UTC_T_Part1_01	ASASASAS
diag_unballast_s27_fo_primary_storage_occupancy_raw D_U_s27_FPSOER_01 (V)	655 35
diag_unballast_s27_fo_primary_storage_voltage_1 D_U_s27_FPSV_1_01	655 35
diag_unballast_s53_vehicle_power_mode D_U_s53_VPM_01	LIFE ON BOARD
diag_timer_unballast_request D_T_U_R_01 (s)	65535
diag_unballast_rec_mapping_version D_U_R_M_V_01	255
diag_unballast_record_id D_U_R_I_01	65535
diag_sliding_window_gpio_y_busbar_x D_S_W_P_X_BB_X_01 (A)	65535
diag_sliding_window_fuse_x D_S_W_F_X_01 (A)	65535

diag_busbar_unballast_request D_BB_U_R_02	65535
diag_fuse_unballast_request D_F_U_R_02	255
diag_unballast_s10_distance_totalizer D_U_s10_D_T_02	ASASASAS
diag_unballast_s22_vehicle_utc_time D_U_s22_V_UTC_T_Part1_02	ASASASAS
diag_unballast_s27_fo_primary_storage_occupancy_raw D_U_s27_FPSOER_02 (V)	655 35
diag_unballast_s27_fo_primary_storage_voltage_1 D_U_s27_FPSV_1_02	655 35
diag_unballast_s53_vehicle_power_mode D_U_s53_VPM_02	LIFE ON BOARD
diag_timer_unballast_request D_T_U_R_02 (s)	65535
diag_unballast_rec_mapping_version D_U_R_M_V_02	255
diag_unballast_record_id D_U_R_I_02	65535
diag_sliding_window_gpio_y_busbar_x D_S_W_P_X_BB_X_02 (A)	65535
diag_sliding_window_fuse_x D_S_W_F_X_02 (A)	65535

(b) Unballast Recorder

FIGURE 20 – Exemples d'écrans Recorder : (a) Unwanted Wakeup, (b) Unballast.

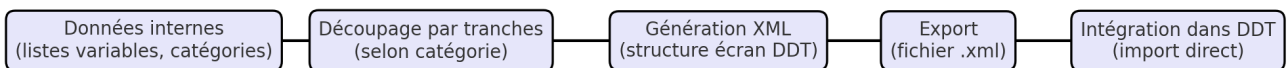
diag_powerlock_timeoutreached_recorder_id D_PL_TOR_R_I_00	65535
diag_powerlock_timeout_reached_recorder_mapping_version D_PL_TOR_R_M_V_00	255
diag_powerlock_timeout_s10_distance_totalizer D_PL_TO_s10_D_T_00	ASASASAS
diag_powerlock_timeout_s22_vehicle_utc_time D_PL_TO_s22_V_UTC_T_Part1_00	ASASASAS
diag_powerlock_timeout_s22_vehicle_utc_time D_PL_TO_s22_V_UTC_T_Part2_00	ASASASAS
diag_powerlock_timeout_s53_vehicle_power_mode D_PL_TO_s53_VPM_00	LIFE ON BOARD
diag_powerlock_timeout_s27_fo_primary_storage_occupancy_raw D_PL_TO_s27_FPSOER_00	655 35
diag_powerlock_timeout_s27_fo_primary_storage_voltage_1 D_PL_TO_s27_FPSV_1_00	655 35
diag_powerlock_timeout_reached_id D_PL_TO_R_I_00	255
diag_powerlock_timeout_status D_PL_TO_S_Part1_00	ASASASAS
diag_powerlock_timeout_status D_PL_TO_S_Part2_00	ASASASAS

diag_powerlock_timeoutreached_recorder_id D_PL_TOR_R_I_01	65535
diag_powerlock_timeout_reached_recorder_mapping_version D_PL_TOR_R_M_V_01	255
diag_powerlock_timeout_s10_distance_totalizer D_PL_TO_s10_D_T_01	ASASASAS
diag_powerlock_timeout_s22_vehicle_utc_time D_PL_TO_s22_V_UTC_T_Part1_01	ASASASAS
diag_powerlock_timeout_s22_vehicle_utc_time D_PL_TO_s22_V_UTC_T_Part2_01	ASASASAS
diag_powerlock_timeout_s53_vehicle_power_mode D_PL_TO_s53_VPM_01	LIFE ON BOARD
diag_powerlock_timeout_s27_fo_primary_storage_occupancy_raw D_PL_TO_s27_FPSOER_01	655 35
diag_powerlock_timeout_s27_fo_primary_storage_voltage_1 D_PL_TO_s27_FPSV_1_01	655 35
diag_powerlock_timeout_reached_id D_PL_TO_R_I_01	255
diag_powerlock_timeout_status D_PL_TO_S_Part1_01	ASASASAS
diag_powerlock_timeout_status D_PL_TO_S_Part2_01	ASASASAS

diag_powerlock_timeoutreached_recorder_id D_PL_TOR_R_I_02	65535
diag_powerlock_timeout_reached_recorder_mapping_version D_PL_TOR_R_M_V_02	255
diag_powerlock_timeout_s10_distance_totalizer D_PL_TO_s10_D_T_02	ASASASAS
diag_powerlock_timeout_s22_vehicle_utc time D_PL_TO_s22_V_UTC_T_Part1_02	ASASASAS
diag_powerlock_timeout_s22_vehicle_utc time D_PL_TO_s22_V_UTC_T_Part2_02	ASASASAS
diag_powerlock_timeout_s53_vehicle_power_mode D_PL_TO_s53_VPM_02	LIFE ON BOARD
diag_powerlock_timeout_s27_fo_primary_storage_occupancy_raw D_PL_TO_s27_FPSOER_02	655 35
diag_powerlock_timeout_s27_fo_primary_storage_voltage_1 D_PL_TO_s27_FPSV_1_02	655 35
diag_powerlock_timeout_reached_id D_PL_TO_R_I_02	255
diag_powerlock_timeout_status D_PL_TO_S_Part1_02	ASASASAS
diag_powerlock_timeout_status D_PL_TO_S_Part2_02	ASASASAS

diag_powerlock_timeoutreached_recorder_id D_PL_TOR_R_I_03	65535
diag_powerlock_timeout_reached_recorder_mapping_version D_PL_TOR_R_M_V_03	255
diag_powerlock_timeout_s10_distance_totalizer D_PL_TO_s10_D_T_03	ASASASAS

FIGURE 21 – Exemple d'écran Recorder : Power Lock Timeout.

FIGURE 22 – Processus de génération des *Recorders* : données internes → tranches → XML → export → intégration DDT.

1. Données internes Les variables et paramètres propres à chaque *Recorder* sont directement codés dans le script, définissant :

- Les **catégories** (*Low-SOC*, *SDE*, *Wakeup Source*, etc.),
- Les **identifiants de variables** à afficher,
- La **logique de découpage en tranches** pour limiter la charge graphique.

2. Création automatisée

1. **Catégorie traitée** indépendamment.
2. **Découpage en tranches** : ex. *Low-SOC* → 0–19 puis 20–39 ; *SDE* → tranches de 20 jusqu'à 243.
3. **Génération XML** via lxml [15], chaque tranche produisant un écran complet (noms, ID, disposition).
4. **Nom systématique** : LowSoc_0_19, SDE_40_59, etc.

3. Export et usage Les fichiers XML sont **exportés automatiquement**, l'utilisateur choisit le nom final, puis ils sont importés directement dans DDT où ils sont immédiatement exploitables.

5.5.4 Avantages et perspectives

La création automatisée des écrans *Recorder* via script Python permet de supprimer presque totalement la charge manuelle et les risques d'erreur. Une fois le script développé et validé, le processus est **entièrement indépendant des documents externes**, stable dans le temps et ne nécessite qu'une **maintenance minimale**.

Critère	Avant (manuel)	Après (script)
Dépendance à des documents externes	Oui	Non
Temps de création	~16,6 h pour 499 écrans	< 10 s, quelle que soit la quantité
Risque d'erreur humaine	Élevé	Très faible
Cohérence visuelle	Variable	Parfaitement uniforme
Maintenance	Lourde	Minimale
Réutilisation	Non	Oui, illimitée

TABLE 11 – Comparatif — création d'écrans Recorder : manuel vs script

Gain de temps

- **Temps manuel** : Les 499 correspondent au nombre total d'écrans *Recorder* à générer. La durée moyenne estimée pour créer un écran manuellement dans DDT est de 2 minutes (configuration, vérification, enregistrement).

$$T_{\text{manuel}} = 499 \times 2 \text{ min} = 998 \text{ min} \approx 16,6 \text{ h} \quad (4)$$

- **Temps avec script** : La génération automatisée produit l'intégralité des 499 écrans en moins de 10 secondes.

$$T_{\text{script}} \approx 0,17 \text{ min} \quad (5)$$

- **Économie par génération** :

$$\Delta T \approx 998 - 0,17 \approx 997,83 \text{ min} \approx 16,6 \text{ h} \quad (6)$$

- Gain relatif :

$$G(\%) \approx \frac{997,83}{998} \times 100 \approx 99,98\% \quad (7)$$

Prise en compte du développement du script

Le développement initial a nécessité entre 1 et 2 journées (≈ 14 h). Même en incluant ce temps dans l'évaluation :

$$T_{\text{total 1}^{\text{re}} \text{ exécution}} \approx 14 \text{ h} + 0,17 \text{ min} \approx 14 \text{ h} \quad (8)$$

On rentabilise l'outil dès la première utilisation, car l'équivalent manuel prend 16,6 h. Chaque exécution ultérieure représente un gain net de 16,6 h.

Perspectives

- Réutilisation immédiate sur d'autres projets avec simple mise à jour des paramètres.
- Mises à jour rapides en cas de modification des besoins.
- Possibilité d'industrialisation (pipeline automatisé, intégration dans un processus plus large).
- Accessibilité totale : un simple double-clic permet la génération, sans compétence technique.

5.6 SDE Check

5.6.1 Objectifs et contexte

L'outil **SDE Check** a été conçu pour automatiser la **vérification de cohérence et de complétude** du fichier SDE (*Schéma de Distribution Électrique*), document central pour la définition et la configuration des SmartMOS.

Il regroupe notamment courants nominaux, protections, niveaux ASIL, modes de contrôle et flux énergétiques.

Jusqu'ici, la validation s'effectuait **manuellement**, feuille par feuille, un processus long, fastidieux et sujet aux erreurs. SDE Check vise à :

- **Appliquer automatiquement** les règles métiers définies pour garantir la conformité du SDE.
- **Détecter rapidement** incohérences et données manquantes pour accélérer les corrections.
- **Générer un rapport structuré** immédiatement exploitable par les équipes.

Le script a été développé pour quatre interlocuteurs principaux : **Baptiste Romieu** (tuteur technique), **Karim Senoun**, **Johann Rodriguez** et **Sébastien Simon**, ingénieurs responsables du SDE et **utilisateurs directs** de l'outil. Leurs retours ont permis de préciser les règles à intégrer et d'adapter le format du rapport.

🎯 Objectif de SDE Check

Fiabiliser et industrialiser le contrôle qualité du SDE en automatisant les vérifications (présence, types, bornes, cohérences) et en produisant un **rapport Excel standardisé**.
Résultat : **détection plus précoce** des écarts et **réduction marquée du délai** entre détection et correction.

5.6.2 Règles de validation intégrées : Comparaison des courants IN et IP

Cette règle contrôle la cohérence des **courants nominaux** (IN) et **de pointe** ($IP = I_{Peak}$) du **Schéma de Distribution Électrique (SDE)**, en les comparant aux seuils définis dans les spécifications. L'objectif est de détecter toute configuration dépassant les capacités prévues des composants, afin d'éviter surchauffes ou déclenchements intempestifs.

Objectif

- Vérifier que :
 - $IN < \text{seuil continu (1000 s)}$,
 - $IP_{Peak} < \text{seuil associé à la durée de crête } t_P$.

Méthode

1. Lecture des valeurs IN et IP depuis le SDE.
2. Comparaison aux seuils (1000 s fixe pour IN , variable selon t_P pour IP).
3. Conversion automatique des formats texte ou numérique (ex. "5A", "N/A").

Seuils appliqués

Type	Durée	Seuil	Erreur si
IN (A)	1000 s	Fixe	$IN \geq \text{seuil}$
IPeak (A)	t_P variable	Dépend de t_P	$IP \geq \text{seuil}$

TABLE 12 – Règles de seuil — IN/IP

Résultat

Toute anomalie est listée dans l'onglet "*IN_IP Violations*" du rapport, avec les composants concernés.

5.6.3 Règles de validation intégrées : Vérification du niveau ASIL

La norme **ISO 26262** impose qu'à chaque fonction de sécurité automobile soit associé un **niveau ASIL** (*Automotive Safety Integrity Level*), de **QM** (aucune exigence de sécurité) à **ASIL-D** (niveau le plus strict) [1]. La règle **SDE Check** vérifie la cohérence du **niveau ASIL** déclaré dans le SDE avec les colonnes associées.

Rappel : niveaux ASIL

QM — sans exigence A — faible B — modéré C — élevé D — maximal

Objectif

- Vérifier que les niveaux ASIL sont bien renseignés.
- Contrôler la cohérence entre :
 - ASIL\Prevent Loss for system control at PIU/DSM,
 - ASIL Controlled (Included column J),
 - ASIL Status.
- Détecter les cas où un **niveau** $> \text{QM}$ n'est pas accompagné des validations requises.

Méthode appliquée

1. Lecture des trois colonnes ASIL.
2. Si un niveau $> QM$ est trouvé :
 - Vérification de SDE Router, Safety Bag, MBSE Distribution Function, MBSE Energy Flow.
 - Alerte si un champ est vide/invalidé ou si la chaîne de validation est rompue.

Seuils et conditions de conformité

Niveau ASIL détecté	Condition obligatoire	Erreur détectée si...
QM	Aucune condition	—
ASIL-A à ASIL-D	Champs cités ci-dessus complets et cohérents	Champ vide ou incohérent

TABLE 13 – Détection des exigences selon le niveau ASIL

Erreur détectée

- Niveau ASIL $> QM$ sans validations associées.
- Incohérence entre les trois colonnes ASIL.
- Donnée manquante ou non interprétable.

5.6.4 Règles de validation intégrées : System Controlling SmartMOS

Le champ **System Controlling SmartMOS** dans le SDE indique *quel système ou composant pilote un SmartMOS donné*. Il est essentiel pour garantir la cohérence des données et éviter les incohérences fonctionnelles.

Objectif

- Contrôler que la valeur respecte les règles métier selon le **type** de composant.
- Détecter les cas particuliers (*Dimming*, **29**, formats spécifiques...).

Méthode

1. Lecture des colonnes **Type** et **System Controlling SmartMOS**.
2. Application des règles :
 - **Type** = "Fuse" : valeur "NA" ou vide.
 - **Autre** : se termine par "T1" ou "T3", ou contient le **Load ID**.

Type	Valeur attendue	Erreur si...
Fuse	Vide ou "NA"	Autre valeur
Autre	"T1", "T3" ou contient Load ID	Aucun de ces critères

TABLE 14 – Règles d'attendus selon le type (extrait)

Conditions de conformité

Erreur détectée

- Fuse avec valeur non vide/NA.
- Autre type ne respectant pas les formats attendus.

5.6.5 Règles de validation intégrées : Vérifications MBSE Energy Flow

Le champ **MBSE Energy Flow** définit le *chemin énergétique* d'un SmartMOS selon la charge alimentée (*ECU, capteur, actionneur...*) [19]. Il doit être **cohérent avec la nature du composant** pour garantir la validité de la modélisation électrique.

Objectif

- Vérifier que la valeur est conforme au type de charge.
- Détecter les incohérences ou valeurs manquantes.

Méthode

1. Lecture des colonnes **Load Type** et **MBSE Energy Flow**.
2. Comparaison avec un mapping prédéfini :
 - **ECU** : ECU_Power, ECU_Energy...
 - **Sensor** : flux faible puissance (Sensor_Power, etc.).
 - **Actuator** : flux forte puissance (Actuator_Power, etc.).

Conditions de conformité

Type de charge	MBSE Energy Flow attendu	Erreur si...
ECU	ECU_Power, ECU_Energy	Hors liste
Sensor	Flux faible puissance	Puissance trop élevée / incohérente
Actuator	Flux forte puissance	Flux faible ou inapproprié

TABLE 15 – Exigences MBSE Energy Flow par type de charge (extrait)

Erreur détectée

- Incohérence type/flux.
- Valeur manquante.
- Flux générique non autorisé (ex. **Generic_Power** pour un capteur).

5.6.6 Règles de validation intégrées : Vérification Exclusive

Le champ **Exclusive and Parallel loads** précise si une charge (*Load*) ne peut fonctionner qu'en exclusivité avec d'autres ou si elle peut être alimentée en parallèle. Une mauvaise configuration peut provoquer des **conflits électriques** ou **surcharges imprévues**.

Objectif

- Vérifier la cohérence avec le champ **Load ID**.
- S'assurer qu'aucune charge non répertoriée ne soit incluse/exclue par erreur.
- Prévenir les conflits fonctionnels entre charges sur la même ligne d'alimentation.

Méthode

1. Lecture des colonnes **Load ID** et **Exclusive and Parallel loads**.
2. Extraction et normalisation des identifiants listés.
3. Vérification que chaque référence existe bien dans le SDE.
4. Signalement des incohérences ou références fantômes.

Conditions de conformité

Vérification	Condition de conformité	Erreur si...
Cohérence des références	Chaque ID existe dans <i>Load ID</i>	Référence inexistante ou mal orthographiée
Concordance fonctionnelle	Les exclusivités respectent les contraintes électriques	Association incompatible
Complétude	Toutes les exclusivités prévues sont listées	Absence d'une exclusivité nécessaire

TABLE 16 – Vérifications et conditions de conformité — tableau de contrôle

Erreur détectée

- Référence absente ou incorrecte.
- Charge listée mais incompatible avec la fonction attendue.
- Liens exclusifs/parallèles incohérents.

5.6.7 Règles de validation intégrées : Vérification I (A) Watt Man

Le champ **I (A) Watt Man (Nominal stabilized)** représente le courant mesuré ou estimé en régime stable pour un composant donné. Une valeur incohérente par rapport au **courant nominal IN (A)** peut indiquer un problème de configuration, de mesure ou de saisie dans le SDE.

Objectif

- Vérifier que le courant nominal stabilisé est **cohérent** avec le courant nominal IN.
- Détecter les erreurs de saisie ou écarts significatifs révélant un problème technique ou documentaire.

Méthode

1. Lecture des colonnes **I (A) Watt Man (Nominal stabilized)** et **IN (A)**.
2. Conversion des valeurs textuelles en nombres exploitables.
3. Comparaison des deux courants avec un seuil d'acceptation défini.
4. Marquage des lignes non conformes pour analyse.

Conditions de conformité

Vérification	Condition de conformité	Erreur si...
Cohérence I Watt Man vs IN	$I \text{ Watt Man} \leq IN \text{ (A)}$	$I \text{ Watt Man} > IN \text{ (A)}$
Donnée exploitable	Valeur numérique valide	Cellule vide, texte non convertible ou unité incorrecte

TABLE 17 – Règles de cohérence — I Watt Man vs IN et validité des données

Erreur détectée

- Valeur de **I Watt Man** supérieure à **IN (A)**.
- Champ vide ou non exploitable, empêchant la validation.

5.6.8 Fonctionnement technique du script

Le script **SDE Check**, développé en Python, analyse automatiquement le **Schéma de Distribution Électrique (SDE)** selon des règles définies avec les architectes et ingénieurs (voir Figure 23). Il détecte les incohérences, génère un rapport multi-onglets et facilite les corrections.

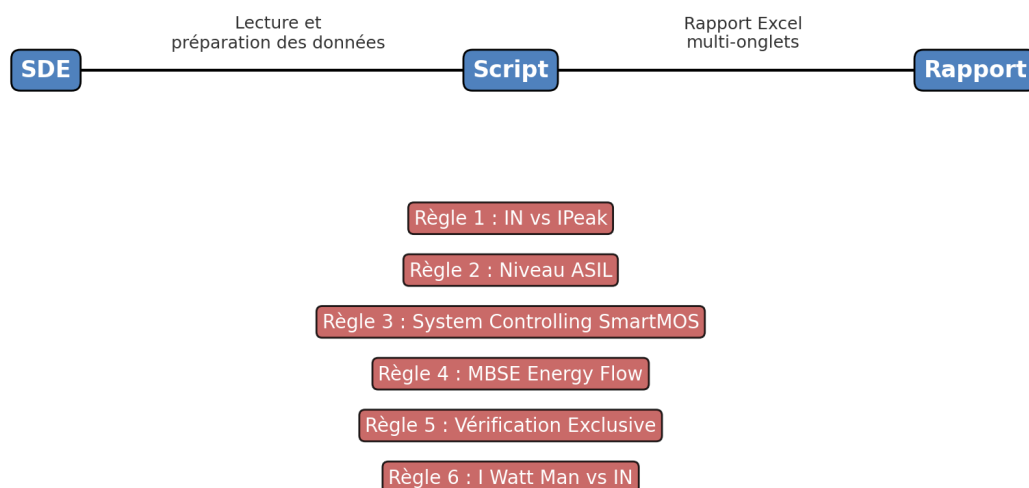


FIGURE 23 – Flux de fonctionnement du script SDE Check

1. Lecture et préparation des données

- Chargement du fichier SDE via `pandas/openpyxl`.
- Conversion des champs textuels en valeurs numériques.
- Gestion des cellules vides ou non exploitables.

2. **Application des règles de validation** Chaque ligne est contrôlée par des modules indépendants :
 - IN & IPeak
 - Niveau ASIL
 - System Controlling SmartMOS
 - MBSE Energy Flow
 - Vérification Exclusive
 - I Watt Man vs IN
3. **Génération du rapport**
 - Feuille récapitulative des erreurs par règle.
 - Feuilles détaillées listant les non-conformités.
 - Mise en forme conditionnelle et filtres.
4. **Organisation interne** Boucle principale, fonctions par règle, dictionnaire centralisant les paramètres et journalisation.
5. **Résultat et intégration** Rapport exploitable pour les revues de conception, corrections ciblées et traçabilité.

5.6.9 Structure du rapport généré

Le script **SDE Check** produit un rapport Excel multi-onglets, structuré pour fournir à la fois une **vision globale** et un **détail exhaustif** des anomalies détectées dans le **Schéma de Distribution Électrique (SDE)**.

1. **Feuilles détaillées par règle** Chaque règle validée par le script dispose d'un onglet dédié (ex. *IN_vs_IPeak*, *ASIL_Check*, *MBSE_Energy_Flow*, etc.). Dans chaque onglet, un tableau présente les lignes non conformes. Voir Figure 24 pour un extrait de la feuille *ASIL_Check* générée automatiquement.

Load ID	Maximum ASIL detected	Condition	Problematic value
S48_128	ASIL_B	3 - Inconsistency between MBSE Function and Energy Flow	SideLeftFlasher / None
S48_128	ASIL_B	4 - Safe state is different from ON	OFF
S48_130	ASIL_B	3 - Inconsistency between MBSE Function and Energy Flow	SideLeftFlasher / None
S48_130	ASIL_B	4 - Safe state is different from ON	OFF
S45_605	ASIL_A	3 - Inconsistency between MBSE Function and Energy Flow	InteriorDoorLockUnlock2 / None
S48_127	ASIL_B	3 - Inconsistency between MBSE Function and Energy Flow	SideRightFlasher / None
S48_127	ASIL_B	4 - Safe state is different from ON	OFF
S48_129	ASIL_B	3 - Inconsistency between MBSE Function and Energy Flow	SideRightFlasher / None
S48_129	ASIL_B	4 - Safe state is different from ON	OFF
S51_300	ASIL_B	3 - Inconsistency between MBSE Function and Energy Flow	RearSeatBeltStatusReceiverControlling / None
S51_300	ASIL_B	5 - Safe state is different from OFF	ON
S29_103A	ASIL_B (CECM)	3 - Inconsistency between MBSE Function and Energy Flow	CentralPanel / None
S28_123	ASIL_A	3 - Inconsistency between MBSE Function and Energy Flow	LeftDoorSecureCargoLock1 / None
S28_123	ASIL_A	4 - Safe state is different from ON	OFF (HW Frozen)
S28_133	ASIL_A	2 - Safety Bag is different from --	--
S28_133	ASIL_A	3 - Inconsistency between MBSE Function and Energy Flow	RightDoorSecureCargoLock1 / None
S28_133	ASIL_A	4 - Safe state is different from ON	OFF (HW Frozen)
S28_124	ASIL_A	3 - Inconsistency between MBSE Function and Energy Flow	DoorSecureCargoCommon / None
S28_124	ASIL_A	4 - Safe state is different from ON	OFF
S28_134	ASIL_A	3 - Inconsistency between MBSE Function and Energy Flow	DoorSecureCargoCommon / None
S28_134	ASIL_A	4 - Safe state is different from ON	OFF

FIGURE 24 – Exemple d'onglet *ASIL_Check* dans le rapport SDE Check

2. **Mise en forme et ergonomie**
 - **Couleurs** : type de l'erreur.
 - **Filtres automatiques** : tri rapide par colonne.
 - **Gel des volets** : entête toujours visible.
3. **Intégration dans le processus** Le rapport est exploitable en revue de conception, facilite la correction ciblée et permet de suivre l'évolution du taux de conformité.

5.6.10 Avantages et perspectives

Avantages immédiats Le **SDE Check** a révolutionné la manière dont les vérifications du **Schéma de Distribution Électrique** sont réalisées. Là où l'analyse complète nécessitait **1 à 2 jours de travail**, elle est désormais **quasi-instantanée**, avec seulement une phase de vérification dans l'OSDE pouvant prendre **une demi-journée au maximum**.

Critère	Avant (manuel)	Après (SDE Check)
Durée totale	~16 h (2 jours de travail)	~4 h (dont < 10 s pour générer le rapport)
Temps de génération des résultats	Inclus dans les 16 h	< 10 secondes
Fiabilité	Variable, erreurs humaines possibles	100 % conforme aux règles définies
Traçabilité	Notes manuelles dispersées	Rapport Excel structuré, multi-onglets
Capacité de traitement	Limitée par le temps disponible	« Illimitée », même pour des fichiers massifs

TABLE 18 – Comparatif — SDE Check : manuel vs automatisé

Calcul du gain de temps

On définit :

$$T_{\text{gain}} = T_{\text{manuel}} - T_{\text{automatisé}} \quad (9)$$

Avec :

$$T_{\text{manuel}} = 16 \text{ h} \quad (10)$$

$$T_{\text{automatisé}} = 4 \text{ h} \quad (\text{phase de vérification incluse}) \quad (11)$$

D'où :

$$T_{\text{gain}} = 16 - 4 = \mathbf{12 \text{ h}} \quad (12)$$

Résultat : un gain direct de **12 h par analyse complète**, soit **75 % de temps économisé**. Autrement dit, deux jours de travail se réduisent à **une demi-journée**.

Impact et valeur ajoutée

- **Temps de traitement divisé par 4** : plus besoin de deux jours entiers pour une analyse.
- **Focalisation sur la correction** plutôt que sur la recherche d'erreurs.
- **Fiabilité constante** : suppression des variations de qualité entre analyses.
- **Expertise démontrée** : formalisation de règles complexes dans un outil industrialisable.

Perspectives d'évolution

1. **Optimisation du contrôle** : lien direct vers la donnée problématique.
2. **Automatisation complète** : déclenchement à chaque mise à jour du SDE et diffusion automatique du rapport.

En résumé Le **SDE Check** transforme une tâche chronophage en un processus **rapide, fiable et traçable**, réduisant de **75 %** le temps d'analyse et améliorant la qualité des résultats.

5.7 CCVC - Creta Cross Verification Check

5.7.1 Objectifs et contexte

Le **Creta Cross Verification Check** (CCVC) a été développé pour répondre à un problème récurrent de cohérence entre les calibrations documentées dans nos spécifications et celles effectivement fournies par le fournisseur.

Dans le cadre d'un projet basé sur l'architecture **AUTOSAR** (*AUTomotive Open System ARchitecture*), les logiciels embarqués sont organisés de manière normalisée pour assurer l'interopérabilité entre différents modules et fournisseurs. Dans ce contexte, certaines fonctions critiques du véhicule reposent sur des calibrations précises définies en interne. Ces calibrations sont intégrées dans les fichiers **CDFX/XML** [5] (format standard AUTOSAR) transmis par le fournisseur, puis utilisées dans l'outil Creta pour valider que la documentation a bien été respectée.

Le fournisseur a pour mission de tester et de renseigner **toutes** les calibrations demandées. Cependant, il peut arriver :

- qu'il **mal documente** certaines calibrations (mauvais format, mauvaise valeur, oubli de paramètres),
- qu'il **interprète incorrectement** les spécifications qui lui ont été transmises,
- ou que l'on constate des **écarts dans les index** ou autres champs techniques par rapport à nos référentiels internes.

Ces incohérences peuvent avoir un impact direct sur le comportement du système. Par exemple, si une calibration de protection électrique est incorrecte, la ligne de distribution associée pourrait ne pas réagir comme prévu en cas de surcharge, exposant ainsi le véhicule à un risque de dysfonctionnement ou de dégradation prématurée des composants.

Jusqu'à présent, la vérification de ces calibrations s'effectuait **manuellement** par les architectes système responsables de chaque fonction (F0 – distribution, F3 – shutdown, F4 – activation et autorisation, F6 – wake-up, F8 – OEM). Cette méthode, en plus d'être chronophage, est rendue complexe par la grande diversité des calibrations, la variabilité des formats documentaires, et les différences d'approche entre fonctions.

📋 Objectif du CCVC

L'objectif du CCVC est de **centraliser et automatiser** ces contrôles afin :

- de détecter rapidement tout écart, qu'il provienne d'une mauvaise documentation, d'une lecture incorrecte de la spécification ou d'une divergence dans les index,
- de fournir un rapport clair et exploitable aux architectes,
- et, à terme, de permettre la correction automatique des fichiers **CDFX/XML** avant transmission aux architectes pour validation.

5.7.2 Fonctionnement technique de l'outil

L'outil **CCVC** (*Creta Cross Verification Check*) automatise le contrôle des calibrations AUTOSAR entre les fichiers CDFX/XML **par PIU — Master, Sub, Hood** (un fichier par

PIU) — et les référentiels internes (SDE, dictionnaires d'indices/IDs SmartMOS, hypothèses i^2t /activation). L'objectif est de détecter **systématiquement** tout écart de valeur, d'index, de présence ou de cohérence documentaire.

1. Entrées & normalisation

- **Entrées PIU** : 3 fichiers AUTOSAR CDFX/XML (ASAM CDF [6]) {Master, Sub, Hood}.
- **Référentiels** :
 - **SDE** (valeurs attendues, variantes, P83, package, modes Led/Bulb, etc.),
 - **Dictionnaire** indices $\rightarrow$ SmartMOS ID (par PIU),
 - **APSW** i^2t (table d'hypothèses / *Leoni Value*),
 - **Activation** (matrices POWER/CAN/Wakeup : VPM $\times$ Enable/Disable).
- **Tuples extraits** pour chaque calibration C_i :

$$C_i = \{\text{PIU, Nom, Index } (I_{C_i}), \text{ Valeur } (V_{C_i}), \text{ Unité, Métadonnées}\}.$$

2. Appariement aux références

- **Mapping** indices $\rightarrow$ SmartMOS ID par PIU via le dictionnaire.
- **Jointure** SDE/*refs* pour obtenir la ou les lignes attendues R_i par calibration :

$$R_i = \{\text{SmartMOS ID, Valeur attendue } (V_{R_i}), \text{ Index } (I_{R_i}), \text{ Package, Mode, ...}\}.$$

- En cas de variantes : filtrage projet (**P83**), validité **PIU**, disponibilité des champs SDE requis.

3. Règles de contrôle (par cas techniques) Les règles sont appliquées **par calibration** et **par PIU**, avec classification des écarts. On note $\Delta V_i = V_{C_i} - V_{R_i}$ et $\Delta I_i = I_{C_i} - I_{R_i}$.

CAS 1 — Wp_CalibData (tableau d'UINT).

Contrôle **présence** et **cohérence** avec la valeur i^2t SDE :

$$\text{Erreur si } V_{C_i} \neq V_{R_i} \text{ ou si SDE incomplet/incohérent (PIU/P83/}i^2t\text{)}.$$

Gestion des variantes : toutes doivent être renseignées et **cohérentes** (mêmes valeurs i^2t) ; sinon, écart *variantiel*.

CAS 2 — Sh_CalibData.Data[X].I2tCurrentSelec. (a) Règle VN9 (package spécifique) : calibration **attendue** = 255.

(b) Sinon (package i^2t « classique ») :

- extraction du **courant de référence** (A) en SDE,
- recherche de la *Leoni Value* correspondante dans **APSW** (arrondi $\pm 0,1$ A),
- contrôle :

$$\text{Erreur si } V_{C_i} \neq V_{\text{Leoni}} \text{ ou si variantes SDE vides/incohérentes.}$$

CAS 3 — Dh_CalibData.Data[X].I2T_Curve.

Projection de la valeur entière sur un nom de courbe **CURVEk**.

Package BTG : la courbe SDE doit être **présente** et **unique** sur variantes ; on exige

$$\text{Erreur si } \text{Curve}_{C_i} \neq \text{Curve}_{\text{SDE}} \text{ ou variantes vides/incohérentes.}$$

Package BTS (sans i^2t) : **doit rester vide/NA** côté SDE et calibration ; toute donnée non vide = erreur.

CAS 4 — `Sh_CalibData.Data[X].OperationMode`.

Dérivation de l'attendu selon SDE :

$$\text{Selected mode} = \begin{cases} \text{Bulb} & \Rightarrow V_{R_i} = 0, \\ \text{LED} & \Rightarrow V_{R_i} = 1, \\ \text{NA} & \Rightarrow V_{R_i} = 255. \end{cases}$$

Mode mixte (LED/Bulb) : décision par règles S48 (p. ex. $F8310, U8310 \in \{\text{LED}, \text{BULB}\}$).

Contrôle final : $V_{C_i} = V_{R_i}$; sinon, **écart de mode**.

Matrices d'activation — *POWER* / *CAN* / *Wakeup*.

Extraction des triplets (idx, VPM, Enable/Disable) côté XML et jointure à la **matrice Excel**.

$$\text{Erreur} = \begin{cases} \text{Paramètre manquant XML/Excel,} \\ \text{ou } V_{C_i} \neq V_{R_i} \text{ (comparaison décimale, reformatage hex à longueur attendue).} \end{cases}$$

4. **Taxonomie des erreurs** Chaque écart est catégorisé pour prioriser les corrections :

- **Calibration** (valeur différente) : $\Delta V_i \neq 0$,
- **Index** (désalignement d'index) : $\Delta I_i \neq 0$,
- **Présence** (manquant/excédentaire),
- **Structure** (PIU/P83/ID non trouvés, champs SDE absents),
- *i²t* / **Courbes** (vides ou incohérentes entre variantes),
- **Mode** (LED/Bulb/NA/Mixte incohérent),
- **Activation** (POWER/CAN/Wakeup : *missing/different*).

5. **Consolidation & rapport**

- **Un onglet Excel par fonction** (F0, F3, F4, F6, F8) avec filtres + gel de volets,
- **Code couleur** par type d'écart (lecture rapide),
- **SUMMARY** : pivots (type d'erreur $\times$ PIU), top erreurs, répartitions, indicateurs **OK/-NOK** (SmartMOS, PIU).

6. **Vers la correction automatique** En perspective, le CCVC applique des **règles de patch** directement sur les CDFX/XML :

$$V_{C_i} \leftarrow V_{R_i} \quad \text{et/ou} \quad I_{C_i} \leftarrow I_{R_i}$$

lorsqu'une règle est **non ambiguë** (référentiel unique, variantes concordantes, package valide). Cela transforme l'outil de **détection** en **assistant de remédiation** prêt pour revue/validation par l'architecte.

5.7.3 Résultats obtenus

Le CCVC génère un *rapport Excel* unique et standardisé, pensé pour une lecture rapide par PIU et une priorisation immédiate des corrections.

Onglets livrés

- **MASTER, SUB, HOOD** : un onglet par PIU (un fichier AUTOSAR analysé par PIU). Chaque feuille liste les écarts détectés sur le périmètre du PIU.
- **ACTIVATION** (si fourni) : comparaison POWER / CAN / Wakeup entre la matrice de référence et les valeurs extraites des CDFX.
- **SUMMARY** : synthèse transversale (tableaux croisés, graphiques, KPIs).

Structure des feuilles PIU (Voir Figure 25)

- **Colonnes minimales** : *Index*, *ID* (si dispo), *Erreur* (libellé normalisé), *Attendu*, *Trouvé*, + champs contextuels selon le cas (package, courbe i^2t , mode LED/Bulb, HEX/DEC pour activation).
- **Libellés d'erreur normalisés** : p. ex. *Calibration $\neq$ Valeur SDE*, *Calibration $\neq$ Leoni Value*, *Calibration $\neq$ Curve SDE*, *VN9 mal documenté*, *Mode mixte incohérent*, *Structure*, *Activation*, ...
- **Mise en forme ergonomique** : code couleur par type d'erreur, filtres, volets figés, ajustement auto des largeurs.

Index	ID	Erreur	Valeur du SDE	Valeur trouvée	Cas
25	BM20	Valeurs I2T incohérentes entre variantes : 5.0, 4.0		4 Wp	
64	CM20	Valeurs I2T incohérentes entre variantes : 1.0, 4.0		5 Wp	
15	CM70	Calibration $\neq$ Curve SDE	CURVE1 (HW Frozen)	CURVE1	Dh
16	CM80	Calibration $\neq$ Curve SDE	CURVE1 (HW Frozen)	CURVE1	Dh
22	CM20	Package BTS $\rightarrow$ calibration ou SDE non vide	nan	CURVE5	Dh
23	CM10	Calibration $\neq$ Curve SDE	CURVE5 (HW Frozen)	CURVE5	Dh
24	CM30	Calibration $\neq$ Curve SDE	CURVE5 (HW Frozen)	CURVE5	Dh
3	AM80	Calibration $\neq$ Curve SDE	CURVE1 (HW Frozen)	CURVE1	Dh
4	AM90	Package BTS $\rightarrow$ calibration ou SDE non vide	nan	CURVE1	Dh
10	BM65	Mode mixte incohérent ou incomplet (F8310/U8310)	F8310 = 'NAN' / U8310 = 'NAN'	1 OpMode	
11	BM61	Mode mixte incohérent ou incomplet (F8310/U8310)	F8310 = 'NAN' / U8310 = 'NAN'	1 OpMode	
12	BM62	Mode mixte incohérent ou incomplet (F8310/U8310)	F8310 = 'NAN' / U8310 = 'NAN'	1 OpMode	
15	BM60	Mode mixte incohérent ou incomplet (F8310/U8310)	F8310 = 'NAN' / U8310 = 'NAN'	1 OpMode	
21	BM50	Calibration $\neq$ Mode attendu	0	1 OpMode	
28	AM50	Mode inconnu ou non applicable : 'nan'		0 OpMode	
2	AM33	Calibration $\neq$ Mode attendu	0	1 OpMode	
30	AM53	Calibration $\neq$ Mode attendu	1	0 OpMode	
31	AM52	Calibration $\neq$ Mode attendu	1	0 OpMode	
33	AM55	Mode inconnu ou non applicable : 'light thresholds'		1 OpMode	
34	BM42	Mode mixte incohérent ou incomplet (F8310/U8310)	F8310 = 'LED' / U8310 = 'BULB'	1 OpMode	
35	BM41	Mode mixte incohérent ou incomplet (F8310/U8310)	F8310 = 'NAN' / U8310 = 'NAN'	1 OpMode	
36	BM43	Mode mixte incohérent ou incomplet (F8310/U8310)	F8310 = 'LED' / U8310 = 'BULB'	1 OpMode	
38	CM65	Mode mixte incohérent ou incomplet (F8310/U8310)	F8310 = 'NAN' / U8310 = 'NAN'	1 OpMode	
39	CM62	Mode mixte incohérent ou incomplet (F8310/U8310)	F8310 = 'NAN' / U8310 = 'NAN'	1 OpMode	
3	AM32	Mode inconnu ou non applicable : 'tbc'		0 OpMode	
3	AM32	Calibration $\neq$ Mode attendu	1	0 OpMode	
40	CM60	Mode mixte incohérent ou incomplet (F8310/U8310)	F8310 = 'NAN' / U8310 = 'NAN'	1 OpMode	
41	CM61	Mode mixte incohérent ou incomplet (F8310/U8310)	F8310 = 'NAN' / U8310 = 'NAN'	1 OpMode	
42	CM63	Mode mixte incohérent ou incomplet (F8310/U8310)	F8310 = 'NAN' / U8310 = 'NAN'	1 OpMode	
43	CM64	Mode mixte incohérent ou incomplet (F8310/U8310)	F8310 = 'NAN' / U8310 = 'NAN'	1 OpMode	

FIGURE 25 – Exemple — feuille MASTER

Feuille *ACTIVATION* (optionnelle)

- **Colonnes** : *Index*, *Type* (POWER/CAN/Wakeup)(ISO 11898-1 :2024 [2]), *VPM*, *Mode* (Enable/Disable), *Erreur*, *Attendu_hex*, *Attendu_dec*, *Creta_hex*, *Creta_dec*.
- **Entiers longs** écrits en *texte* pour éviter la notation scientifique d'Excel.
- **Règles de détection** : *Paramètre manquant dans XML/Excel* ou *Paramètre Différent* (avec reformatage HEX à longueur attendue).

Feuille *SUMMARY* (Voir Figure 26)

- **Tableau croisé** *Type d'erreur* $\times$ *PIU* (+ TOTAL), trié par criticité/volume.
- **Top erreurs** (barres horizontales), **répartition des erreurs** par PIU (doughnut) et **barres empilées** par type.

- KPI SmartMOS OK/KO par PIU et taux de conformité :

$$\%OK_{PIU} = \frac{OK}{OK + NOK} \times 100 \quad (13)$$

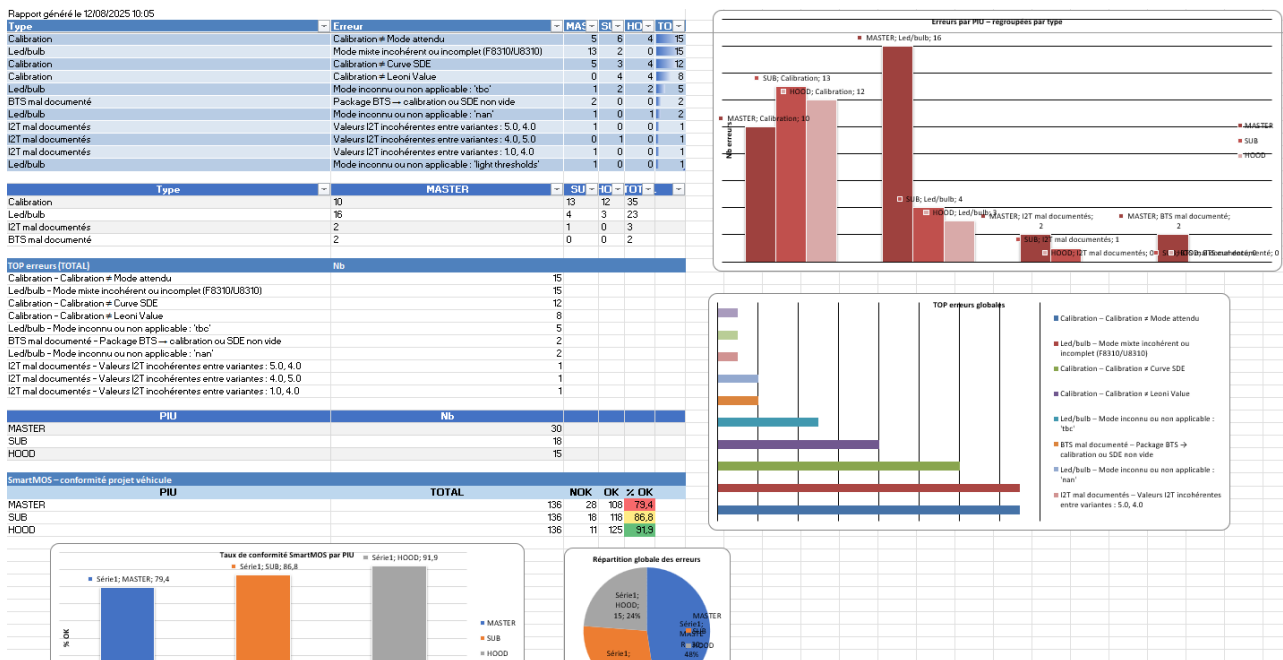


FIGURE 26 – Feuille **SUMMARY** : pivots, top erreurs, répartition par PIU et KPI %OK.

Exemples de lignes typiques (génériques)

- **Calibration ≠ Curve SDE** : la valeur codée pour I2T_Curve ne correspond pas à la courbe attendue (package BTG) ; ou présence d'une valeur non autorisée pour un package BTS (qui doit rester vide).
- **Calibration ≠ Leoni Value** : la calibration codée ne correspond pas à la valeur attendue issue du couple {package, courant i^2t }.
- **Calibration ≠ Mode attendu** : OperationMode (LED/Bulb/NA) non aligné avec le mode dérivé du SDE (cas mixte résolu via règles F8310/U8310).
- **Activation — Paramètre manquant / Différent** : écart entre la matrice d'activation de référence et les valeurs HEX/DEC extraites du CDFX.

Génération & packaging

- Livrable exporté au format `report_error.xlsx`.
- Les feuilles PIU sont écrites puis stylisées ; **SUMMARY** est (re)générée (pivots + graphiques).
- Si la matrice d'activation est fournie, l'onglet **ACTIVATION** est ajouté et colorisé avec sa légende.

5.7.4 Apports, plus-value et perspectives

Avant, la vérification AUTOSAR reposait sur une lecture manuelle et éclatée de documents (logiques différentes selon les architectes). Une tentative de *parsing* existait mais ne permettait ni de tracer correctement, ni de distinguer les cas (modes LED/Bulb, i^2t , BTG/BTS, matrices

POWER/CAN/Wakeup). Résultat : tâche réservée aux experts, très difficile pour un nouvel arrivant. **CCVC** encapsule ces règles, parcourt automatiquement les trois PIU (**Master/Sub/Hood**) et livre un **rapport standardisé** (feuilles PIU, **SUMMARY**, **ACTIVATION**) exploitable par tous.

Critère	Avant (manuel)	Après (CCVC)
Lecture CDFX / mapping	Partiel, sujet à erreurs	Automatisé , stable
Reconstruction via docs multiples	Dépend d'un expert	Codifiée (règles outillées)
Couverture PIU × fonction	Variable	Homogène (Master/Sub/-Hood × F0/F3/F4/F6/F8)
Traçabilité / preuve	Dispersée	Rapport Excel (PIU, ACTIVATION, SUMMARY)
Risque d'erreur humaine	Élevé	Très faible
Reproductibilité	Faible	Constante
Compétence requise	Architecte AUTOSAR	Utilisateur lambda (exécution guidée)
Auto-correction CDFX	Non	Prévue (mode <i>autofix</i> contrôlé)

TABLE 19 – Comparatif avant / après sur le périmètre de vérification AUTOSAR

Gain de temps

Scénario conservateur :

- Périmètre : ≈ 540 calibrations ($\approx 180/\text{PIU}$)
- Manuel : ≈ 3 min/calibration (lecture, vérif, croisement) + 30 min de bilan

$$T_{\text{manuel}} \approx 540 \times 3 \text{ min} + 30 \text{ min} = 1650 \text{ min} \approx \mathbf{27,5 \text{ h}} \quad (14)$$

- CCVC : ≈ 3 min d'exécution + ≈ 45 min de revue (≈ 15 min/PIU)

$$T_{\text{CCVC}} \approx 48 \text{ min} \approx \mathbf{0,8 \text{ h}} \quad (15)$$

$$\text{Gain par exécution : } \Delta T \approx \mathbf{26,7 \text{ h}} \quad \text{Gain relatif : } \approx \mathbf{97\%} \quad (16)$$

En pratique : l'outil est **rentabilisé dès le premier run** et chaque exécution suivante économise $\sim 27 \text{ h}$ net.

Gains additionnels concrets

- **Onboarding** : d'un apprentissage de plusieurs semaines à $< 2 \text{ h}$ (lecture du rapport + filtres).

- **Moins d'itérations tardives** : une incohérence de mode/courbe détectée tôt évite $\frac{1}{2}$ -1 journée de re-tests.
- **Qualité documentaire** : taxonomie d'erreurs normalisée, directement exploitable en KPI / audits.

Plus-value technique

- Abstraction **AUTOSAR** (CDFX) + mapping index ↔ SmartMOS + règles VN9, BTG/BTS, i^2t , modes, matrices POWER/CAN/Wakeup.
- **Livrable actionnable** (PIU détaillés, **SUMMARY** KPI/graphes, **ACTIVATION** HEX/-DEC).
- **Scalabilité** : mêmes règles pour d'autres projets/variants sans réapprendre la logique des architectes.

Perspectives (objectif produit)

Vers l'autonomie complète :

1. **Analyser et détecter** les écarts,
2. **Proposer** un journal de modifications,
3. **Appliquer** les corrections dans le **CDFX** (*autofix* : *dry-run* → *apply*),
4. **Tracer** toutes les corrections (diff + horodatage) pour validation finale.

C'est la **vraie plus-value** : passer d'un contrôleur d'écarts à un **assistant de remédiation AUTOSAR**, industrialisable et intégrable en **CI/CD**.

5.8 ASCS — Automatic Suitable Curve Selection

5.8.1 Objectifs et contexte

Pendant des années, le dimensionnement des **SmartMOS** s'appuyait sur un **outil Excel** qui séparait les vues : un graphe pour le *profil de courant*, un pour les *BTG*, un pour les *VNF*, un pour les *courbes software*. Concrètement, l'ingénieur devait :

1. **préparer ou ressaisir** son profil de courant,
2. **ouvrir plusieurs feuilles/graphes** et **afficher les composants un par un**,
3. **changer de fenêtre** pour comparer BTG, VNF et courbes software,
4. **recommencer** à chaque modification (température, composant, point du profil...).

Avec Excel — remarquable pour du tableur mais **pas conçu pour de la comparaison graphique interactive** — cela se traduisait par des **temps de chargement élevés**, des **comportements variables selon les postes** (latences, bugs, fichiers qui ne s'ouvrent pas toujours) et, surtout, par l'impossibilité pratique de **superposer plusieurs composants** pour décider rapidement. Beaucoup d'architectes ont fini par **faire la comparaison à la main**, faute d'un outil fiable et fluide.

Le **dimensionnement des SmartMOS** est **structurant** à deux niveaux :

- **Économique** : choisir systématiquement « au max » **alourdit le coût BOM** (Bill of Materials cost – coût de nomenclature) sans nécessité ; un **bon dimensionnement** permet de **minimiser le coût** tout en respectant les contraintes.
- **Sécurité / fiabilité** : un **sous-dimensionnement** peut conduire à des **incidents graves** (surchauffe, coupure intempestive ou trop tardive, dégradation de faisceau).

L'enjeu métier est donc de **sélectionner le composant le moins coûteux qui tient le profil réel et les températures PIU**, avec une **marge maîtrisée** et une **décision traçable**.

Objectifs ASCS (Automatic Suitable Curve Selection)

- **Overlay unique** (profil + BTG + VNF + software) sur **un seul graphique interactif**, pour comparer **plusieurs composants en parallèle** sans changer d'écran.
- **Bande de tolérance $\pm 10\%$** autour du profil pour un statut visuel **Vert/Orange/Rouge**. Le critère est :

$$\forall t, \quad I_{\text{comp}}(t) \geq I_{\text{profil}}(t) \quad \text{et} \quad \frac{|I_{\text{comp}}(t) - I_{\text{profil}}(t)|}{I_{\text{profil}}(t)} \leq 10\% \quad (17)$$

où :

- $I_{\text{comp}}(t)$ est le **courant du composant choisi** à l'instant t ,
- $I_{\text{profil}}(t)$ est le **courant de référence** issu du profil attendu à l'instant t .
- **Classement automatique (*Top-N*)** selon l'**adéquation** au profil (proximité + respect du couloir), avec départage par critères techniques (**RDS(on)**, *overcurrent*, température, n° de channels) et **objectif coût** si l'information est disponible.
- **Lien SDE** pour **restreindre** aux SmartMOS **réellement disponibles** et afficher leurs **IDs** (mode « global » vs « disponibles SDE »), afin d'orienter vers un **choix pertinent coût/disponibilité**.
- **Praticité** : saisie/import du profil, sélection de la **température** (65 °C Master/Sub, 75 °C Hood, 85 °C DSM), exports **PNG/CSV/Excel**.

Synthèse : ASCS remplace une **succession d'étapes hétérogènes et chronophages** par un **comparateur unifié, rapide et fiable**, qui **réduit les coûts** (évite le sur-dimensionnement), **sécurise** le choix (évite le sous-dimensionnement) et **accélère** la décision de dimensionnement des SmartMOS.

5.8.2 Architecture et choix technologiques

L'ASCS est conçu comme une **application web locale** en trois couches — **sources Excel** → **services Flask** [12] → **interface HTML/JS** — afin d'obtenir une UI fluide, portable (navigateur) et une logique de traitement centralisée en Python.

Pourquoi ce choix (et comment il est appliqué ici)

- **Flask côté serveur** : sert la page HTML, expose des **endpoints JSON** et encapsule tout le traitement (lecture Excel, normalisation, préparation des données à tracer, génération des exports). On évite ainsi les limites d'Excel (latence, VBA hétérogène, dépendances de poste) et on garde la **maîtrise du calcul en Python**. Concrètement, le serveur est initialisé dans `routes.py`, avec une page d'accueil `index()` et un **health check /ping**.
- **Interface web (HTML/JS + Plotly [11])** : un seul écran, dans `index.html`, concentre les contrôles (édition/import du profil, choix famille/composant, sélection de température **65/75/85 °C**, chargement SDE, bouton *Classer*) et un **graphe Plotly** pour la superposition interactive. L'utilisateur manipule tout depuis le navigateur, sans plugin Excel ni installation côté client.
- **Démarrage simple & local-only** : un lanceur ouvre le navigateur après vérification du serveur (`/ping`) ; aucune donnée ne sort du poste.

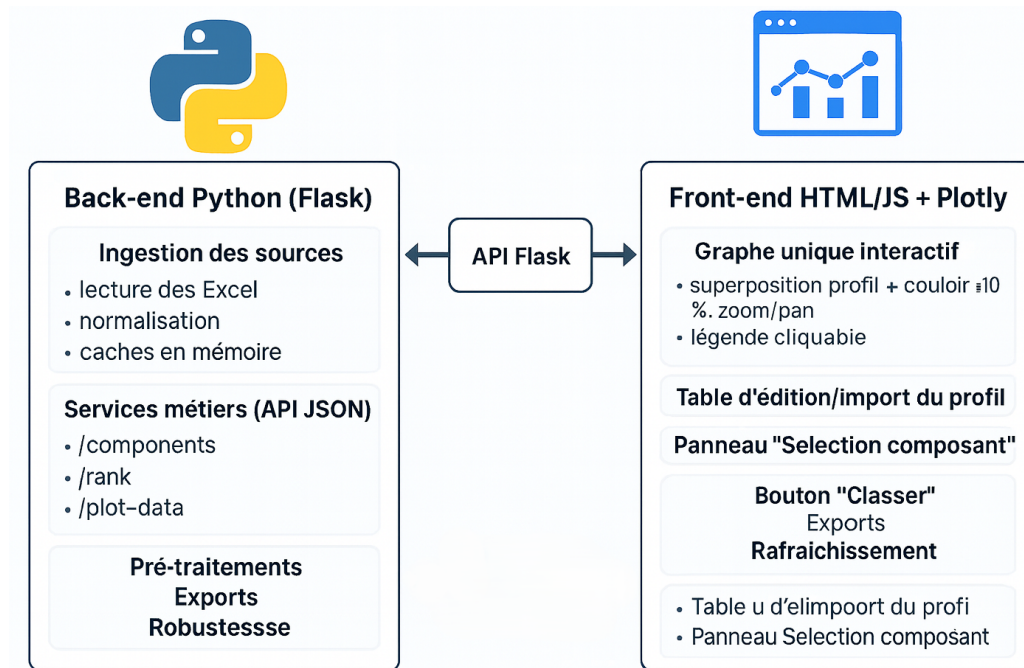


FIGURE 27 – Architecture ASCS : Flask (API, données) ↔ HTML/JS + Plotly (graphe, édition, sélection).

Organisation concrète du back-end (ce projet)

- **Chargement/normalisation des bibliothèques :**
 - **BTG** : lecture des feuilles `*_Courant`, `*_Temp_Jonction`, `*_Infos` et des i^2t dans `*_i2t` ; assemblage en dictionnaire $\{65/75/85 \rightarrow \text{courbe } I_{\min}\}$ + métadonnées.
 - **VNF** : même principe (courbes et compléments), avec extraction des **courbes** « **I step min ...** ».
 - **SW (autres SmartMOS)** : lecture des **Infos** (références, channels, type), courbes $I_{\min}$ (gestion des variantes ChX) et **protections SWn** communes ; les références multi-canaux sont dupliquées proprement.
- Les trois familles sont **fusionnées** dans une base unique DB (clé = référence), chaque entrée portant sa **famille**, ses **courbes** et ses **extras** ($i^2t/I_{\text{nom}}/\text{SWn}$) pour un traitement homogène côté API.
- **Services Flask (endpoints JSON [3])** — l'interface n'appelle que des routes claires ; le serveur renvoie des objets *prêts à afficher* :
 - `/compute` : densifie le **profil** (rééchantillonnage), calcule le **RMS** [14] (utilitaires dans `helpers.py`), renvoie les séries temps/courant.
 - `/list/<mode> & /info/<mode>/<ref>` : liste des références par famille (**BTG/VNF/SW/all**) et **métadonnées** d'un composant.
 - `/curve/<mode>/<ref>/<temp> & /extra/<mode>/<ref>` : courbe $I_{\min}$ à 65/75/85 °C et **courbes complémentaires** ($i^2t / I_{\text{nom}} / \text{SWn}$) prêtes à tracer.
 - `/sde/upload & /sde/status` : chargement du **SDE** (feuille *Power Distribution*), index (référence normalisée, RDS(on)@25) → IDs SmartMOS pour la vue « disponibles SDE ».
 - `/rank/<mode>` : retourne un **tableau de classement** (Top-N) formaté pour l'UI ; les critères restent côté serveur.
- **Performance & maintenance :**
 - **Lecture unique & cache mémoire** : les fichiers Excel sont lus une fois ; en cas de modification (*mtime*), le cache est invalidé et reconstruit $\Rightarrow$ temps de réponse stables.

- **Séparation I/O / services / UI** : modules I/O Excel, utilitaires traitement signal (`helpers.py`), routes API (`routes.py`), templates/actifs front $\Rightarrow$ évolutions ciblées (nouvelle famille, propriété, export) sans casser l'ensemble.

Fonctionnement côté interface (ce projet)

- **Édition/import du profil** (table HTML + import CSV), bouton **Tracer** $\rightarrow$ appel /compute $\rightarrow$ mise à jour du graphe unique.
- **Sélection du mode** (BTG/VNF/SW/Global), filtrage **type** (High-Side / H-Bridge), choix **température 65/75/85 °C** $\rightarrow$ appels /list, /info, /curve, /extra pour alimenter le même graphe.
- **Chargement SDE** (optionnel) $\rightarrow$ /sde/upload ; l'UI affiche le badge d'état et propose la vue « **Disponibles SDE** ».
- **Classement** $\rightarrow$ /rank/<mode> ; l'UI affiche le **Top-N** dans un tableau, sans logique métier côté client.

En d'autres termes, **Flask** centralise les calculs et l'accès aux données Excel, fournit une API simple et **locale**, tandis que **HTML/JS + Plotly** offrent une visualisation unique et réactive. Cette architecture sépare proprement **données / services / interface**, supprime les lourdeurs d'Excel et permet d'ajouter familles, propriétés ou exports **sans remettre en cause l'ensemble**.

5.8.3 Modèle de données et règles

Le profil est saisi dans l'interface (table éditable) ou importé (CSV/Excel). Les unités sont **secondes** (temps) et **ampères** (courant). À l'ingestion, il est :

- **validé** (temps strictement croissant, valeurs numériques) ;
- **rééchantillonné** sur une grille commune pour comparer proprement aux courbes composants ;
- entouré d'un **couloir de tolérance** $\pm 10\%$ défini par :

$$I^-(t) = 0,9 \cdot I_{\text{profil}}(t), \quad I^+(t) = 1,1 \cdot I_{\text{profil}}(t) \quad (18)$$

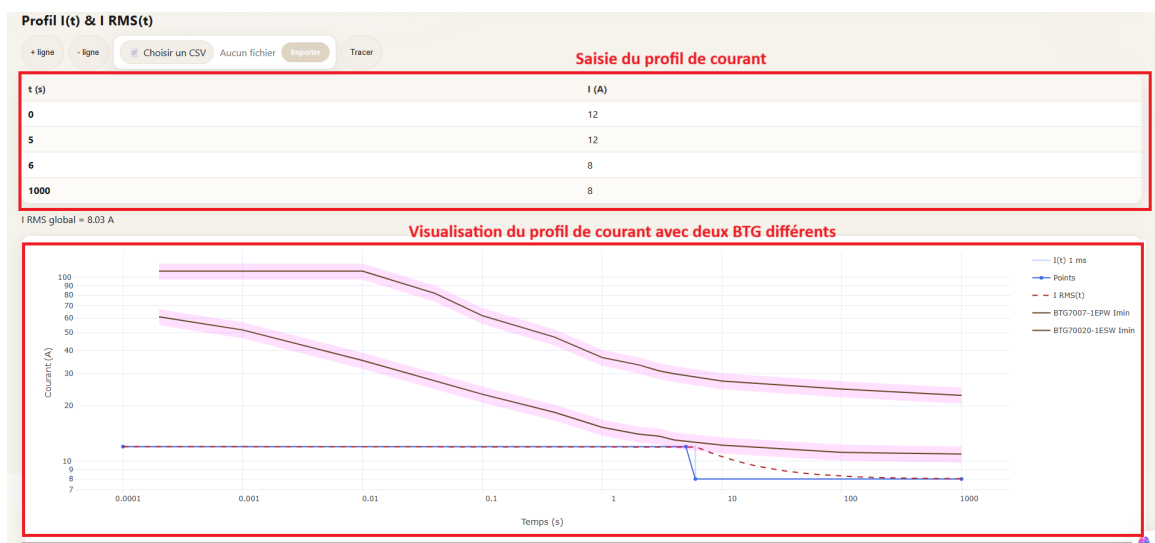


FIGURE 28 – Saisie/import du profil $I(t)$, calcul $I_{\text{RMS}}(t)$ et couloir $\pm 10\%$, superposés à deux courbes I_{min} de BTG différents.

Chaque référence composant dispose de courbes $I_{\min}(t)$ par **température** (65 °C, 75 °C, 85 °C). Les fichiers Excel de référence sont normalisés en une base unique avec, pour chaque composant :

- **famille** (BTG/VNF/software) ;
- **courbes par température** (séries temps-courant $I_{\min}(t)$) ;
- **propriétés techniques** disponibles (p. ex. nombre de *channels*, *overcurrent*, *RDS(on)*, type High-Side/H-Bridge, infos d'auto-échauffement si présentes). [21] [22] [23]

Pour une référence et une température données, on compare $I_{\min}(t)$ au profil :

- **Vert (valide)** : la courbe composant **reste au-dessus** du profil **dans le couloir** sur tout l'intervalle,

$$\forall t, \quad I_{\min}(t) \geq I_{\text{profil}}(t) \quad \text{et} \quad \frac{|I_{\min}(t) - I_{\text{profil}}(t)|}{I_{\text{profil}}(t)} \leq 10\%.$$

- **Orange (limite)** : respect global mais **marge faible** ou contacts ponctuels avec $I^-(t)$ sur des points caractéristiques.
- **Rouge (invalide)** : passage **sous** le profil ou **sortie durable** du couloir.

Des indicateurs simples (marge minimale, écarts relatifs sur pics/plateaux) sont calculés pour **affichage** et **tracés** ; ils servent à qualifier visuellement la conformité, indépendamment des règles de *classement*.

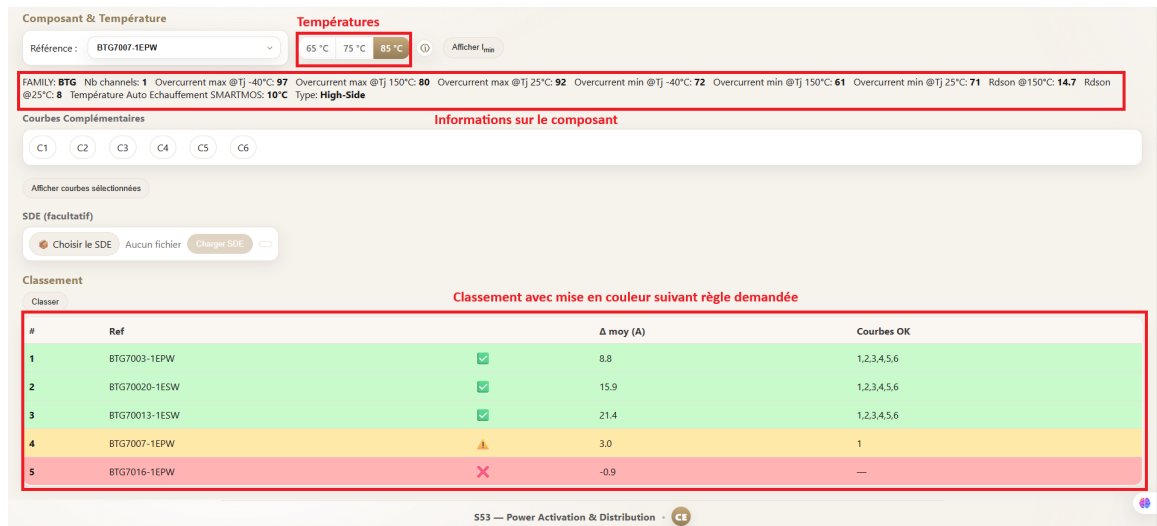


FIGURE 29 – Vue de contrôle : informations composant, sélection de température et *classement* coloré selon la règle d'acceptation.

5.8.4 Algorithme de classement

🎯 Objectif

Proposer, pour un **profil de courant** donné et une **température** choisie (65/75/85 °C), une **liste ordonnée** de composants pertinents, en privilégiant ceux qui **respectent le profil** tout en **évitant** le sur-dimensionnement.

Principe général : on discrétise le profil en K points $\{t_k\}_{k=1..K}$. Pour un composant (courbe $I_{\min}$ à la température demandée), on calcule la **marge relative**

$$m_k = \frac{I_{\min}(t_k) - I_{\text{profil}}(t_k)}{I_{\text{profil}}(t_k)}.$$

- $m_k < 0$: passage **sous** le profil (non-conformité) ;
- $0 \leq m_k \leq 0,10$: **dans le couloir** $\pm 10\%$ (zone idéale) ;
- $m_k > 0,10$: **au-dessus** du couloir (surdimensionnement).

Fonction de coût (proximité + pénalités) : on évalue chaque composant par un score scalaire J :

$$J = \underbrace{\frac{1}{K} \sum_{k=1}^K \left[w_{\downarrow} \max(0, -m_k)^2 + w_{\uparrow} \max(0, m_k - 0,10)^2 \right]}_{\text{respect du profil et du couloir}} + \lambda \underbrace{\text{Var}(m_1, \dots, m_K)}_{\text{régularité de la marge}}.$$

- $w_{\downarrow} \gg w_{\uparrow}$: **sanction forte** de toute descente sous le profil ;
- $w_{\uparrow}$ modéré : **dissuasion** du sur-dimensionnement persistant ;
- λ lisse les comportements hétérogènes (pics).

Le classement croissant sur J favorise les courbes proches du profil, régulières, et qui restent dans le couloir.

Listes produites

- **Classement global** : tous les composants chargés ;
- **Par famille** : BTG, VNF, software ;
- « **Disponibles SDE** » : restreint aux références réellement présentes dans le SDE (IDs/PIU).

Règles de départage (scores proches) : à score J similaires, sélection **parcimonieuse** et compatible avec le besoin :

1. **Marge minimale** $\min_k m_k$ la plus proche de 0 (meilleure « justesse ») ;
2. **Température** : priorité à la conformité à la température la plus contraignante demandée ;
3. **Propriétés techniques** : adéquation **channels**/architecture, puis **RDS(on)** (plus faible si coût inconnu), puis **overcurrent** (marge suffisante sans excès).

Sortie côté UI : l'interface affiche un **Top-N** avec statut **Vert/Orange/Rouge**, le score J , les **marges min/moy/max**, et les attributs clés (famille, channels, RDS(on), overcurrent, température, ID SDE le cas échéant) pour une décision **rapide, comparable et traçable**.

Complexité : pour N composants et K points de profil, l'évaluation est en $O(NK)$ et se parallélise trivialement (composant par composant).

5.8.5 Parcours utilisateur

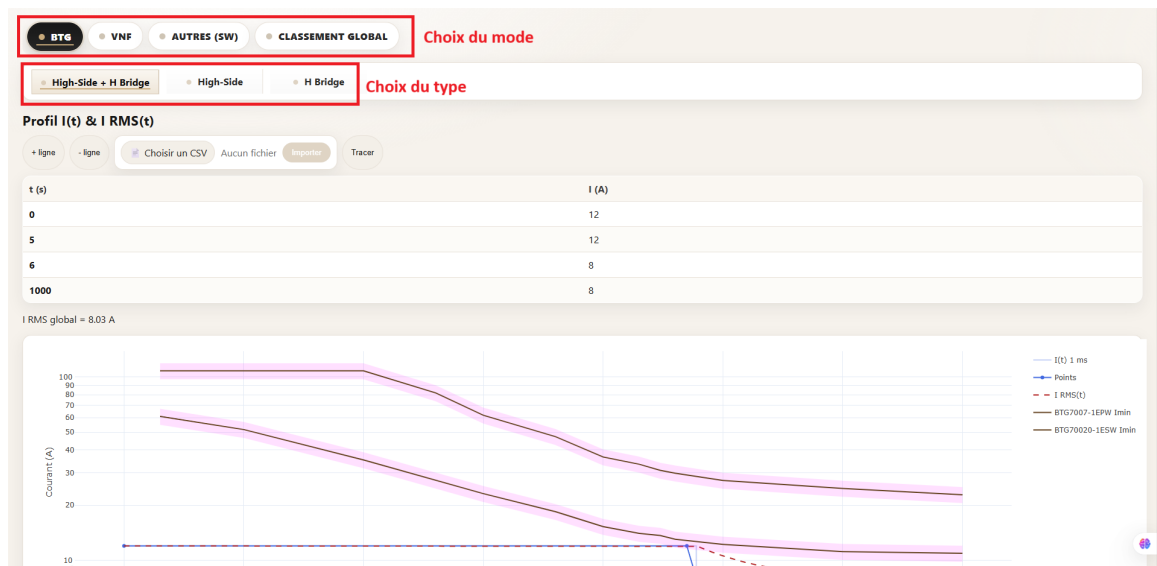
L'écran regroupe tout : choix du contexte, profil, composant/température, et classement. On travaille sans changer de page, avec un **graphe central interactif**.

1. Choisir le contexte d'analyse

En haut, on sélectionne le **mode** (BTG / VNF / AUTRES (SW) / CLASSEMENT GLOBAL) et le **type** (*High-Side + H Bridge*, ou filtré *High-Side / H Bridge*) (Voir la Figure 30). Ces segments pilotent ce que l'interface propose ensuite.

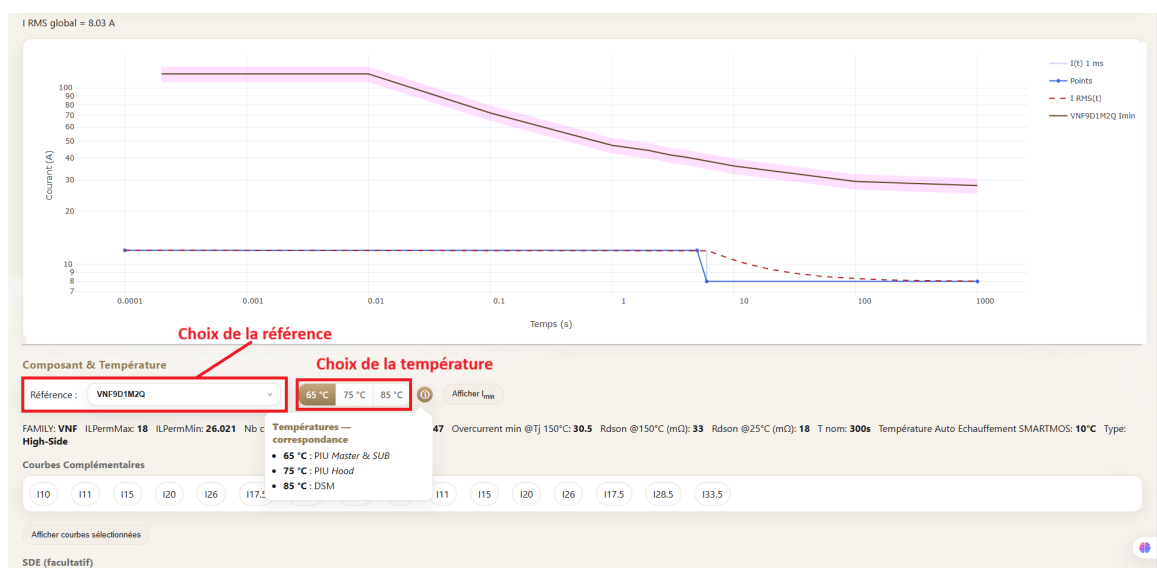
2. Saisir / importer le profil et le tracer

Dans « *Profil $I(t)$ & $I_{\text{RMS}}(t)$* », on ajoute/supprime des lignes (*+ ligne / - ligne*), on peut *Choisir un CSV* puis *Importer*, et on clique *Tracer*. Le graphe se met à jour et l' I_{RMS} global apparaît sous la table. Voir la Figure 28 (déjà présentée).

FIGURE 30 – Segments *Mode & Type*.

3. Sélectionner le composant et la température

Dans « *Composant & Température* », on choisit une **référence**, puis la **température** via le sélecteur **65 / 75 / 85 °C**. Le bouton **①** rappelle la correspondance PIU (65 °C : Master/Sub, 75 °C : Hood, 85 °C : DSM). Un clic sur *Afficher I_{min}* superpose la courbe; la carte *propriétés* liste famille, # channels, RDS(on), overcurrent, type, etc. La section *Courbes complémentaires* permet d'ajouter C1...C6 ou I_{nom} (Voir Figure 31).

FIGURE 31 – Carte composant & sélecteur **65/75/85 °C**.

4. Visualiser la superposition sur un seul graphe

Le graphe PLOTLY central affiche **profil** (et I_{RMS}), **température** choisie et **courbes composants** (BTG/VNF/SW), avec zoom/pan/hover et légende cliquable (C1...C6). Voir la Figure 28 (overlay profil + couloir $\pm 10\%$ + courbes).

5. (Option) Charger un SDE

Dans « *SDE (facultatif)* », on *Choisit le SDE* puis *Charge*. Un **badge** confirme l'état; la vue peut alors se restreindre aux **références réellement présentes** (IDs visibles côté classements)(Voir Figure 32).

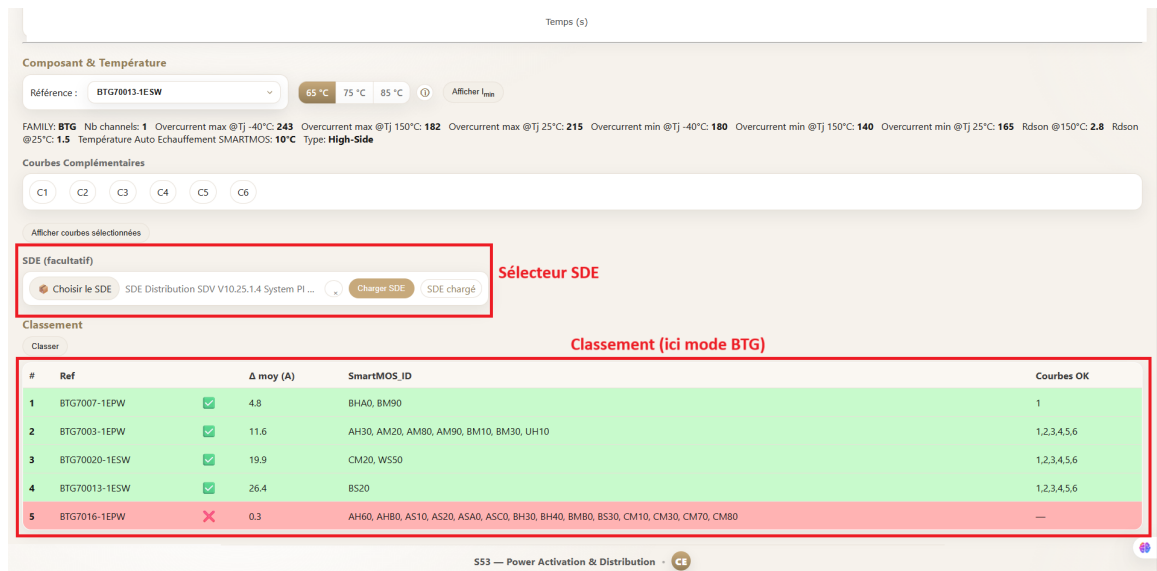


FIGURE 32 – Sélecteur SDE et badge d'état.

6. Lancer le classement et lire le Top-N

Dans « Classement », on choisit l'onglet (BTG / VNF / AUTRES (SW) / CLASSEMENT GLOBAL) et on clique *Classer*. Le **Top-N** s'affiche : # / Ref / Δ moy (A) / Courbes OK (C1...C6) avec statut (✓/⚠/✗)(Voir Figure 32).

5.8.6 Livrables et formats

Le livrable principal est le **graphe unique « overlay »**. C'est lui qui fait foi dans le dossier : il montre le **profil de courant** comparé aux **courbes composant** sur une même figure. Le **classement** reste une **aide à la décision**, utile à l'écran, mais **n'est pas un livrable**.

Overlay à fournir (Voir Figure 33)

- **Contenu attendu :**
 - **Profil $I(t)$** (ligne continue) + **points saisis/importés**.
 - $I_{RMS}(t)$ (ligne pointillée) pour lecture de l'effort énergétique.
 - **Couloir $\pm 10\%$** autour du profil (bande magenta).
 - **Courbe matérielle I_{min}** du composant sélectionné.
 - **Courbes software C1...C6** superposées, **colorées** selon le statut :
 - Vert* : courbe **respecte** le profil dans le couloir sur tout l'intervalle.
 - Rouge* : courbe **hors** couloir ou **passé sous** le profil à un moment.
 - **Légende explicite** (références, C1...C6, température 65/75/85 °C).
- **Format & nommage recommandés** : `ASCS_overlay_<ref|global>_<65|75|85>C_<YYYYMMDD-HHMM>.csv`

Éléments complémentaires (facultatifs mais conseillés pour la traçabilité)

- **Profil brut (CSV)** : points (t, I) utilisés pour le tracé.
`ASCS_profil_<YYYYMMDD-HHMM>.csv`
- **Table « état des courbes » (CSV/Excel)** : pour chaque C1...C6, **statut** (OK/NOK) et **marge minimale** avec l'instant critique t_{min} .
`ASCS_etat_courbes_<ref>_<temp>.xlsx`

En résumé : le **seul livrable requis** est l'**image overlay** (profil + I_{RMS} + couloir $\pm 10\%$ + I_{min} + C1...C6 en vert/rouge). Les fichiers CSV/Excel sont optionnels pour la **preuve de calcul** et la **reproductibilité**.

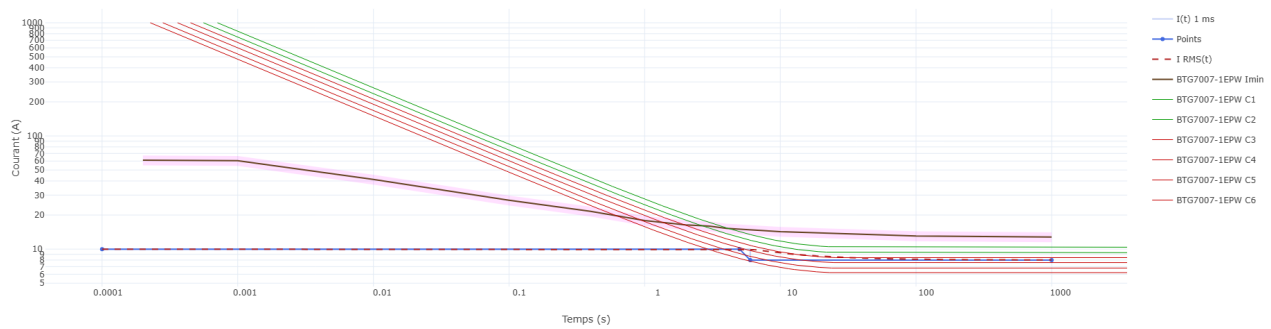


FIGURE 33 – Overlay — Profil + I_{RMS} + couloir $\pm 10\%$ + courbes composant (I_{min} et C1...C6).

5.8.7 Apports, plus-value et ROI

Avant l'ASCS, dimensionner un SmartMOS signifiait jongler entre plusieurs graphiques et fichiers lents, comparer les composants **un par un**, et recommencer à la moindre modification. L'équipe a exprimé le besoin d'un **comparateur unifié et fiable**. En **deux semaines** de développement, une application locale **Flask** a été livrée : elle **concentre la connaissance métier** (profils, températures PIU, bibliothèques BTG/VNF/SW, SDE) et la met à disposition dans une interface unique et réactive. La valeur perçue est telle que l'outil fait l'objet d'une **déclaration de brevet** où je suis cité.

Critère	Avant (outil Excel multi-graphiques)	Après (ASCS — Flask + Plotly)
Visualisation	3-4 graphes séparés (profil, BTG, VNF, SW)	Overlay unique interactif
Comparaison	Affichage un par un des composants	Superposition de plusieurs références
Temps de réponse	Lenteur, ouvertures aléatoires	Réactif , calculs Python + cache
Connaissance requise	Forte (références, onglets, astuces)	Guidé (sélecteurs, vue unifiée)
Température	Gestion manuelle	Sélecteur 65/75/85 °C
Données SDE	Hors outil	Filtre Disponibles SDE (+ IDs/PIU)
Traçabilité	Captures dispersées	PNG overlay + CSV/Excel normalisés
Robustesse	Variable selon poste	Local-only , erreurs gérées, logs
Maintenance/évolutivité	Difficile (VBA, fichiers lourds)	Modules Python + end-points JSON

TABLE 20 – Avant / Après : impact fonctionnel et technique de l'ASCS

Gain de temps

On note T_{avant} le temps par profil avec l'ancien outil, $T_{après}$ celui avec l'ASCS, $\Delta T = T_{avant} - T_{après}$ le gain par profil, P le nombre de profils d'une campagne, et T_{dev} l'effort initial

($\approx 70\text{--}80$ h).

Cas conservateur :

$$\begin{aligned} T_{\text{avant}} &\approx 55 \text{ min}, & T_{\text{après}} &\approx 3,5 \text{ min} \\ \Rightarrow \Delta T &\approx 51,5 \text{ min (0,86 h)}. \end{aligned} \quad (19)$$

Gain de campagne :

$$\text{Gain}(P) = P \times \Delta T. \quad (20)$$

Exemples : $P=10 \Rightarrow \sim 8,6$ h ; $P=50 \Rightarrow \sim 43$ h ; $P=100 \Rightarrow \sim 86$ h.

Point mort (amortissement) :

$$P_{\text{break-even}} \approx \frac{T_{\text{dev}}}{\Delta T}. \quad (21)$$

$$T_{\text{dev}} = 75 \text{ h} \Rightarrow P_{\text{break-even}} \approx 87 \text{ profils.} \quad (22)$$

Au-delà, tout est **gain net** ; à l'échelle d'un projet (campagnes multiples, variantes), l'économie devient **structurelle**.

Bénéfices qualitatifs

- **Décision cohérente et objectivée** : mêmes règles, **statut visuel** immédiat (couloir $\pm 10\%$, vert/rouge).
- **Onboarding rapide** : un nouvel arrivant est opérant en < 1 h (profil $\rightarrow$ overlay).
- **Réduction du risque** : évite le **sur-dimensionnement** (coût BOM) et le **sous-dimensionnement** (incidents/rétests).
- **Capitalisation de la connaissance** : bibliothèques, températures PIU, conventions SDE et métriques sont **encapsulées** et **reproductibles**.
- **Industrialisation** : architecture **données** $\rightarrow$ **services** $\rightarrow$ **interface**, prête à s'intégrer aux autres outils (SDE Check, CCVC) et à évoluer.

En bref, l'ASCS apporte un **levier immédiat de productivité** et une **hausse de qualité** mesurable, tout en pérennisant la **connaissance métier** — d'où son intérêt stratégique et sa **brevabilité**.

5.8.8 Perspectives

Nous avons engagé, en parallèle de l'ASCS, un travail sur une **IA de suggestion** destinée à accélérer la recherche du bon composant, sans remplacer les règles métiers ni la vérification graphique. L'idée est d'exploiter les **SDE** comme base d'apprentissage, via les couples « paramètres électriques du profil » $\leftrightarrow$ « référence SmartMOS retenue ». Les entrées sont uniquement **électriques** (5 à 10 variables typiques : **IN**, **TN**, **IP/IPIC**, **TP/TPIC**...), et la cible est la **référence** (BTG, VNF ou autre SW). Le corpus reste modeste (**400–500 exemples**), mais suffisant pour une première assistance crédible.

Nous avons retenu **Random Forest** et **XGBoost**, adaptés aux petits jeux de données, robustes aux non-linéarités et offrant des **importances de variables** utiles. L'entraînement repose sur une **cross-validation stratifiée** avec mélange aléatoire et pondération éventuelle des classes. Les métriques sont orientées usage : **Top-1**, **Top-3** et **macro-F1**. Pour interpréter une prédiction, nous utilisons les importances et, si besoin, des valeurs **SHAP**.

La philosophie est **assistive** : l'IA *suggère* mais ne *décide* pas. Le flux reste **hybride** : l'ASCS vérifie la faisabilité (overlay, tolérance $\pm 10\%$), l'IA propose un **Top- k** restreint au SDE, et l'ingénieur tranche selon le contexte. L'outil matérialise cela par un onglet dédié affichant

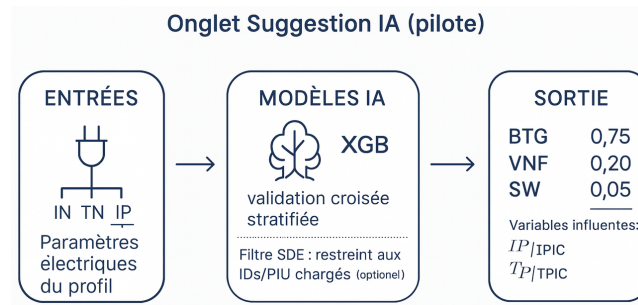


FIGURE 34 – Onglet *Suggestion IA (pilote)* : paramètres électriques → modèles (RF/XGB, CV stratifiée, filtre SDE) → sortie probabilisée et variables influentes.

Top- k , probabilités et 2–3 variables clés. Chaque essai est **journalisé** pour nourrir une boucle d'amélioration continue.

Les **limites** sont connues : volume restreint, biais possibles, dérive avec de nouveaux profils. Les **garde-fous** — validation croisée, modèles sobres, filtres SDE, primauté des règles déterministes — garantissent une aide pragmatique visant à **réduire le temps de recherche** en mettant rapidement en avant 2–3 références plausibles, sans jamais déposséder l'ingénieur de sa décision.

6 Résultats et impact

6.1 Résultats techniques

À l'issue du stage, un **ensemble d'outils cohérents** couvre toute la chaîne : préparer, dimensionner, vérifier et livrer. Les livrables sont **standardisés** (Excel, XML, PNG/CSV), l'exécution reste **locale** et l'accès est facilité par l'interface commune **ENSOFT**. Chaque outil est autonome mais interopérable, réduisant fortement la charge manuelle et homogénéisant les résultats.

Outil	Fonction principale	Livrable	Intégration
Wire Protection Calib	Vérification croisée SDE/APSW	Excel	ENSOFT
PowerSW (Leoni)	Extraction paramètres de calibration	Excel / CSV	ENSOFT
ScreenBuilder	Génération automatique d'écrans DDT	XML	ENSOFT
ENSOFT	Interface centralisée pour les modules	UI locale	—
Recorder (auto)	Génération automatique Recorder	Fichiers projet	ENSOFT
SDE Check	Contrôles SDE référentiels	Excel	Autonome / ENSOFT
CCVC	Vérification AUTOSAR (CDFX/XML)	Excel	Autonome
ASCS	Dimensionnement SmartMOS (overlay, exports)	PNG / CSV / Excel	Autonome

TABLE 21 – Synthèse des outils développés et intégration

Synthèse. L'ensemble forme une boîte à outils robuste : **contrôles fiabilisés, livrables homogènes, décisions accélérées**. ENSOFT assure une entrée unique et une diffusion simplifiée.

6.2 Bénéfices concrets pour l'équipe

Les outils livrés (Wire Protection Calib, PowerSW, ScreenBuilder, ENSOFT, Recorder auto, SDE Check, CCVC, ASCS) transforment des vérifications et productions **manuelles, hétérogènes et lentes** en un **flux outillé, standardisé et traçable**. Résultat : moins d'erreurs, des décisions plus rapides et des livrables immédiatement exploitables.

Activité clé	Avant	Après	Impact concret
Dimensionnement Smart-MOS (ASCS)	Multi-graphiques, comparaison à la main	Overlay unique + exports	45 à 57 min/-profil ; décision plus cohérente ; réduction sur/sous-dimensionnement
Vérif. calibrations AUTOSAR (CCVC)	Lecture CDFX manuelle par PIU	Rapport PIU + ACTIVATION + SUMMARY	Heures gagnées par campagne ; priorisation immédiate via KPI
Cohérence SDE (SDE Check)	Contrôles dispersés, recoupes manuelles	Rapport consolidé, règles explicites	Détection précoce des incohérences ; moins de retours tardifs
Wire Protection Calib	Aller-retour SDE/APSW, recopie	Croisement automatisé	Suppression des erreurs de saisie ; gain de temps au tri
PowerSW (Leoni)	Paramètres épars	Extraction/structuration	Préparation propre pour les autres modules
ScreenBuilder (DDT)	Création manuelle d'écrans	Génération XML	Secondes au lieu d'heures ; uniformité garantie
Recorder auto	Montage manuel	Génération automatique	Production instantanée , sans risque de mise en forme
ENSOFT (hub)	Lancements épars	Point d'entrée unique	Adoption facilitée, accès centralisé aux livrables

TABLE 22 – Avant / Après : effets concrets des outils livrés sur le flux d'ingénierie

Effets transverses observés

- **Traçabilité et audit** : horodatage, versions de sources, formats normalisés (PNG/CSV/Excel/XML) ⇒ revues plus rapides, passation facilitée.
- **Onboarding** : un nouvel arrivant peut analyser un profil et produire un overlay en < 1 h (ASCS), et parcourir un rapport **en autonomie** (CCVC/SDE Check).
- **Réduction des retours/itérations** : les écarts sont identifiés **tôt**, avant intégration ou essais, limitant les cycles correctifs.
- **Cohérence documentaire** : mêmes conventions (couleurs, pivots, colonnes) ⇒ lecture commune pour architectes, fournisseurs et qualité.
- **Autonomie locale** : exécution **offline**, sans dépendance réseau ; moins de friction IT et de variabilité poste-à-poste.
- **Capacité d'industrialisation** : modules réutilisables, interfaces stables, exports prêts à s'insérer dans des revues ou pipelines internes.

Exemple parlant (ASCS). Une campagne de **20 profils** représente ~ 15 à ~ 19 h économisées

rien que sur le dimensionnement, auxquelles s'ajoutent les gains de production (ScreenBuilder/-Recorder) et de contrôle (CCVC/SDE Check).

6.3 Motivations personnelles

6.3.1 Retours positifs des responsables d'équipe

Tout au long du stage, mes livrables ont été salués pour leur **rapidité d'exécution**, leur **qualité** et ma **capacité à prendre du recul** dans un contexte technique exigeant. Les responsables ont particulièrement apprécié ma **force de proposition** et ma **maturité** dans la conduite et la présentation des outils.

Sur **ScreenBuilder** (écrans DDT), l'**automatisation complète** a été identifiée comme un véritable **accélérateur** : plus de saisie manuelle, un rendu uniforme et des délais fortement réduits. Les retours de l'équipe ont été très positifs, soulignant un **gain de temps énorme** et une adoption immédiate.

Sur **CCVC**, en fin de stage, l'outil a permis de **mettre en évidence de nombreux checks Creta** et de **détecter des écarts de calibrations fournisseur** jugés critiques. **Vincent Grecco** m'a explicitement **remercié** pour ma **réactivité** et la pertinence de la réponse à son besoin.

Mon **tuteur, Baptiste Romieu**, avec qui j'ai le plus échangé, a exprimé sa **satisfaction** de m'avoir à ses côtés : la collaboration a été fluide, à la fois **professionnelle et humaine**, et a **accéléré** les avancées. Cette dynamique s'est étendue à l'ensemble de l'équipe ; je me suis **intégré rapidement**, notamment avec **Karim Senoun, Thibaut Cochet** et d'autres collègues, qui ont **reconnu ma valeur** et facilité les validations.

Sylvain Henry (responsable de stage / CUET) a, de son côté, souligné ma **vitesse de livraison**, ma **rigueur de code** et ma **capacité à proposer** des améliorations concrètes sans attendre un cadrage exhaustif. **Lors de réunions (notamment devant Séverine Rausz / CDS Architecture Transversale)**, mes présentations claires des objectifs, des choix techniques et des impacts (gains de temps, standardisation) ont été relevées comme un signe de maturité et d'aisance. Les retours ont été élogieux et constants jusqu'à la fin du stage.

6.3.2 Outils réellement utilisés par les collègues

Ce stage n'a pas été « un stage pour faire un stage » : il a été **utile** et **opérationnel** pour l'équipe. Les outils livrés ont été **vraiment utilisés** au quotidien — pour dimensionner, vérifier, générer et partager des livrables. On s'en est servi en réunions techniques (overlays ASCS), en préparation de livrables (écrans DDT avec ScreenBuilder), et en contrôle croisé (rapports CCVC / SDE Check) avant échanges interne-fournisseur.

Parce qu'ils étaient utilisés, j'ai reçu des **retours terrain** en continu et j'ai publié des **correctifs** tout au long du stage : ajustements d'exports, colonnes complémentaires, messages d'erreur plus explicites, petits gains d'ergonomie. L'usage réel a aussi fait émerger des **automatismes supplémentaires** auxquels on ne pensait pas au départ ; je les ai intégrés au fil de l'eau pour coller au besoin concret des architectes.

Pour éliminer la friction d'adoption, j'ai **packagé des exécutables (.exe)** quand c'était pertinent (double-clic, pas d'environnement à installer) et **rendu le code accessible** à tous sur **SharePoint** et **GitLab**. L'objectif : que chacun puisse **lancer, relire, adapter** ou rejouer

les outils **facilement et rapidement**, sans dépendre de moi.

En bref : mes livrables ont été **pris en main, améliorés grâce aux retours** et **déployés** de façon à rester utilisables après mon départ. C'est ce qui fait la différence entre une preuve de concept et un **outillage réellement intégré** dans les pratiques de l'équipe.

6.3.3 Projet intégré au flux de production FlexEvan

Les outils développés n'ont pas vécu « à côté du projet » : ils ont été **branchés sur le flux de production FlexEvan** et utilisés là où ils créent le plus de valeur — au moment de décider, de vérifier et de livrer.

Concrètement, le **parcours outillé** suivi par l'équipe est devenu le suivant :

1. **Préparer & dimensionner avec ASCS** : overlay (profil + couloir $\pm 10\%$ + courbes) présenté en revue pour orienter le choix de SmartMOS sur la base des **profils réels** et des **températures PIU**.
2. **Vérifier & assainir avec SDE Check puis CCVC** : contrôle systématique des SDE et des calibrations AUTOSAR/Creta, **détection des écarts fournisseur** avant envoi. Les **retours rapides** effectués durant le stage ont été décisifs.
Piste CI pour CCVC : le dépôt **GitLab** permet d'envisager une intégration **CI**. L'objectif est de faire tourner CCVC en pipeline et de produire automatiquement le rapport à chaque mise à jour. Pour un fonctionnement robuste en CI, il est **prévu** de faire évoluer les documents d'entrée vers des formats **XML/JSON** (au lieu d'Excel) afin de garantir une ingestion fiable et non interactive sur les *runners*.
3. **Générer & livrer avec ScreenBuilder (DDT) et les Recorders** : **production automatique et uniforme** des écrans, prêts à intégrer dans les packages techniques.

Cette intégration a été facilitée par **ENSOFT** (point d'entrée unique), par le **packaging en exécutables** pour un usage immédiat, et par la **mise à disposition du code** sur **SharePoint** et **GitLab** pour que chacun puisse relancer, adapter ou rejouer les outils sans friction. Les corrections et ajustements réalisés **au fil des retours terrain** ont permis d'aligner exactement le comportement des scripts sur les besoins du projet.

Au final, les modules livrés sont **réellement dans la boucle FlexEvan** : ils **accélèrent les revues**, **réduisent les retours** liés aux incohérences SDE/calibrations, et **standardisent** les livrables (overlays PNG, rapports Excel, XML DDT), ce qui sécurise les échanges internes et fournisseur.

7 Bilan personnel

7.1 Compétences techniques acquises

7.1.1 Python avancé

Je sais désormais **construire des pipelines de bout en bout** : lire/écrire des classeurs **Excel multi-feuilles** (*pandas, openpyxl*), **normaliser** colonnes/unités, faire des **jointures/agrégations** (*merge, groupby, pivot*), et **exporter** des rapports prêts à l'usage (*filtres, volets figés, formats stables*). Côté **XML**, je crée/édite des arbres avec *lxml* (*namespaces, XPath*) et je génère des fichiers **déterministes** (ex. écrans DDT). Le code est **modulaire**, avec **gestion d'erreurs** claire, **logs** horodatés et **packaging en venv** ou **exécutable Windows**.

7.1.2 Architecture d'outils

Je conçois des applications selon une architecture **données** → **services** → **interface** : ingestion/validation des sources, **services Python** exposés en JSON [3], **interface** indépendante. J'emploie des **caches mémoire** invalidés au changement des fichiers pour garder une UI réactive, et je gère proprement les **formats d'entrée/sortie** (CSV/Excel/XML/PNG) pour assurer l'interopérabilité et la stabilité des livrables.

7.1.3 Web & visualisation

Je développe des applis **Flask** (routes **REST/JSON**, templates) et je rends les résultats via **Plotly** sous forme de **graphes interactifs** (overlay, zoom, hover, légende). Je sais concevoir une **UI/UX minimaliste orientée productivité** : sélecteurs explicites, actions en un clic, messages d'erreur utiles, sans dépendances lourdes côté poste.

7.1.4 Data engineering « métier »

Je **normalise des référentiels** (SDE, APSW, bibliothèques **BTG/VNF/SW**) : harmonisation de clés, unités, nomenclatures. Je mets en place des **pipelines d'ingestion** robustes avec **contrôles de qualité** (présence, types, bornes, cohérences) et je garantis l'**idempotence** : mêmes entrées → mêmes sorties, avec traçabilité complète pour rejouabilité.

7.1.5 Qualité logicielle

Je travaille avec **GitLab/SharePoint** (versioning, revues), des **conventions** de code/doc, et une **documentation utilisateur** concise. J'intègre des **tests ciblés** sur les cas critiques, des **validations d'entrée** systématiques et des marqueurs de **reproductibilité** (horodatage, versions de sources), afin d'obtenir des livrables stables et auditable.

7.1.6 Data science (pilote)

Je sais prototyper une **classification** supervisée avec **Random Forest** et **XGBoost**, en **cross-validation stratifiée**, avec des métriques adaptées (Top-1 / Top-3, macro-F1). Je produis des éléments d'**interprétabilité** (*feature importance*, aperçu **SHAP**) pour expliquer les suggestions — dans une logique **d'aide à la décision**, pas de remplacement des règles métier.

7.2 Compétences humaines et organisationnelles

7.2.1 Communication technique

J'ai appris à **vulgariser** des sujets complexes (AUTOSAR, SDE, dimensionnement Smart-MOS) pour des **responsables** comme pour des **architectes**. Mes **présentations structurées** (contexte → objectifs → choix techniques → impacts mesurés) ont facilité les arbitrages : démonstrations d'ASCS (overlay, tolérance $\pm 10\%$) ou de CCVC (rapports PIU/ACTIVATION/SUMMARY) avec un discours clair sur les **gains de temps** et la **standardisation** des livrables.

7.2.2 Travail en équipe

En **coopération étroite** avec mon tuteur et les architectes, j'ai instauré un **cycle court** : recueil de besoins, livraison rapide, **retours terrain**, correctifs. Cette boucle m'a permis d'intégrer des demandes concrètes (colonnes/exports, ergonomie) et d'ancrer les outils dans

l'usage quotidien. L'écoute active et la **reformulation** ont été clés pour éviter les malentendus et **accélérer** l'adoption.

7.2.3 Autonomie & initiative

J'ai pris l'habitude d'être **force de proposition** (automatisations supplémentaires, packaging .exe, mise à dispo SharePoint/GitLab) et de **prioriser** en environnement **multi-projets** (ASCS, CCVC, ScreenBuilder, SDE Check). Planification, jalons courts, messages d'état explicites : j'ai su **tenir le rythme** sans sacrifier la qualité ni la traçabilité.

7.2.4 Leadership & pédagogie

J'ai mené des **démos** ciblées et des **formations express** pour transmettre l'outillage (lancer, filtrer, exporter, interpréter). J'ai accompagné la **conduite du changement** : documentation simple, UI cohérente, réponses rapides aux retours. Résultat : une **adoption réelle** des outils et une **montée en compétence** des collègues, capable de perdurer après mon départ.

7.3 Méthodes de travail et bonnes pratiques

7.3.1 Cycle de développement court

J'ai fonctionné en **itérations rapides** : *POC* → *beta* → *correctifs* → *release*. Chaque boucle comporte une **démo courte**, un **retour terrain** immédiat, puis un *fix* livré sous forme de **version taguée**. Les lots sont **petits et livrables**, avec critères d'acceptation simples (entrée, sortie, temps d'exécution, message d'erreur attendu). Quand c'était utile, j'ai activé des **switchs** (options UI/CLI) pour livrer sans bloquer l'existant et permettre un **rollback** sans friction.

7.3.2 Traçabilité & auditabilité

Tous les exports intègrent **horodatage**, **version des sources** (référentiels, bibliothèques), **contexte d'exécution** (température, mode, filtres) et un **ID de build**. Les pipelines réalisent des **contrôles d'intégrité** (présence de colonnes, types, bornes) et conservent des **logs horodatés**. Les entrées critiques (Excel/SDE) sont **signées** par hachage léger (ou *mtime* + taille) pour rattacher un rapport à un jeu de données précis. Les releases sont **taguées** sur GitLab et le binaire associé est déposé sur SharePoint.

7.3.3 Documentation

Chaque outil a un **guide d'usage** (prérequis, étapes, exemples), un **README** concis (objectif, I/O, temps typique), et une **FAQ** des erreurs fréquentes avec messages **explicites** et actions de remédiation. J'ai ajouté des **checklists** (avant run, avant livraison) et un **CHANGELOG** synthétique à chaque release pour faciliter la passation et le support.

7.3.4 Standardisation

J'ai aligné les **formats de sortie** (PNG/CSV/Excel/XML) et les **taxonomies** (noms de colonnes, codes couleur, libellés d'erreurs) pour garantir une lecture homogène entre outils. Les conventions couvrent : **nommage de fichiers** (<outil>_<contexte>_<date>.ext), **unités** (s, A), **mises en forme** (filtres, volets figés), **schémas JSON/CSV** [3] stables et **règles UI** (overlay unique, légende cliquable). Résultat : des livrables **interchangeables**, **reproductibles** et immédiatement exploitables par les collègues comme par les autres scripts.

7.4 Difficultés rencontrées et solutions

7.4.1 Hétérogénéité des sources

Problème. Entrées issues de référentiels et formats variés (SDE, APSW, bibliothèques BTG/VNF/SW) avec **colonnes hétérogènes**, **unités différentes**, noms changeants, valeurs manquantes et encodages/virgules décimales.

Solutions mises en place.

- **Schéma canonique** (taxonomie unique) et **adapters** par source : renommage de colonnes, mapping d'unités, normalisation des clés.
- **Contrôles d'intégrité** systématiques (présence de colonnes, types, bornes, duplicats) avec messages d'erreur **explicites**.
- **Idempotence & déterminisme** : mêmes entrées $\Rightarrow$ mêmes sorties (tri stable, formats verrouillés), horodatage + contexte dans chaque export.
- **Tolérance « propre »** : gestion des NA/vides documentée (remplissage ou exclusion selon règle), logs horodatés pour audit.

7.4.2 Contrainte de temps

Problème. Plusieurs besoins en parallèle (ASCS, CCVC, ScreenBuilder, SDE Check) avec délais serrés.

Solutions mises en place.

- **Découpage en incréments livrables** (*POC* $\rightarrow$ *beta* $\rightarrow$ *correctifs* $\rightarrow$ *release*), démos courtes et feedback immédiat.
- **Priorisation stricte** (MoSCoW) : d'abord le **chemin critique** métier, puis le confort.
- **Réutilisation** de briques communes (I/O Excel/XML, validation, export) pour accélérer.
- **Documentation concise** (README + modes d'emploi) pour limiter la charge de support.

7.4.3 Robustesse poste-à-poste

Problème. Variabilité des postes (versions d'Office/Python, droits, performances) et dépendances instables.

Solutions mises en place.

- **Exécutables** Windows et **venv** prêts à l'emploi ; dépendances **pinnées** (*requirements*), moteurs stables (*openpyxl*).
- **Exécution locale/offline** (pas de réseau requis), gestion d'erreurs lisibles, **logs** activés par défaut.
- **Tests de non-régression** sur fichiers représentatifs, **pré-checks** (taille, *mtime*, schéma) avant run.
- **Formats de sortie homogènes** (PNG/CSV/Excel/XML) et conventions partagées pour une lecture identique quel que soit le poste.

7.5 Ouverture professionnelle

7.5.1 Périmètres ciblés

Je souhaite poursuivre dans l'automobile et plus largement dans l'embarqué. Au-delà de Python, j'apprécie d'intervenir sur toute la pile technique : développement en **C** pour le bas niveau, utilisation de **ROS** et des systèmes d'aide à la conduite, applications **Android**

Automotive en Java/Kotlin, et, lorsque c'est pertinent, exploitation de la **donnée** pour l'ingestion, le contrôle et la visualisation. Ce stage m'a confirmé que je suis **pertinent et rapide** sur ces sujets et que je peux créer de la valeur au croisement du **métier électrique** et du **logiciel**.

7.5.2 Évolutions envisagées

Je me projette dans des fonctions où je **conçois** et fais tourner des briques utiles au véhicule : ingénierie **logicielle embarquée** orientée intégration système et tests, **outillage & validation** pour la production et l'automatisation, **data engineering** industriel lorsque la donnée apporte un levier opérationnel. Mon apport principal est la capacité à **faire le pont** entre **développement embarqué** et **outillage orienté données**, afin de livrer vite des solutions **fiables** et réellement **exploitables** par les équipes.

7.5.3 Environnement souhaité

L'idéal réunit une **forte proximité produit**, des échanges réguliers avec des **architectes** et des **experts**, des **cycles courts** avec des itérations fréquentes, et une culture qui valorise la **clarté technique** et la **mesure de l'impact**. Je reste **ouvert** sur le plan technologique et organisationnel, prêt à rejoindre des équipes diverses et, si l'opportunité se présente, à **travailler à l'international**. L'essentiel est de contribuer dans un cadre où **rigueur**, **vitesse d'exécution** et **esprit d'équipe** permettent de produire des résultats concrets.

7.6 Axes d'amélioration plan d'apprentissage

7.6.1 Industrialisation (CI/CD & packaging)

J'ai **touché** à la CI sans pouvoir l'industrialiser complètement. Objectif : **renforcer** ce volet indispensable en contexte industriel, rendre les outils **exécutables en pipeline**, produire des **artefacts** reproductibles et faire évoluer les I/O critiques vers des formats **XML/JSON** versionnés.

Actions à court/moyen terme

- Chaînes **GitLab CI** et **Jenkins** : lint → tests → build → artefacts.
- Stabiliser une **CLI** (arguments, codes retour, logs structurés).
- Releases versionnées (wheel Python + exécutable Windows) avec **notes de version**.
- Standardiser les **formats d'entrée/sortie** en **XML/JSON** et fixer les schémas.

7.6.2 Tests & monitoring

Les outils ont bien tenu, mais **quelques erreurs ont échappé** avant correction. Objectif : **prévenir** et **détecter** plus tôt les dérives.

Actions à court/moyen terme

- Systématiser **tests unitaires & d'intégration** avec *golden files*.
- Ajouter des **validations de schéma** (XML/JSON) dans les pipelines.
- Instrumenter des **métriques** (durée, taux d'erreurs, couverture de règles).
- **Alertes CI** si seuils dépassés (temps, échecs, dérive de couverture).

7.6.3 IA & recommandations (Data)

Deux **certifications** (FreeCodeCamp, IBM) m'ont donné des bases solides et un **regard critique** : l'IA *n'est pas magique* et ne remplace pas les règles métier. Objectif : **approfondir** pour des suggestions vraiment utiles à l'ingénieur.

Actions à court/moyen terme

- Enrichir le **feature engineering** sur *signaux électriques*.
- Mettre en place une **validation croisée** rigoureuse avec **calibration** des probabilités (Top-1/Top-3).
- Intégrer l'**explicabilité** (importances, SHAP) dans une vue *Suggestion IA*.
- Définir des **protocoles d'acceptation** (critères d'usage, seuils) avant mise à dispo.

7.6.4 Domaine (AUTOSAR & flux SDE → CDFX → DDT)

Je veux consolider la **maîtrise du domaine** pour mieux cadrer les règles d'outillage/validation et gagner en autonomie.

Actions à court/moyen terme

- Approfondir **AUTOSAR** (ARXML/CDFX) et conventions internes.
- Rejouer un **cas fil rouge** complet : SDE → contrôles → CDFX → écrans DDT.
- Mise à jour sur **sécurité/ASIL** & exigences documentaires associées.

7.6.5 Stack technique élargie

Au-delà de Python, rester **polyglotte embarqué** (C/ROS/Android Automotive en Java/-Kotlin) tout en poursuivant la **data** lorsque le levier est concret.

Actions à court/moyen terme

- Refactor d'un mini-outil **Flask/Plotly** (overlay + export) en code « clean ».
- Prototype **FastAPI** pour comparer performances & testabilité.
- **Routine d'apprentissage** continue : C (bas niveau), ROS, Android Automotive (Java/-Kotlin).

8 Conclusion

En une phrase : j'ai conçu et livré un **écosystème d'outils** (ASCS, CCVC, SDE Check, ScreenBuilder, ENSOFT) qui a transformé des tâches manuelles en **processus standardisés et rapides**. Concrètement, la décision technique est **objectivée** (overlays, KPI), les livrables sont **reproductibles**, et l'intégration au flux **FlexEvan** est effective. La suite logique, côté personnel, est d'**industrialiser** mes pratiques (CI/CD, tests/monitoring) et de poursuivre dans l'**embarqué** au plus près du produit, avec ouverture data/IA.

8.1 Compétences techniques acquises

Le stage a abouti à un **ensemble cohérent d'outils réellement utilisés**, avec des **livrables standardisés et reproductibles**. Dans l'ordre du rapport, **Wire Protection Calib** a d'abord posé la base de la vérification croisée entre SDE et APSW sur la wire protection : les contrôles sont **automatisés**, les écarts clairement qualifiés, et le **rapport Excel** est immédiatement exploitable (filtres, couleurs, volets figés) pour enclencher les corrections. **PowerSW** a ensuite structuré les paramètres de calibration fournisseur en **tables normalisées** ; cette consolidation réduit les ambiguïtés, facilite la relecture par les architectes et alimente les étapes de vérification ultérieures. **ScreenBuilder** a permis la **génération intégrale** des écrans DDT à partir du SDE, supprimant la saisie manuelle, garantissant l'uniformité visuelle et rendant la remise à jour quasi instantanée en cas d'évolution des données. **ENSOFT** a fédéré ces modules dans une **interface locale unique**, ce qui a fluidifié les lancements, homogénéisé l'expérience utilisateur et accéléré l'appropriation par l'équipe. Dans la continuité, la **génération des écrans Recorder** a automatisé la production de ces vues, en assurant une cohérence de forme et de contenu sans retouches manuelles.

SDE Check a joué un rôle d'assainissement en **consolidant les contrôles de cohérence** du SDE ; les écarts sont détectés en amont, documentés et priorisés pour éviter des retours tardifs. **CCVC** a sécurisé la fin de chaîne en vérifiant les **calibrations AUTOSAR et Creta** par PIU et par fonction ; il a révélé des écarts fournisseurs critiques et a structuré leur traitement via des **feuilles PIU**, une feuille **ACTIVATION** et une **SUMMARY** avec indicateurs. Une **piste d'intégration continue** sur GitLab est engagée pour CCVC, avec passage prévu des entrées en **XML/JSON** afin de permettre un fonctionnement sans interaction et des rapports générés automatiquement. Enfin, **ASCS** a transformé le dimensionnement des SmartMOS grâce à un **overlay unique** combinant profil, couloir de tolérance et courbes, une vue **Disponibles SDE** et des exports prêts à l'usage ; le gain de l'ordre de **45–57 min** par profil par rapport à l'outil historique a été constaté, et l'approche a été jugée suffisamment différenciante pour alimenter une **déclaration de brevet**.

Au-delà des démonstrations, ces outils ont été **intégrés au flux de production FlexEvan**, utilisés en revue comme en préparation de livrables, puis ajustés au fil des retours. Les sorties sont **homogènes, traçables, versionnées et partageables** via SharePoint et GitLab, avec un **packaging prêt à l'emploi** qui garantit la continuité après le stage. *En synthèse*, le périmètre a été couvert de bout en bout, les **gains de temps** sont mesurables, la **qualité documentaire** s'est renforcée, la **traçabilité** est au niveau attendu et les bases de l'**industrialisation** sont posées.

8.2 Compétences humaines et organisationnelles

Sur le plan **technique**, j'ai consolidé un socle complet allant de **Python avancé** à l'**architecture d'outils**. Je sais aujourd'hui bâtir des **pipelines robustes** sur Excel et XML,

structurer un projet en **modules** avec **gestion des erreurs** et des **logs**, et livrer des **exécutables** prêts à l'emploi. Côté interfaces, au-delà de **Flask/Plotly**, j'ai également développé des écrans en **Custom Tkinter** pour proposer une **UI locale légère** aux collègues non développeurs (sélecteurs de fichiers, exécution guidée, logs visibles). J'ai renforcé mon **data engineering** métier en normalisant SDE/APSW/bibliothèques composants, tout en garantissant **idempotence** et **traçabilité**. J'ai posé les premières briques d'**industrialisation** (CLI stable, versioning propre, exposition à la CI) et cadré plus rigoureusement **tests** et **monitoring**. Enfin, j'ai ouvert un chantier **IA** raisonnable (RF/XGBoost, interprétabilité) comme **aide** à la décision.

Ces acquis se sont traduits par des **résultats concrets** au service du projet : **génération intégrale et reproductible** d'écrans DDT, **contrôles SDE/AUTOSAR** plus fiables via des rapports structurés et actionnables, **dimensionnement SmartMOS** objectivé par un overlay unique avec des **gains de temps mesurables** à l'échelle d'une campagne. L'ensemble est **intégré au flux FlexEvan** et outillé pour **perdurer après mon départ**.

Sur le plan **personnel**, j'ai confirmé ma capacité à **communiquer clairement** des sujets techniques complexes, à présenter **objectifs/choix/impacts** devant des responsables, et à m'intégrer rapidement dans une équipe experte. J'ai aussi enrichi ma **culture terrain** : interventions **sur banc** et sur **PIE** pour des vérifications concrètes, **manipulation de multimètres** et d'éléments électroniques, mesures et contrôles simples pour confronter les résultats logiciels à la **réalité physique**. Ces expériences ont affûté mon **sens pratique**, ma **rigueur de validation** et mon aisance à faire le lien **logiciel** ↔ **matériel**. Au final, j'ai non seulement accru mes **compétences techniques**, mais aussi affirmé une **posture d'ingénieur outillage** capable de transformer un besoin métier en solution **fiable, adoptée et utile**, tout en restant proche des **contraintes du terrain**.

8.3 Méthodes de travail et bonnes pratiques

Ce stage m'a conforté dans une conviction simple : je veux continuer à travailler **au croisement de l'automobile et du logiciel**, au plus près du produit et des essais. C'est là que je me sens utile, que ma **vitesse d'exécution** et ma **force de proposition** font gagner du temps aux équipes, et que je peux transformer un besoin flou en outil concret. J'ai aussi mesuré ma **valeur ajoutée** : je sais livrer vite, proprement, et faire le lien logiciel ↔ matériel sans perdre de vue la décision d'ingénierie derrière l'outil.

Je reste lucide sur le contexte français, notamment pour les profils **junior**. Rien ne garantit une embauche immédiate à l'issue du stage, y compris chez Renault. C'est pourquoi je me projette **à l'international** : une piste existe à **Taïwan** sur des sujets d'**électronique et de diagnostic**, et je m'y intéresse parce qu'elle renforcerait mon socle matériel. **Mais je reste ouvert à toutes les autres opportunités**, en France comme ailleurs, dès lors qu'elles me permettent de contribuer à des systèmes embarqués exigeants et à des chaînes de validation/outillage qui apportent un impact réel.

Mon cap est clair : **me professionnaliser encore** pour devenir un ingénieur complet et fiable. Concrètement, cela veut dire approfondir l'**électronique et le diagnostic** terrain (mesures, bancs, PIE), poursuivre l'**embarqué** (C, ROS, Android Automotive) et consolider les pratiques **qualité/industrialisation** qui font la différence dans la durée (tests, monitoring, CI/CD, maîtrise d'AUTOSAR et du flux SDE → CDFX → DDT). Je continuerai aussi à utiliser la **data/IA** quand elle apporte un levier concret, avec la même exigence d'explicabilité et de bon sens.

Je ne m'étendrai pas davantage : l'essentiel, pour conclure, est que ce stage m'a permis de **prendre la mesure de ma pertinence** et de mon **utilité** dans un environnement industriel

complexe. Je sais la direction que je veux suivre, je sais ce que je dois renforcer, et je suis prêt à poursuivre cette trajectoire — ici ou ailleurs — pour **devenir un ingénieur pleinement compétent**, au service de produits robustes et de décisions techniques mieux outillées.

Fiche tuteur entreprise et Fiche d'évaluation étudiant

Cette fiche est à remplir par le tuteur entreprise et doit être retournée avant le 25 aout 2025 à ecole-entreprise@seatech.fr

Organisme d'accueil : AMPERE SOFTWARE TECHNOLOGY
 Etudiant : Enzo CHERIF
 Sujet du stage : Développeur d'outils d'automatisation

Evaluation : développement des compétences			
La grille d'évaluation est en cohérence avec le référentiel de compétences de la formation. Les 6 compétences se développent au cours des trois années de la formation. Merci d'évaluer le stagiaire selon les différents critères.			
Références pour la cotation			
(A) Excellent	Au-delà des attentes (au-delà des étudiants du même niveau)		
(B) Adéquat	Répond aux attentes (dans la norme des étudiants du même niveau)		
(C) Insuffisant	Ne répond pas aux attentes (en deçà des étudiants du même niveau)		
(N) Non applicable	Critère non évaluable dans le contexte du stage		
Compétence	Critère	Eval	Commentaire
Concevoir des solutions d'ingénierie	Analyser le besoin	A	Enzo a su très rapidement identifier les activités pouvant être automatiser, en interagissant avec tous les membres de l'équipe.
	Répondre au besoin	A	Tous les outils développés par Enzo sont actuellement utilisés dans le processus interne de l'équipe. Ce qui démontre la justesse des outils développés.
	Utiliser les outils appropriés	A	Enzo a mobilisé Python et ses bibliothèques (Pandas, OpenPyXL, lxml, Tkinter, Flask...) ainsi que des outils internes (DDT, SDE) pour automatiser efficacement des tâches complexes.
	Documenter ses choix et ses sources	A	Rapport hebdomadaire des activités réaliser toute au long du stage, ainsi que commentaires approfondis de tous les outils développés (code, user guide, ...)
Mettre en œuvre des solutions	Analyser et améliorer une solution existante	A	Avant le stage d'Enzo, la solution de vérification du SDE était une macro Excel et des check manuel. Enzo a su améliorer la solution existante en générant un outil user friendly, robuste et fiable.
	Proposer une solution nouvelle	A	Enzo a proposé la génération de rapport détaillé, qui permettent d'aider à la traçabilité des modifications. Enzo a également, pour le développement de l'ASCS, proposé l'utilisation d'une interface Web. Solution qui s'avère être très efficace.
	Utiliser les principes de l'amélioration continue	A	Pour la génération des écrans DTT, Enzo a intégré dans son outil les retours de tous les membres de l'équipe, afin d'arriver à un résultat au-delà de la demande initiale.
	Rédiger un document scientifique et technique d'appui	A	Le rapport de stage et les user guides rédigés par Enzo, sont d'une grande clarté, et aide au quotidien les membres de l'équipe à utiliser les outils développés.
Développer une	Réaliser une veille technologique / un état de l'art	A	Enzo a su définir les meilleurs outils et bibliothèques dans le cadre de ses activités. Il a également pris le temps de se documenter sur les sujets d'IA, et de faire part de ses recherches aux membres de l'équipe intéressés.
	Formuler des hypothèses	A	Enzo a su proposer des fonctionnalités et des solutions techniques qu'il a présenté à l'équipe afin de définir l'hypothèse de référence sur laquelle s'appuyer pour la suite de ces activités.

démarche R&D	Proposer une démarche expérimentale, un protocole ou un modèle	A	Enzo a défini l'architecture logiciel des outils développés selon un modèle qui permet leurs intégrations dans une interface commune. Ce qui est très pratique pour leurs utilisations.
	Adopter une démarche d'innovation	A	Enzo a su adopter une démarche innovante, notamment dans le développement de l'outil ASCS, en proposant des solutions alternatives améliorant la fiabilité et l'ergonomie des outils existants.

Piloter des projets d'ingénierie	S'insérer dans ou conduire une ou plusieurs étapes d'un projet	A	Enzo a développé 8 outils, qui sont tous utilisés dans le processus interne de génération des livrables de l'équipe.
	Prendre en compte la gestion globale des organisations ou les règles de fonctionnement, économiques ou juridiques	A	Enzo a veillé à respecter les contraintes du processus AMEPERE afin que les livrables de l'équipe s'appuyant sur les outils développés soient conformes aux standards internes (qualité, cohérence, ...) Enzo a également veillé à ce que son travail soit exploitables par les autres équipes.
	Utiliser les outils de gestion de projet et outils collaboratifs	A	Enzo a organisé son espace de travail dans les outils de l'équipe afin de donner de la visibilité et accès à ces avancés à tous les membres de l'équipe.
	Identifier ou mobiliser les ressources appropriées	A	Enzo a su identifier et solliciter les membres de l'équipe (architectes, PFS, CUET) nécessaires à la réussite de ses activités.
Encadrer une équipe	S'insérer et collaborer	A	Tout au long de son stage, Enzo a su collaborer avec les membres de l'équipe afin d'assurer la qualité de ses activités.
	Travailler en équipe pluridisciplinaire et/ou internationale	A	Enzo s'est intégré et à réaliser des outils utilisés par l'équipe dans un contexte international (France, Tunisie, Inde et Allemagne)
	Assurer une responsabilité d'animation	A	Enzo a su présenter son travail avec brio à de multiple reprises (à l'équipe, et même au service).
	Communiquer (écrit et oral) de manière adaptée	A	Le rapport de stage et les user guides rédigés par Enzo, sont d'une grande clarté, et aide au quotidien les membres de l'équipe à utiliser les outils développés. Le rapport de stage et les user guides rédigés par Enzo, sont d'une grande clarté, et aide au quotidien les membres de l'équipe à utiliser les outils développés. De plus, Enzo a su adapter sa communication à son interlocuteur.
Agir en professionnel responsable	Prendre en compte les enjeux éthiques et sociétaux (RSE, DD, RGPD, ...)	A	Enzo a respecté les règles de confidentialité et de protection des données internes définie par AMPERE, en veillant à l'usage responsable des informations manipulées.
	Assumer la responsabilité de ses actes et décision	A	Enzo a travaillé en autonomie sur mes développements et assumé mes choix techniques, tout en sollicitant des validations lorsque nécessaire.
	Porter un regard critique sur le sens de l'activité conduite	A	Enzo a analysé l'impact des outils développés sur les processus de l'équipe, en évaluant leurs apports en termes de gain de temps, fiabilité et pérennité.
	Être dans une dynamique d'apprentissage	A	Enzo a constamment cherché à progresser en approfondissant mes compétences en Python, en automatisation et en interface utilisateur, tout en intégrant les retours de l'équipe.

Appréciation globale du stagiaire et recommandations en vue d'occuper un poste d'ingénieur (il est recommandé de discuter de cette évaluation avec le stagiaire)

Pour résumer ce stage, un mot me vient à l'esprit : **la collaboration**. Durant ces six mois passés avec Enzo, j'ai autant **enseigné** que **appris**, sur le plan professionnel mais aussi personnel.

Enzo m'a démontré quotidiennement, par sa rigueur et son sérieux, par ses connaissances et ses compétences, par sa capacité à communiquer — que ce soit techniquement ou en vulgarisant — par son esprit critique et sa force de proposition, qu'il possède tous les atouts pour réussir une grande carrière d'ingénieur.

Enzo, encore une fois, **toutes mes félicitations**. Je te souhaite **plein de réussite** dans tes expériences futures, qui seront, j'en suis convaincu, **des réussites**. Au plaisir de te recroiser, que ce soit dans la vie professionnelle ou personnelle.

Le 21/08/2025, à Guyancourt,



AMPERE

Baptiste ROMIEU
AMS-AAT2
PFS Power Distribution
baptiste.romieu@ampere.cars



Fiche d'évaluation étudiant

Ce document est à insérer dans le rapport. Le rapport est à fournir pour le 25 aout dernier délai.

Organisme d'accueil :

Ampere Software

Technology

Sujet du stage :

Développer des outils

d'automatisation en

Pyhton

Evaluation qualitative du stage	I	S	B	E	N	Commentaires
Qualité de l'entreprise						L'entreprise m'a fourni tous les moyens nécessaires (ordinateur, équipements, etc.) pour réaliser ma mission. Elle connaissait le groupe INP, même si j'ai dû préciser mon école. Le stage a été très enrichissant techniquement et humainement, avec un environnement pleinement orienté vers une carrière d'ingénieur grâce à la collaboration avec des architectes et pilotes fonction système.
L'entreprise offre-t-elle un contexte propice à une carrière d'ingénieur ?				X		
L'entreprise connaissait elle l'école (accueil de stagiaire, embauche, relations autres...) ?			X			
L'entreprise a-t-elle mis à votre dispositions les moyens nécessaires pour réaliser votre mission (documents, éléments d'information, matériels) ?				X		
Qualité de la missions						Les missions confiées correspondaient parfaitement à ma formation et étaient en adéquation avec les objectifs fixés dès le départ. Elles m'ont permis de mettre en pratique mes compétences tout en les approfondissant dans un cadre concret et professionnalisant.
Vos missions étaient-elles en rapport avec votre formation ?				X		
Les missions effectuées étaient-elles bien celles définies au départ ?				X		
Qualité de l'encadrement						Mon tuteur, Baptiste Romieu, a été exceptionnel tout au long du stage. Il a pris le temps de m'intégrer à l'équipe, de m'expliquer la structure et de m'accompagner avec beaucoup de disponibilité et de conseils, ce qui a grandement contribué à la réussite et à la richesse de mon expérience.
Votre tuteur organisme d'accueil a-t-il pris le temps de vous présenter le fonctionnement de la structure et l'équipe ?				X		
Votre tuteur organisme d'accueil vous a-t-il aidé et conseillé quand cela était nécessaire ?				X		
Votre enseignant référent vous a-t-il aidé et conseillé lorsque cela était nécessaire ?				X		

Explication des cotations	
I	(I) Insuffisant
S	(S) Suffisant
B	(B) Bien
E	(E) Excellent
N	(N) Non applicable

NB : Dans le but d'alléger la lecture du document, le genre masculin est utilisé sans discrimination pour le genre masculin et féminin.

Fiche d'évaluation étudiant

Autoévaluation : développement des compétences et trajectoire professionnelle En prenant un peu de recul sur votre activité durant le stage pensez-vous avoir travaillé / développé certaines des compétences du référentiel de la formation Seatech ? Lesquelles ? Pourquoi et comment ? D'autres compétences ?		
Compétence	Critère	Commentaire
Concevoir des solutions d'ingénierie	Analyser le besoin	J'ai analysé les besoins des architectes et pilotes système en termes d'automatisation (vérification SDE, génération d'écrans DDT, calibration, etc.) afin de proposer des solutions adaptées.
	Répondre au besoin	J'ai développé plusieurs outils (SDE Check, ScreenBuilder, CCVC, ASCS...) qui répondent directement aux besoins métiers, en réduisant le temps de traitement et en sécurisant les données.
	Utiliser les outils appropriés	J'ai mobilisé Python et ses bibliothèques (Pandas, OpenPyXL, lxml, Tkinter, Flask...) ainsi que des outils internes (DDT, SDE) pour automatiser efficacement des tâches complexes.
	Documenter ses choix et ses sources	J'ai systématiquement documenté mes scripts, mes choix techniques et mes méthodologies dans des supports clairs (rapports, guides utilisateurs), permettant la réutilisation par l'équipe.
Mettre en œuvre des solutions	Analyser et améliorer une solution existante	J'ai repris certains fichiers Excel et outils internes existants pour en améliorer la robustesse et l'ergonomie, en optimisant les traitements et en corrigeant des incohérences.
	Proposer une solution nouvelle	J'ai conçu de nouveaux outils d'automatisation (ASCS, CCVC, ScreenBuilder...) permettant de répondre à des besoins non couverts et d'apporter des fonctionnalités inédites.
	Utiliser les principes de l'amélioration continue	J'ai intégré les retours des utilisateurs (architectes et pilotes système) pour améliorer progressivement les scripts, ajouter des fonctionnalités et rendre les interfaces plus intuitives.
	Rédiger un document scientifique et technique d'appui	J'ai produit des documentations techniques et guides utilisateurs détaillés, facilitant la prise en main et la réutilisation des outils développés par l'équipe.
Développer une démarche R&D	Réaliser une veille technologique / un état de l'art	J'ai étudié les outils déjà utilisés en interne ainsi que des bibliothèques Python existantes (Pandas, OpenPyXL, lxml, Flask...) afin de sélectionner les plus adaptés aux besoins métiers. J'ai également mené une veille sur l'intelligence artificielle.
	Formuler des hypothèses	J'ai formulé différentes approches techniques (scripts modulaires, interfaces Tkinter vs Flask, intégration de règles métiers dans le SDE) et testé leur faisabilité avant validation.
	Proposer une démarche expérimentale, un protocole ou un modèle	J'ai défini des protocoles de vérification (comparaison SDE, règles ASIL, validation des calibrations) permettant d'assurer la fiabilité et la reproductibilité des résultats.
	Adopter une démarche d'innovation	J'ai conçu des outils totalement nouveaux (ASCS pour les protections câblage, ScreenBuilder pour générer automatiquement les écrans DDT) apportant un gain de temps significatif et une meilleure fiabilité par rapport aux méthodes manuelles.

Fiche d'évaluation étudiant

Piloter des projets d'ingénierie	S'insérer dans ou conduire une ou plusieurs étapes d'un projet	J'ai participé activement aux différentes étapes de développement d'outils automatisés, depuis l'analyse des besoins jusqu'à leur mise en service au sein de l'équipe.
	Prendre en compte la gestion globale des organisations ou les règles de fonctionnement, économiques ou juridiques	J'ai veillé à respecter les contraintes du processus Renault (qualité des SDE, règles ASIL, validation des calibrations) afin que mes livrables soient conformes aux standards internes et exploitables par les autres équipes.
	Utiliser les outils de gestion de projet et outils collaboratifs	J'ai utilisé Teams, Outlook, SharePoint et GitLab interne pour partager les scripts, documentations et suivre l'avancement des tâches en coordination avec les architectes système.
	Identifier ou mobiliser les ressources appropriées	J'ai su identifier et solliciter les bonnes personnes (architectes, PFS, CUET) ainsi que les documents et bases de données internes nécessaires à la réussite de mes missions.
Encadrer une équipe	S'insérer et collaborer	Je me suis pleinement intégré dans l'équipe et j'ai collaboré quotidiennement avec les architectes et pilotes fonction système.
	Assurer une responsabilité d'animation	J'ai présenté mes travaux lors de réunions d'équipe et proposé des démonstrations d'outils, contribuant ainsi à l'animation technique.
	Travailler en équipe pluridisciplinaire et/ou internationale	J'ai travaillé dans un environnement pluridisciplinaire, en interaction avec différents métiers (architectes, PFS, CUET) et dans un contexte international.
	Communiquer (écrit et oral) de manière adaptée	J'ai rédigé des documentations techniques claires et présenté mes résultats à l'oral de façon adaptée selon l'audience (collègues, direction).
Agir en professionnel responsable	Prendre en compte les enjeux éthiques et sociétaux (RSE, DD, RGPD, ...)	J'ai respecté les règles de confidentialité et de protection des données internes, en veillant à l'usage responsable des informations manipulées.
	Assumer la responsabilité de ses actes et décision	J'ai travaillé en autonomie sur mes développements et assumé mes choix techniques, tout en sollicitant des validations lorsque nécessaire.
	Porter un regard critique sur le sens de l'activité conduite	J'ai analysé l'impact des outils développés sur les processus de l'équipe, en évaluant leurs apports en termes de gain de temps, fiabilité et pérennité.
	Être dans une dynamique d'apprentissage	J'ai constamment cherché à progresser en approfondissant mes compétences en Python, en automatisation et en interface utilisateur, tout en intégrant les retours de l'équipe.
Trajectoire professionnelle A la suite de votre stage, avez-vous confirmé ou affiné votre projet professionnel d'être ingénieur (métier plus précis, secteur, contexte ou type d'entreprise, ...) ? Si oui, quelles actions pensez-vous devoir entreprendre pour y arriver (renforcer certaines connaissances, développer certaines compétences, lesquelles)? À la suite de ce stage, mon projet professionnel d'ingénieur s'est confirmé et affiné vers le domaine de l'automatisation, du développement logiciel et de l'intégration de solutions techniques dans l'industrie automobile. Cette expérience m'a conforté dans l'idée de travailler dans un environnement pluridisciplinaire et innovant, en lien direct avec des architectes système et des projets concrets. Pour y parvenir, je souhaite renforcer mes compétences en développement logiciel (Python, IA appliquée à l'ingénierie), approfondir mes connaissances en systèmes embarqués et poursuivre mon apprentissage de méthodes de travail collaboratives (CI/CD, gestion de projet). Date et signature de l'étudiant : 17/08/2025 ENZO CHERIF		

Glossaire

ASCS — Automatic Suitable Curve Selection Outil qui classe/suggère les courbes de protection i²t (BTG/VNF/Software) à partir d'un profil courant-temps, avec exports (PNG/CSV/Excel) et visualisation web.

ASIL — Automotive Safety Integrity Level Niveaux de sûreté (ISO 26262) de QM à D ; guident exigences et contrôles de cohérence des données.

APSW Dictionnaire/fichier fournisseur (p. ex. Leoni) contenant indices, valeurs et résolutions de calibration, utilisé pour croiser/valider le SDE.

AUTOSAR Standard d'architecture logicielle automobile (fichiers CDFX/XML) structurant composants, calibrations et interfaces ECU.

BFT Élément d'architecture présent dans les schémas de distribution (terme interne de nommage/répartition).

BTG Famille de courbes de protection câblage (tenue courant-temps) utilisée pour le dimensionnement et la vérification des lignes/SmartMOS.

CAN / CAN-FD Bus de communication véhicule (FD = débit/charge utile étendus) ; états et diagnostics affichés dans les écrans DDT.

CCVC — Creta Cross Verification Check Vérification croisée CDFX/AUTOSAR ↔ SDE ↔ dictionnaires (APSW, variantes...), avec rapport par PIU.

CDFX Format AUTOSAR (XML) de configurations/calibrations logicielles consommées par les ECU.

DDT / DDT2000 Outil de diagnostic ; les écrans (Distribution, Shutdown, OEM, Recorder) sont générés en XML.

DOORS Gestionnaire d'exigences (requirements) ; permet des regroupements (p. ex. par PIU) pour structurer écrans/contrôles.

DSM Module/boîtier d'architecture cité dans le SDE (p. ex. Trailer/Cabin).

ECU — Electronic Control Unit Calculateur embarqué consommant calibrations et pilotant actionneurs/SmartMOS.

ENSOFT (EnSoft) Hub/interface locale (.exe) rassemblant plusieurs outils (ScreenBuilder, Wire Protection Calib, PowerSW) via une UI unifiée.

FlexEVan Programme utilitaire 100 % électrique (plateforme SDV) visant évolutivité logicielle, diagnostics avancés et connectivité.

High-Side / H-Bridge Topologies de commutation de puissance : high-side commute le + batterie ; pont en H permet pilotage bidirectionnel/inversion.

HV / LV Réseaux Haute Tension / Basse Tension du véhicule, distingués dans les schémas SDE.

i²t Intégrale courant².temps : indicateur d'échauffement câble pour valider la tenue d'un profil de courant.

IRMS Courant efficace extrait du profil $I(t)$, comparé aux seuils/courbes de protection.

JSON / CSV / XML Formats d'échange : API/exports (JSON, CSV) et écrans DDT (XML).

Leoni (dictionnaire PowerSW) Table Excel de calibrations (index, valeurs, résolutions) consultée par le module PowerSW.

LOB — Life On Board État/commande d'alimentation embarquée (modes étendus) affiché dans certains écrans.

MBSE — Model-Based Systems Engineering Champs « Distribution Function / Energy Flow » décrivant fonctions et flux ; vérifiés dans SDE Check.

OEM (écrans OEM) Écrans statiques de supervision (modes/états, VPM, boutons), indépendants du SDE ; utiles pour banc et debug.

OTA — Over-The-Air Mises à jour logicielles/paramètres à distance, facilitées par l'architecture SDV.

P83 Code/variante projet servant de clé de filtrage pour croisements SDE, CDFX et dictionnaires.

PDC+ Élément d'architecture de distribution mentionné dans le SDE (terme interne).

PIU — Power Interface Unit Nœud d'interface de puissance (Hood/Master/Sub) regroupant SmartMOS par zone/fonction.

PIU Hood / Master / Sub Répartition par zones véhicule (avant, central, arrière/confort) pour organiser écrans, rapports et diagnostics.

Plotly Bibliothèque JavaScript de visualisation interactive utilisée côté front (ASCS) pour le graphe unique.

PO — Power On Commande/bouton d'alimentation affiché dans les écrans (p. ex. PO / LOB Extended).

PowerSW (module) Outil d'extraction/consultation des données de calibration par SmartMOS/SWC/adresse (à partir du dictionnaire Leoni).

Recorder (écrans) Écrans DDT générés automatiquement pour observer/enregistrer des signaux et états clés.

RDS(on) Résistance à l'état passant d'un MOSFET ; impacte pertes et échauffement d'un

SmartMOS.

S53 — Power Activation & Distribution Périmètre « réveil / endormissement / distribution et activation d'énergie » du véhicule.

SDE — Schéma de Distribution Électrique Référence unique (schéma + Excel) listant IDs, courants IN/IP, BTG/VNF, ASIL, PIU, etc.

SDE Check Contrôle qualité/cohérence du SDE (règles IN/IP, ASIL, System Controlling, MBSE, exclusivités...) avec rapport Excel multi-onglets.

SFA Code système affiché dans les tableaux d'écrans DDT pour identifier rapidement une fonction/liens diagnostic.

SDV — Software-Defined Vehicle Architecture centrée logiciel : fonctions évolutives, connectivité et mises à jour facilitées (FlexEVan).

SmartMOS Interrupteur de puissance intelligent (MOSFET + logique) remplaçant le fusible, avec mesure/diagnostic/protection intégrés.

SSOT — Single Source of Truth Principe de « source unique » : le fichier Excel SDE fait foi pour conception, validation et diagnostic.

SWC — Software Component Composant logiciel AUTOSAR référencé pour lier calibrations, diagnostics et écrans.

System Controlling SmartMOS Champ SDE désignant le système qui pilote un SmartMOS ; soumis à règles de forme et de contenu.

V2G — Vehicle-to-Grid Échanges d'énergie véhicule ↔ réseau (charge/décharge) rendus possibles par l'architecture électrique/logicielle.

Vehicle / END Marqueurs texte dans l'Excel SDE pour délimiter automatiquement le périmètre des données à traiter.

VNF Famille de courbes de protection câblage (alternative/complément des BTG) utilisée pour la validation.

VPM — Vehicle Power Mode États globaux de puissance (Power OFF/ON, LoB...) affichés et pilotés dans les écrans.

Wake-up / Shut-down Stratégies de réveil/endormissement ; structurent les écrans « Shutdown » (seuils, regroupements par PIU).

Wire Protection Calib (module) Croisement SDE ↔ courbes BTG/VNF pour détecter sous/sur-dimensionnements et générer un rapport.

Bibliographie

Références

- [1] ISO, *Road vehicles — Functional safety (ISO 26262-1 :2018)*, 2018. Disponible : <https://www.iso.org/standard/68383.html>. Consulté le 18 août 2025.
- [2] ISO, *Controller Area Network (CAN) — Part 1 : Data link layer and physical coding sublayer (ISO 11898-1 :2024)*, 2024. Disponible : <https://www.iso.org/standard/86384.html>. Consulté le 18 août 2025.
- [3] T. Bray, *The JavaScript Object Notation (JSON) Data Interchange Format (RFC 8259)*, IETF, 2017. DOI : 10.17487/RFC8259. <https://datatracker.ietf.org/doc/html/rfc8259>. Consulté le 18 août 2025.
- [4] W3C, *Extensible Markup Language (XML) 1.0 (Fifth Edition)*, 2008. <https://www.w3.org/TR/xml/>. Consulté le 18 août 2025.
- [5] IANA, *Media Type application/CDFX+XML*, 2014. <https://www.iana.org/assignments/media-types/application/CDFX%2BXML>. Consulté le 18 août 2025.
- [6] ASAM e.V., *ASAM CDF — Calibration Data Format*, 2025. <https://www.asam.net/standards/detail/cdf/>. Consulté le 18 août 2025.
- [7] CAN in Automation (CiA), *CAN FD — The Basic Idea*, 2024. <https://www.can-cia.org/can-knowledge/can-fd-the-basic-idea>. Consulté le 18 août 2025.
- [8] ISO, *Road vehicles — Vehicle to grid communication interface — Part 20 (ISO 15118-20 :2022)*, 2022. <https://www.iso.org/standard/77845.html>. Consulté le 18 août 2025.
- [9] ISO, *Road vehicles — Vehicle to grid communication interface — Part 1 (ISO 15118-1 :2019)*, 2019. <https://www.iso.org/standard/69113.html>. Consulté le 18 août 2025.
- [10] IEC, *Electric vehicle conductive charging system — Part 1 : General requirements (IEC 61851-1 :2017)*, 2017. <https://webstore.iec.ch/en/publication/33644>. Consulté le 18 août 2025.
- [11] Plotly Technologies Inc., *Plotly Python Graphing Library*, 2025. <https://plotly.com/python/>. Consulté le 18 août 2025.
- [12] Pallets Projects, *Welcome to Flask — Documentation (3.1.x)*, 2025. <https://flask.palletsprojects.com/>. Consulté le 18 août 2025.
- [13] PyData Development Team, *pandas 2.3.1 Documentation*, 2025. <https://pandas.pydata.org/docs/>. Consulté le 18 août 2025.
- [14] NumPy Developers, *NumPy 2.3 Manual*, 2025. <https://numpy.org/doc/>. Consulté le 18 août 2025.
- [15] lxml Project, *lxml — Processing XML and HTML with Python*, 2025. <https://lxml.de/>. Consulté le 18 août 2025.
- [16] OpenPyXL Project, *openpyxl — Read/Write Excel 2010 xlsx/xlsm/xltx/xltm*, 2025. <https://openpyxl.readthedocs.io/>. Consulté le 18 août 2025.
- [17] PyInstaller Development Team, *PyInstaller Manual*, 2025. <https://www.pyinstaller.org/>. Consulté le 18 août 2025.
- [18] Python Software Foundation, *tkinter — Python interface to Tcl/Tk*, 2025. <https://docs.python.org/3/library/tkinter.html>. Consulté le 18 août 2025.
- [19] INCOSE, *Systems Engineering Handbook, Fifth Edition — Overview*, 2023. [https://www.incose.org/home/2023/10/11/default-calendar/incose-los-angeles-incose-systems-engineering-handbook-fifth-edition-\(overview\)](https://www.incose.org/home/2023/10/11/default-calendar/incose-los-angeles-incose-systems-engineering-handbook-fifth-edition-(overview)). Consulté le 18 août 2025.

- [20] IBM, *Engineering Requirements Management DOORS Next — Product Page*, 2025. <https://www.ibm.com/products/requirements-management>. Consulté le 18 août 2025.
- [21] Infineon Technologies AG, *Power PROFET™+ — Automotive Smart High-Side Switches*, 2025. <https://www.infineon.com/cms/en/product/power/smart-power-switches/high-side-switches/power-profet-plus-12v-automotive-smart-high-side-switch/>. Consulté le 18 août 2025.
- [22] STMicroelectronics, *High-Side Switches — VIPower Family*, 2025. <https://www.st.com/en/automotive-analog-and-power/high-side-switches.html>. Consulté le 18 août 2025.
- [23] STMicroelectronics, *UM3352 — VIPower M0-9 Standard High-Side Drivers (User Manual)*, Rev. 1, 2024. https://www.st.com/resource/en/user_manual/um3352-vipower-m09-standard-high-side-drivers-stmicroelectronics.pdf. Consulté le 18 août 2025.
- [24] Littelfuse, *Fuseology — Selection Guide and Technical Information*, 2021. <https://www.littelfuse.com/assetdocs/littelfuse-fuseology>. Consulté le 18 août 2025.
- [25] Renault Group, *Ampere (Entity) — Renault Group*, 2025. <https://www.renaultgroup.com/en/group/brands-entities/ampere-entity/>. Consulté le 18 août 2025.
- [26] Renault Group, *Renault Group and Volvo Group complete creation of Flexis SAS* (communiqué de presse), 22 mars 2024. <https://media.renaultgroup.com/?p=234108>. Consulté le 18 août 2025.
- [27] Renault Group Magazine, *All about Software Defined Vehicle (SDV)*, 2023. <https://www.renaultgroup.com/en/magazine/our-group-news/all-about-software-defined-vehicle/>. Consulté le 18 août 2025.
- [28] Eclipse Foundation, *Eclipse Software Defined Vehicle Working Group*, 2025. <https://projects.eclipse.org/working-group/eclipse-software-defined-vehicle>. Consulté le 18 août 2025.